

2.1 Tipuri fundamentale de date și constante

Tipurile de date fundamentale în limbajul C cu mărimea lor în baiți sunt prezentate în tabelul 2.1.

Tip de dată	void	char	int	float	double
Mărimea în baiți	0	1	2 / 4	4	8

Tab. 2.1 Tipurile de dată fundamentale în limbajul C cu mărimea lor în baiți

În implementările pe 32 biți ale limbajului, tipul întreg este memorat pe 4 baiți.

Tipurile fundamentale pot fi însoțite și de modificatori de tip de dată. Aceștia împreună cu semnificația lor se prezintă în tabelul 2.2.

Modificatorul	Efectul
unsigned	nu tratează bitul cel mai semnificativ ca fiind bit de semn
signed	tratează bitul cel mai semnificativ ca fiind bit de semn
long	extinde dimensiunea zonei de memorie ocupată de variabilă
short	reduce dimensiunea zonei de memorie ocupată de variabilă

Tab. 2.2 Modificatori de tip de dată

Între tipurile de bază și modificatorii de tip se pot face combinații. Dintre combinațiile posibile doar o parte sunt acceptate în limbaj. În tabelul 2.3 se prezintă combinațiile corecte.

Tipul *long double* este folosit pentru a lucra cu numere reprezentate în virgulă mobilă precizie extinsă, valabil pe calculatoarele cu coprocesor matematic.

2.1.1 Tipul caracter

Reprezentat în limbaj prin cuvântul cheie *char*, este o subtipologie a tipului întreg și suplinește întregii mici, pe 1 B (bait), cu sau fără semn; în

plus, când acești întregi sunt citați sau editați cu descriptorul de caracter* (%c), tipul *char* acoperă și mulțimea caracterelor din setul ASCII extins.

TIPURI DE DATE	LUNGIME IN BAITI	DOMENIU DE VALORI
char	1	-128 ÷ 127
signed char	1	-128 ÷ 127
unsigned char	1	0 ÷ 225
int	2 sau 4	-32768 ÷ 32767 sau -2147483648 ÷ 2147483647
signed int	2 sau 4	-32768 ÷ 32767 sau -2147483648 ÷ 2147483647
unsigned int	2 sau 4	0 ÷ 65535 sau 0 ÷ 4294967295
short int	2	-32768 ÷ 32767
signed short int	2	-32768 ÷ 32767
unsigned short int	2	0 ÷ 65535
long int	4	-2147483648 ÷ 2147483647
signed long int	4	-2147483648 ÷ 2147483647
unsigned long int	4	0 ÷ 4294967295
float	4	1.2E-38 ÷ 3.4E+38
long float	8	2.2E-308 ÷ 1.8E+308
double	8	2.2E-308 ÷ 1.8E+308
long double	10	3.4E-4932 ÷ 1.2E+4932

Tab. 2.3 Combinațiile modificador - tip de dată fundamental, admise în limbaj

Exemplu de definire a unor variabile de tip caracter:

char x, litera;
unsigned char y;
signed char z;

Interpretarea *signed* - *unsigned* este și implicită, în funcție de fixarea unei opțiuni din mediul de programare; în general, opțiunea este fixată pe *unsigned*, tipul *char* fiind frecvent folosit pentru manipularea caracterelor.

Descriptorii asociați tipului *char* sunt:

- %c pentru citirea / afișarea unui caracter (a simbolului grafic);
- %d pentru citirea / afișarea codului ASCII corespunzător unui caracter.

După cum se observă, interpretarea unei zone de 1 B, drept caracter sau întreg, este lăsată la latitudinea programatorului. Din modul de reprezentare a întregilor și a caracterelor reiese că o valoare *unsigned* între 128 și 255 poate fi interpretată ca fiind echivalentul unui caracter din zona extinsă ASCII, sau un întreg negativ, între -128 și 0.

O secvență de forma:

```
unsigned char t = 217;
printf(" %c %d ", t, t);
```

va afișa: 217

Constantele de tip caracter se reprezintă pe 1 B și conțin un caracter.

Exemplu: 'a' 'A' '\$' '?'

Datorită interpretării ca întregi, variabilele de tip *char* participă la calcule aritmetice ca întregi, rezultatul fiind supus aceleiași interpretări:

Exemplu:

```
char v='c';
printf(" %d %c", v+1, v+1);
```

va afișa: 100 d

Secvențele de evitare (de escape) conțin coduri ASCII pe un caracter, care nu au echivalent imprimabil, ci desemnează funcții de control ale editării. Semnificația lor este dată în tabelul 2.4.

constantă	semnificația
'\a'	alarmă
'\b'	BS
'\f'	FF
'\n'	NL
'\r'	CR
'\t'	HT

constantă	semnificația
'\nnn'	ASCII octal nnn
'\xdd'	ASCII hexa dd
'\v'	VT
'\'	\
'\"'	"
'\''	'
'\0'	NULL

Tab. 2.4 Secvențe de escape

Se observă că apar și secvențe numerice, în octal sau hexazecimal. De exemplu '\377' reprezintă pe un bait pe -1, iar '\0x26' reprezintă valoarea 38, pe un bait.

2.1.2 Tipul întreg

Este implementat în limbaj cu ajutorul cuvântului cheie *int* și se reprezintă în virgulă fixă, pe 2B sau 4B, în varianta pe 32 biți. Intră în combinații cu modificatorii de tip de dată, iar consecințele se pot observa din tabelul 2.3. Are asociați mai mulți descriptori de format, după cum reprezintă un întreg în baza 10 cu semn (%d), fără semn (%u), în octal (%o), în hexa (%x). La întregul pe 2B se poate adauga pentru descriptorii prezențați și caracterul l obținându-se %ld cu semnificația de întreg lung în baza 10, %lu, %lx, %lo, cu semnificații ușor de dedus.

Variabilele ce nu au specificat tipul de dată sunt implicit considerate ca fiind de tip întreg.

Exemplu:

unsigned a; // declarație echivalentă cu unsigned int a
long t; // declarație echivalentă cu long int t

Constantele întregi pot fi date în mai multe feluri:

- în octal (baza 8), formate din simbolurile 0,1,2,3,4,5,6,7, prefixate cu simbolul 0(zero);

Exemplu: 032 are valoarea zecimală 26;

- în baza 10 (zecimal), formate din simbolurile 0,1,2,3,4,5,6,7,8,9, eventual caracterul de semn, primul simbol fiind diferit de zero;

Exemplu: 32, -123

- în hexazecimal (baza 16), formate din simbolurile 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c, d, e, f, prefixate cu simbolul 0x sau 0X; simbolurile corespunzătoare lui 10, 11, 12, 13, 14, 15 pot fi scrise cu litere mari sau mici.

Exemplu: 0xa1 are valoarea zecimală 161;

0X0B are valoarea zecimală 11;

Constantele întregi se reprezintă pe 2 B sau 4 B, în funcție de mărimea lor. Dacă depășesc intervalul [-32768, 32767] se alocă automat pe 4 B. Alocarea pe 4 B poate fi forțată precizând atributul de long (L sau l) în finalul constantei.

În varianta pe 32 de biți constantele întregi se memorează numai pe 4B.

Exemplu:

322 se reprezintă pe 2B;

32768 ocupă 4B.

322L se reprezintă pe 4B;

02 se reprezintă pe 2B;

02l se reprezintă pe 4B;

0x3L se reprezintă pe 4B;

Constantele urmate de atributul U sau u (unsigned) semnifică interpretarea lor fără semn;

Exemplu: 123u ocupă 2B; 32768U ocupă doar 2B; 333uL ocupă 4B.

2.1.3 Tipul flotant

Se declară prin cuvintele cheie *float*, *long float* sau *double* și *long double*. Variabilele de acest tip se reprezintă în virgulă mobilă, pe 4B simplă precizie, 8B dublă precizie și 10B precizie extinsă, conform tabelului 2.3. Afișarea, respectiv introducerea valorilor flotante folosesc una din notațiile zecimală sau exponențială (științifică). Constantele flotante se reprezintă pe

majoritatea implementărilor în dublă precizie și urmează aceleași tipuri de notație.

Exemplu:

```
float a=12.321; // notatie zecimala
double b= 0.12E+3; // notatie exponentiala
long double c=123.56e-21 // notatie exponentiala
```

Celor două notații li se asociază pentru editarea sau citirea variabilelor descriptorii de format, **%f** pentru notația zecimală și **%e** pentru notația exponențială. Dacă se dorește forma cea mai scurtă dintre cele două notații (sub aspectul lungimii ocupate) se poate opta pentru descriptorul **%g**. Pentru *double* se adaugă caracterul **l** (**%lf**) pentru dublă precizie, iar pentru precizia extinsă se folosește caracterul **L** (**%Lf**).

2.1.4 Definirea unor tipuri speciale de constante

Alături de constantele aferente tipurilor de date fundamentale prezentate în limbaj, se mai pot utiliza:

- **constantele simbolice** ce sunt parametri înlocuiți la preprocesare cu valorile de definire.

Exemplu:

```
#define NMAX 50
#define PI 3.141592
#define BELL '\007'

.....
int x[NMAX];
.....
lungime=2*PI*raza;
.....
```

- **constantele enumerative** sunt constante simbolice puse în corespondență cu numere întregi, în secvențe controlate de utilizator.

Exemplu:

```
enum { NU, DA };
în acest caz NU reprezintă constanta enumerativă 0, DA constanta 1;
enum { LUNI=1, MARTI, MIERCURI, JOI, VINERI, SAMBATA, DUMINICA };
în acest caz LUNI desemnează valoarea 1, MARTI valoarea 2, MIERCURI
valoarea 3 ș.a.m.d.
```

Dacă primei constante nu i se dă o valoare explicită va lua implicit valoarea 0, iar celelalte se vor majora progresiv cu 1.

- **constantele șir de caractere** sunt recunoscute prin încadrarea între ghilimele a unor caractere.

Exemplu: "aceasta este o constanta de tip sir", " ", "a", "?", "\n rând nou", ""

La alocare se adaugă automat 1B peste lungimea șirului pentru a memora terminatorul de șir care este '\0'.

Dacă o constantă șir conține ghilimele interioare ele se dau ca secvențe de evitare.

Exemplu:

"supranumit \"buldozerul\""

Descriptorul de format aferent șirurilor de caractere este %s.

Un alt mod de a defini constante în limbajul C constă în folosirea modificatorului **const**. Acesta permite definirea unei variabile al cărei conținut nu se mai poate modifica pe parcursul execuției programului.

Se folosește sub forma

const tip nume = valoare

sau

tip **const** nume = valoare.

Exemplu:

const float a=3.3;

// în acest caz a devine o constantă de tip float ce are valoarea 3.3.

float const a=3.3;

// reprezintă o declarație echivalentă cu cea precedentă.

const b=7; // declară o constantă de tip întreg cu valoarea 7.

Se poate observa că dacă lipsește tipul din declarația unei constante, atunci el este implicit întreg.

2.2 Specificatorul de format

Funcțiile de intrare / ieșire care realizează și formatarea datelor efectuează această operație datorită existenței unui specificator de format. În exemplele prezentate, specificatorul a fost folosit în forma lui cea mai simplă, cu rolul de a citi și afișa diferite tipuri de dată.

Specificatorul de format este mai complex și se referă la cadraj, precizie în cazul numerelor reale, forțarea afișării semnului, chiar dacă numerele sunt pozitive, forțarea afișării prefixelor de octal și hexa etc. El este diferit în funcție de natura operației, de intrare sau de ieșire.

A. Specificatorul de format pentru ieșire (afișare) are forma:

$$\% \left[\begin{array}{c} - \\ + \\ 0 \\ \# \end{array} \left\{ \begin{array}{c} \lg_1 \\ * \end{array} \right\} \bullet \left\{ \begin{array}{c} \lg_2 \\ * \end{array} \right\} \right] \text{descriptor_de_format}$$

unde:

- [] - reprezintă parte opțională;
- { } - are sensul de a alege o opțiune din cele prezentate;
- - forțează cadrarea stânga a datei afișate; implicit ea se cadrează la dreapta;
- + - forțează afișarea semnului chiar în cazul în care variabila este pozitivă;
- # - forțează afișarea prefixelor de octal și hexa;
- 0 - forțează afișarea de zero-uri înaintea datei până la completarea lungimii câmpului de afișare; operează doar pentru tipuri integrale;
- lg₁ - reprezintă numărul de caractere al zonei în care se va face cadrulajul;
- lg₂ - reprezintă numărul de caractere ce se vor afișa în cazul șirurilor de caractere, respectiv precizia (numărul cifrelor ce succed punctul) pentru numere reale;
- * - are aceeași semnificație cu lg₁ sau lg₂ doar că valoarea este preluată din lista de variabile a funcției *printf*;
- - face parte din sintaxa formatului, fiind obligatoriu dacă apare menționată precizia;

descriptor_de_format - este descriptorul de format aferent tipului de dată ce urmează a se afișa (c,s,d,x,o,lf etc.).

Se impun unele observații:

- câmpul de definire a lungimii de afișare, lg₁ poate avea două consecințe asupra datei ce urmează a se afișa: trunchiere sau extindere. În cazul funcției de ieșire, trunchierea nu se face pentru nici un tip de dată, în schimb supradimensionarea câmpului de afișare se face pentru toate tipurile de date, adică lungimea zonei de afișare este adusă implicit, cel puțin la lungimea preciziei;

- dacă se dorește afișarea caracterului %, atunci el se va dubla %%.

Exemple:

a) Secvența:

```
int a=5671;
printf("\n %7d",a);
printf("\n %-7d",a);
```

```
printf("\n %-%10x",a);
printf("\n %010d",a);
printf("\n %2d",a);
```

afișează:

			5	6	7	1				
5	6	7	1							
0	x	1	6	2	7					
0	0	0	0	0	0	5	6	7	1	
5	6	7	1							

b) Secvența:

```
double a=5671.7892;
printf("\n %10.2lf",a);
printf("\n %-%10.3lf",a);
printf("\n %e",a);
printf("\n %2.2lf",a);
```

afișează:

			5	6	7	1	.	7	9				
5	6	7	1	.	7	8	9						
5	.	6	7	1	7	8	9	e	+	0	0	3	
5	6	7	1	.	7	9							

c) Secvența:

```
char *sir="abcd";
printf("\n %s",sir);
printf("\n %10.5s",sir);
for(int i=1;i<=4;i++) printf("\n %-%10.*s",i,sir);
```

afișează:

a	b	c	d							
					a	b	c	d		
a										
a	b									
a	b	c								
a	b	c	d							

Ca observație, secvența:

```
for(int i=1;i<=4;i++) printf("\n %-%10.*s",i,sir);
```

este echivalentă cu:

```
printf("\n %-%10.1s",sir);
printf("\n %-%10.2s",sir);
printf("\n %-%10.3s",sir);
printf("\n %-%10.4s",sir);
```

Funcția de afișare *printf* ca și cele derivate din ea (*fprintf*, *sprintf* etc.) returnează un întreg ce semnifică numărul de caractere tipărite de apelul respectiv.

B. Specificatorul de format pentru intrare (citire) are forma:

`%[*][lg]descriptor_de_format`

unde:

- * - rupe asignarea dintre lista valorilor citite și lista variabilelor citate în *scanf*, sărind peste o valoare; folosește la citirea selectivă a unor date dintr-un fișier existent;
- lg - reprezintă numărul maxim de caractere pentru data citită.

Exemple:

a) Fiind dată secvența:

```
int a;
scanf("%d",&a);
printf("%d",a);
```

dacă la citire se va introduce 4521, se va afișa doar 45.

b) În secvența:

```
int a=1,b=1;
double t=1;
scanf("%d%*d%lf",&a,&t);
printf("a=%d b=%d t=%lf",a,b,t);
```

dacă se introduce de la tastatură:

44 678 34.2

se va afișa:

a=44 b=1 t=34.2

deoarece 678 nu se asignează variabilei *b*, ci este sărit.

Separatorii recunoscuți de funcția *scanf* sunt spațiu (' '), tab ('\t') și newline ('\n').

În ce privește descriptorii, în afara celor menționați, când se introduc șiruri de caractere se pot specifica caracterele care să fie acceptate sau caracterele care să reprezinte terminatori ai operației de citire. Forma descriptorului este:

`%[lista_caractere]`

unde, *lista_caractere* reprezintă mulțimea caracterelor acceptate în variabila șir, sau:

`%[^lista_caractere]`

unde, *lista_caractere* reprezintă mulțimea caracterelor considerate a fi terminatori ai operației de citire.

Când caracterele descrise în *lista_caractere* formează un interval compact din codul ASCII, atunci ele se descriu folosind caracterele limită unite prin liniuța de despărțire (-), de exemplu A-Z, înseamnă toate caracterele literă mare.

Un exemplu sugestiv îl constituie cazul în care se dorește a se introduce numele și prenumele unei persoane.

Dacă se dă secvența:

```
char nume[100];  
scanf("%s", nume);
```

și se introduce Ioan Preda, se va stoca în *nume* doar Ioan. Dacă se rescrie apelul funcției *scanf* în forma:

```
scanf("%[A-Z a-z]", nume);
```

atunci în *nume* se va stoca Ioan Preda, deoarece spațiu prezent în descriptor nu mai este terminator, ci caracter acceptat la citire. În consecință, se vor stoca numai caracterele: litera mare, litera mică sau spațiu, restul fiind considerați terminatori.

Funcția *scanf* returnează un întreg ce semnifică numărul de valori citite și asiguate variabilelor, sau -1 când s-a tastat CTRL/Z (sfârșit de fișier).

Fiind dată secvența:

```
int a,b;  
int k=scanf("%d%d",&a,&b);
```

dacă se introduce:

- 20 5 *k* ia valoarea 2;
- 45 abc *k* ia valoarea 1;
- fdd *k* ia valoarea 0;
- CTRL/Z *k* ia valoarea -1.

2.3 Clase de memorie

Clasele de memorie sunt asociate variabilelor pentru a evidenția momentul alocării, tipul memoriei folosite, durata de viață a variabilelor, precum și domeniul lor de vizibilitate. Se disting în limbaj următoarele clase de memorie: *automatic*, *static*, *extern* și *register*.

Clasa **automatic** corespunde variabilelor comune și se definește implicit (când nu e precizată altă clasă) sau explicit prin cuvântul cheie *auto*. Variabilele aparținând acestei clase se definesc numai în cadrul unui bloc. Ele nu sunt inițializate implicit de compilator deoarece se alocă în memorie dinamică, pe parcursul execuției programului și conțin valori reziduale.

Variabilele automate sunt recunoscute doar în blocul în care au fost definite, iar la ieșirea din bloc, spațiul de memorie aferent lor se eliberează. Pot fi inițializate explicit la definire, caz în care inițializarea se va face la fiecare intrare în blocul respectiv.

O variabilă este inițializată dacă la definire i se atribuie o valoare concretă.

```
#include<stdio.h>
void main()
{
    auto a; double b=67.9;
    a=56;
    printf("\n a=%d b=%lf ", a,b);
    {
        int x=789; char y;
        y='r';
        printf("\n a=%d b=%lf ", a,b);
        printf("\n x=%d y=%c ", x,y);
    }
    printf("\n a=%d b=%lf ", a,b); // x si y invizibili aici
}
```

Programul prezentat folosește variabile din clasa de memorie automat. Se pot efectua diferite modificări astfel încât să se evidențieze toate caracteristicile acestei clase de memorie.

Clasa **static** corespunde variabilelor permanente pentru un bloc sau funcție. Ele se alocă în memorie statică și se inițializează automat la compilare cu 0, păstrându-și conținutul între reveniri succesive în blocul respectiv. Clasa *static* este implicită pentru unele categorii de date (masive cu inițializări, variabile globale, șiruri de caractere) sau explicit cerută prin cuvântul cheie *static*. Când sunt definite în afara oricărui bloc, variabilele au caracter global; definite în interiorul unui bloc, domeniul de vizibilitate se reduce la acel bloc, deși variabilele există și după ieșirea din bloc.

Programul:

```
#include<stdio.h>
void main()
{
    int x=1;
    for(int i=0; i<5; i++)
    {
        auto a=78;
        static b;
        printf("\n a=%d b=%d x=%d i=%d", a,b,x,i);
        a ++; b++; x++;
    }
}
```

urmărește prezentarea caracteristicilor variabilelor din clasa de memorie *static* prin comparație cu cele ale clasei de memorie *automatic*. El va afișa:

```
a=78 b=0 x=1 i=0
a=78 b=1 x=2 i=1
a=78 b=2 x=3 i=2
a=78 b=3 x=4 i=3
a=78 b=4 x=5 i=4
```

Se observă că variabila *automatică* la intrarea în bloc se inițializează, iar la ieșirea din bloc își pierde valoarea, în timp ce variabila *statică* se inițializează o singură dată, implicit cu 0 și își păstrează conținutul de la o secvență la alta. Variabilele din această clasă de memorie se pot inițializa și explicit, de exemplu: `static double t=5.6;` dar acest lucru se face tot o singură dată, la momentul compilării.

Clasa **extern** se definește în raport cu împărțirea unui program în mai multe fișiere și funcții, compilabile separat și permite definirea și utilizarea variabilelor globale. Se va face distincție între definirea și declararea variabilelor din această clasă. Pentru a fi văzute în alte funcții, variabilele se definesc obligatoriu, în afara oricărei funcții. Utilizarea lor înainte de definire presupune ca ele să fie în prealabil declarate prin cuvântul cheie *extern*; acesta instruește compilatorul că nu trebuie făcută alocare, identificarea efectivă a variabilei făcându-se abia la linkeditare, când sunt reunite toate fișierele.

Declararea acestor variabile se impune și în cazul în care programul este împărțit în fișiere aparținând unui proiect, dacă o variabilă externă este folosită într-un fișier și definită în altul. Se inițializează implicit de compilator cu valoarea 0 și sunt alocate în memorie statică.

Așadar, o variabilă externă apare totdeauna în două ipostaze, la definire când nu poartă specificatorul *extern* și compilatorul o alocă și la folosire, când poartă specificatorul *extern*, nu i se mai alocă spațiu, pentru că e alocată în altă parte.

Declarația *extern* pentru variabile poate apare:

- în interiorul unui bloc, caz în care ele sunt recunoscute numai acolo;
- în afara oricărei alte funcții, caz în care ele sunt recunoscute din acel loc până la sfârșitul fișierului sursă.

```
extern i;
void f() {    i++;    }
#include<stdio.h>
int i;
void main()
{
    void f();
```

```
i=23;    printf("\n i=%d ", i);
f();     printf("\n i=%d ", i);
}
```

De observat că definirea unei variabile globale poate fi realizată și prin cuvântul *static*. În acest caz, variabila este recunoscută din acel punct până la sfârșitul fișierului sursă respectiv. Aceste variabile nu pot fi declarate *extern*, deci ele nu pot fi utilizate în alte fișiere (se realizează în acest fel ascunderea unor informații globale).

Când o variabilă locală (sau parametru de intrare în funcție) coincide ca nume cu una globală, este referită implicit cea locală. Cea globală poate fi referită explicit folosind contextul program (operatorul de rezoluție "::"):

```
#include <stdio.h>
int a = 10;
void f( )
{
    int a = 1;
    printf("a local este %d", a);
    printf("\n a global este %d", ::a);
}

void main( ) {    f();    }
```

Clasa **register** forțează alocarea variabilelor în unul din registrele calculatorului. Această clasă se asociază variabilelor de tip întreg, caracter sau pointer. Deoarece numărul de registre este limitat se impune ca numărul de astfel de variabile să fie redus, altfel ele se comportă ca și variabilele automate, neputând fi alocate în registre. Se recomandă ca această clasă de memorie să fie asociată variabilelor cu o frecvență mare de utilizare în program pentru a crește viteza lui de execuție (variabile contor, variabile de indexare etc.). Adresarea regiștrilor făcându-se altfel decât pentru memoria obișnuită, nu este permisă efectuarea operației de extragere a adresei pentru astfel de variabile (operatorul &); în consecință, nici masivele nu pot avea asociată clasa *register*. Cuvântul cheie prin care se indică această clasă de memorie este *register*.

```
#include<stdio.h>
void main()
{
    for( register int i=0; i<100; i++) printf("\n i=%d ", i);
}
```

Utilizarea exagerată a acestei clase poate avea și efecte negative, nelăsând compilatorului posibilitatea optimizării codului executabil prin evaluarea de subexpresii, în regiștrii.

Expresiile sunt formate din operanzi și operatori.

Operandii în limbajul C pot fi: constante, constante simbolice, variabile simple, masive, variabile tip structură, elemente ale unor masive, membrii unei structuri, apeluri de funcții etc.

Operatorii limbajului C sunt numeroși formând mai multe grupe, în funcție de prioritatea lor și de direcția de evaluare (asociativitate).

Prioritatea operatorilor scade de la prima clasă către ultima. Majoritatea operatorilor se evaluează de la stânga spre dreapta; doar trei clase se evaluează de la dreapta spre stânga și acestea sunt:

- clasa operatorilor unari;
- clasa operatorului condițional;
- clasa operatorilor de atribuire.

3.1 Clase de operatori

3.1.1 Operatori pentru declarare de funcții, declarare de masive și pentru calificarea în structuri

Evaluarea acestora se face de la stânga spre dreapta.

Operatorul de indexare[] este folosit pentru declararea de masive și în adresarea indexată.

Exemplu:

```
int a[6], b[5][2], c[2][3][5][2];
float m[5][4][2];
.....
b[3][1]=c[0][0][0][0]+10;
```

Operatorul funcție () se folosește la definirea, declararea și apelul funcțiilor, iar în expresii pentru a forța executarea unor operații înaintea altora, mai prioritare.

Exemple:

```
float sqrt(float x);
void h(void);
x=f(y);
```

În expresia: $e = a + b * c + d$; se execută întâi înmulțirea $b * c$ și apoi adunările. În cazul în care se scrie $e = (a + b) * (c + d)$; atunci se fac întâi adunările și apoi se execută înmulțirea.

Operatorii de calificare în structuri . și -> se utilizează la referirea membrilor dintr-o variabilă de tip structură. Primul operator folosește numele variabilei pentru a referi un membru al structurii, cel de-al doilea utilizează adresa unei variabile (pointer) tip structură în același scop.

Exemplu:

```
struct persoana
{
    int marca;
    char nume[30];
};
persoana muncitor, *pmun;
.....
muncitor.marca=300; strcpy (muncitor.nume,"Popescu I.");
printf("\n %s", muncitor.nume);
.....
pmun=&muncitor;
pmun->marca=300;
printf("\n %s", pmun->nume);
```

3.1.2 Operatori unari

Asociativitatea acestei clase este de la dreapta la stânga.

Operatorul ! are semnificația de negație logică, în sensul că !adevărat=fals și !fals=adevărat. În limbajul C adevărat înseamnă diferit de 0, iar fals înseamnă 0.

Exemple:

```
if( ! b ) c=5; else c=4; // dacă b=0 atunci c ia valoarea 5 iar
                        // dacă b nu este egal cu 0, c ia valoarea 4
a=!0; /* a devine 1. */
b=!0; c=!3; /* b devine zero, iar c devine unu */
```

Se observă că în expresii unde intervin operatori logici, compilatorul asociază pentru adevărat numai valoarea 1.

Operatorul ~ are semnificația de negare pe biți numită și complementare la 1 (tabelul 3.1).

Exemple: Dacă a are valoarea binară 0001100011110011, atunci:
 $\sim a$ are valoarea binară 1110011100001100
 $\sim \sim a$ are valoarea binară 0001100011110011

Operatorii ++ și -- se folosesc pentru incrementare, respectiv decrementare. Dacă operatorul precede operandul atunci operația care se

efectuează se numește preincrementare sau predecrementare, iar dacă operatorul se află după operand operația este de postincrementare sau postdecrementare.

Exemple:

```
a++; /* postincrementare */
++a; /* preincrementare */
a=b++; /* dacă b = 5 atunci a ia valoarea 5, după care b devine 6 */
```

Post și prefixarea au efecte diferite doar dacă operatori ++ -- intră în compunerea altor instrucțiuni sau se compun cu alți operatori. Momentul de referință față de care se apreciază ca fiind pre sau post este tocmai momentul realizării operației cu care incrementarea / decrementarea intră în compunere. În exemplul de mai sus, ++a, ++a; au același efect, deoarece incrementarea este de sine stătătoare. În expresia a = b++; incrementarea intră în compunerea unei atribuirii, efectul de pre / postfixare devenind esențial; în acest caz se efectuează mai întâi atribuirea (a primește valoarea lui b, după care b este incrementat, adică postincrementat).

În expresia a = --b; dacă b ar fi 6, el ar fi mai întâi decrementat, luând valoarea 5, după care variabila a ar primi și ea această valoare.

Se observă că predecrementarea, ca și preincrementarea se face înainte ca operandul căruia i se aplică să participe la evaluarea expresiei, după cum postincrementarea se face după ce operandul participă la evaluarea întregii expresii. Spre exemplu, secvența:

```
int i,e[]={0,1,2,3,4,5},a=2;
i=3;
e[i]=a*i++;
```

face ca e[3] să fie 6, după care i devine 4; deci postincrementarea se execută după înmulțire și după indexare și atribuire.

```
a=b+++c; /* dacă b=3 și c=2, atunci a ia valoare 5, după care b= 4 */
```

Se observă că, în acest caz, se consideră postincrementarea lui b adunat cu c. Compilatorul în astfel de cazuri consideră în primul rând operatorul cu prioritatea cea mai mare. Cu ajutorul separatorului spațiu se poate schimba ordinea de evaluare; astfel în cazul a=b+ ++c; se consideră că se adună b la c care va fi în prealabil preincrementat. Lucrurile stau similar și la decrementare.

Într-o expresie e[i++]=5; în care i era 1, se atribuie valoarea 5 lui e[1], după care i devine 2.

Privitor la folosirea pre/post incrementărilor sau decrementărilor în apelul de funcții se poate spune că în majoritatea implementărilor, un apel printf("%d",i++); va afișa o valoare diferită de cea afișată prin apelul printf("%d",++i);

Folosirea pre/post incrementării sau decrementării în structura unor instrucțiuni necesită de asemenea o înțelegere corectă a momentului față de care se consideră pre/post fixarea.

Spre exemplu, secvența de program:

```
int i=5;
while(i--){
    printf(" i=%d ",i);
}
```

afișează: i=4 i=3 i=2 i=1 i=0, pe când secvența:

```
int i=5;
while(--i){
    printf(" i=%d ",i);
}
```

afișează: i=4 i=3 i=2 i=1

Pre / post fixarea decrementării se apreciază în raport cu momentul testului, nu cu momentul efectuării blocului repetitiv. Așadar, aceste secvențe de program prezintă diferența dintre postdecrementare și predecrementare, mai ales când acestea se constituie în expresii ce joacă rol de condiție logică pentru continuarea iterațiilor într-un ciclu.

Operatorul cast realizează o conversie forțată a unui tip în altul și are forma:

(tip)op

unde:

- *tip* este un alt tip de dată decât cel care a fost asociat operandului la declarare;
- *op* reprezintă un operand.

Exemplu:

```
int x=5;
double a;
a=sqrt((double)x); /* forțează ca variabila x să fie convertită în
double înainte de apel*/
```

Funcția *sqrt* calculează radical de ordinul doi dintr-un număr *double*. În exemplu s-a efectuat operația: $a = \sqrt{x}$

Cast-ul este frecvent folosit pentru a forța tipul la care să se evalueze o expresie, pentru a asigura o anumită precizie de calcul; spre exemplu, la împărțirea a doi întregi pentru a nu pierde partea zecimală a rezultatului, unul din operanzi este convertit prin *cast* la tipul *float*, cum se va vedea în subcapitolul 3.2, denumit *Conversii în expresii*.

Operatorul de extragere lungime sizeof(), în baiți, pentru o variabilă, o constantă, un tip, sau extragerea lungimii tipului la care se evaluează o expresie.

Exemple:

```
int d=sizeof(double); // determinare lungime tip; d ia valoarea 8
```

```

char a;
d=sizeof(a); // determinare lungime variabilă
d=sizeof(1233); // determinare lungime constantă
d=sizeof("abcdef"); // determinare lungime constanta sir de caractere
d=sizeof(4.5+20); /* d ia valoarea 8, constantele flotante memorandu-se
in dubla precizie */

```

Operatorul *sizeof* returnează un întreg și se evaluează încă din faza de compilare; este folosit frecvent la studiul implementării limbajului sau pentru a mări lizibilitatea programului.

Operatorii * și & se folosesc în lucru cu variabile pointer (adrese). Operatorul * realizează extragerea conținutului de la o adresă, iar operatorul & obține adresa unei variabile.

Exemple:

```

int x,y=3,*px=&y;
x=*px; /* cum px este adresa unei variabile de tip întreg ce are
valoarea 3, atunci si x va lua valoarea 3 */
px=&x; // px se va încărca cu adresa variabilei x
x=8, y=*&x; /* operatorii * si & sunt complementari, deci își anulează
reciproc efectul, expresia este echivalenta cu y=x; */

```

Operatorii + și - cu semnificația de schimbare de semn.

Exemplu:

```

a= -b; // dacă b=5, atunci a ia valoarea -5

```

3.1.3 Operatori aritmetici multiplicativi

Aceștia sunt * (înmulțire), / (împărțire) și % (restul împărțirii, modulo) și se evaluează de la stânga la dreapta.

Exemple:

```

int a=7,b=2,e;
e=a*b; // inmultire
e=a/b; // impartire
e=a/b*b+a%b; // e ia valoarea variabilei a (regula impartirii cu rest)
e=a%b; //se determină restul împărțirii lui a la b

```

Operatorul de determinare a restului împărțirii nu se aplică decât în cazul în care operanzii sunt de tip integral.

3.1.4 Operatori aritmetici aditivi

Din această grupă fac parte operatorii + și - pentru adunare și scădere și se evaluează de la stânga la dreapta.

Exemple:

`c=a+b;` `c=a-b;`

3.1.5 Operatori de deplasare pe biți

Ei sunt: `<<` pentru deplasare la stânga și `>>` pentru deplasare la dreapta și se evaluează de la stânga la dreapta.

Exemple:

```
int b, a=-5; b=a<<3; // b devine -40
int b, a=32; b=a>>4; // b devine 2
unsigned b, a=-5; b=a<<3; // b devine 40
unsigned b, a=-24; b=a>>3; // b devine 8189
```

Se observă că deplasarea se face *logic* dacă variabila ce se shiftează este declarată *unsigned*, sau *arithmetic*, dacă variabila este declarată ca fiind cu semn. Diferența este esențială și constă în modul diferit în care tratează bitul cel mai semnificativ.

În cazul shift-ului logic, bitul cel mai semnificativ nu e tratat ca bit de semn, el se pierde la deplasările spre stânga, iar la deplasările spre dreapta se comportă ca un bit oarecare, din afară intrând zerouri care-l înlocuiesc, ca în figura 3.1.

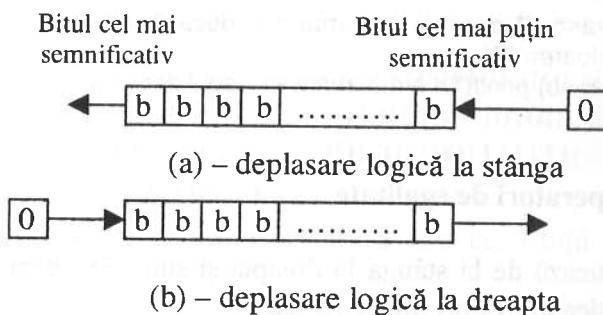


Fig. 3.1 Deplasări logice

În cazul shift-ului aritmetic, bitul cel mai semnificativ e tratat ca bit de semn, el se conservă la deplasările spre stânga, fiind ocolit, datorită reprezentării în cod complementar a numerelor negative, iar la deplasările spre dreapta se propagă bitul de semn, pentru a păstra reprezentarea în cod complementar a numărului, ca în figura 3.2.

Această diferențiere face ca operația de deplasare la stânga $a \ll b$ să fie echivalentă cu o înmulțire a lui a cu 2^b , iar $a \gg b$ să fie echivalentă cu o împărțire întreagă a lui a cu 2^b , atât pentru numere negative, cât și pozitive,

atâta timp cât rezultatul se păstrează în gama de reprezentare a întregilor respectivi.

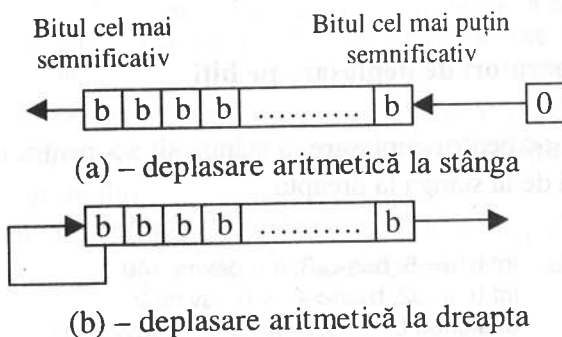


Fig. 3.2 Deplasări aritmetice

3.1.6 Operatori relaționali

Această categorie de operatori se evaluează de la stânga la dreapta și sunt: < (mai mic), <= (mai mic sau egal), > (mai mare), >= (mai mare sau egal). Întorc totdeauna valori logice (0 sau 1).

Exemple:

```
c=a<b; /* dacă a este mai mic decât b atunci c ia valoarea 1, altfel
        valoarea 0 */
if(a>=b) printf("\n a mai mare sau egal decit b");
```

3.1.7 Operatori de egalitate

Se evaluează de la stânga la dreapta și sunt: == (egal cu) și != (diferit de). Întorc totdeauna valori logice (0 sau 1).

Exemple:

```
if(a==b) printf("\n a este egal cu b");
if(x!=y) printf("\n x este diferit de y");
```

3.1.8 Operatorul ȘI pe biți

Operatorul & efectuează operația SI (tabelul 3.1) între biții corespunzători a două zone în reprezentare binară; se evaluează de la stânga la dreapta.

Exemple:

- a) dacă a este 0001110011010101 și
 b este 1010101010101010, atunci:
 $c=a\&b$; va determina ca variabila c să ia valoarea
 0000100010000000;
- b) { if($n\&1$) printf("\n numar impar");
 else printf("\n numar par");
 }
- c) $n\&0177$ // setează pe zero toți biții din n , exceptând ultimii 7 biți;

Operația $\&$ pe biți este utilă în a extrage valoarea unor biți dintr-o zonă de memorie, folosind o mască pe biți:

	b	b	b	b	b	b	b	b	&
masca:	0	1	0	1	1	1	0	1	
rezultat:	0	b	0	b	b	b	0	b	

3.1.9 Operatorul SAU-EXCLUSIV pe biți

Se evaluează de la stânga la dreapta, are simbolul $\wedge$ și efectuează SAU-EXCLUSIV între biții aparținând la două zone de memorie conform tabelului 3.1.

Exemple:

- a) dacă a este 0001110011010101 și
 b este 1010101010101010, atunci:
 $c=a\wedge b$; va prelua valoarea 1011011001111111;
- b) $a=a\wedge a$; /* inițializează pe a cu 0*/

Acest operator se poate utiliza pentru a seta cu 0 biții unei zone de memorie.

3.1.10 Operatorul SAU-INCLUSIV pe biți

Are simbolul $|$ și efectuează SAU-INCLUSIV (tabelul 3.1) între biții corespunzători din două structuri de reprezentare binară. Asociativitatea lui este de la stânga la dreapta.

Exemple:

- a) dacă a este 0001110011010101 și
 b este 1010101010101010, atunci
 $c = a | b$; va lua valoarea 1011111011111111;

b) $x = x | \text{masca};$ /* pune pe 1 toți biții din x ce au valoarea 1 în mască */

Operația este utilă pentru a seta cu valoarea 1 anumiți biți dintr-o configurație, folosind o mască pe biți:

	b	b	b	b	b	b	b	b	1
masca:	0	1	0	1	1	1	0	1	
rezultat:	b	1	b	1	1	1	b	1	

x	y	NOT x $\sim x$	x AND y $x \& y$	x OR y $x y$	x XOR y $x \wedge y$
0	0	1	0	0	0
0	1	1	0	1	1
1	0	0	0	1	1
1	1	0	1	1	0

Tab. 3.1 Operații logice pe biți

3.1.11 Operatorul ȘI - LOGIC

&& efectuează intersecție logică de condiții și se evaluează de la stânga la dreapta. Întoarce totdeauna valori logice (0 sau 1).

Exemple:

```
if(a==b&& a==c) printf("\n triunghi echilateral");
a= b==b && c!=c; /* a ia întotdeauna valoarea 0 */
```

Într-o condiție compusă, dacă prima subexpresie din **&&** e falsă nu se mai evaluează restul, rezultatul fiind 0. Acest lucru are implicații majore atunci când în condițiile de evaluat apar operatori ce ar modifica valorile unor variabile (spre exemplu, incrementări).

3.1.12 Operatorul SAU - LOGIC

|| efectuează reuniunea logică de condiții și se evaluează de la stânga la dreapta. Întoarce totdeauna valori logice (0 sau 1).

Exemple:

```
if(a*a==b*b+c*c||b*b==a*a+c*c||c*c==a*a+b*b)
    printf("\n triunghi dreptunghic");
a= b==b || c<c; /* a ia întotdeauna valoarea 1 */
```

Ca și `&&`, operatorul `||` se aplică optimizat, în sensul că dacă într-o condiție compusă prima parte este adevărată, nu se mai evaluează restul, rezultatul returnat fiind 1. Acest lucru are efecte colaterale când expresia care nu se mai evaluează conține operatori ce ar schimba conținutul unor variabile (exemplu incrementare).

Secvența:

```
int i=0,b=1;
if(b || i++)
    printf("\n i=%d",i);
```

afișează i=0.

3.1.13 Operatorul condițional

El are forma `expr_1 ? expr_2 : expr_3`. Se evaluează logic `expr_1` (**expresie_1**); dacă este adevărată se evaluează și returnează `expr_2`, altfel se evaluează și returnează `expr_3` (figura 3.3); asociativitatea lui este de la dreapta la stânga.

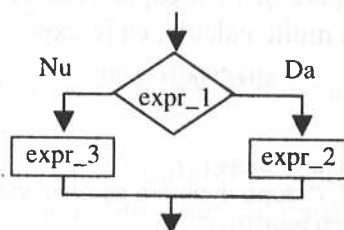


Fig. 3.3 Operatorul condițional

Exemple:

- `max= a>b ? a:b; // determina maximul dintre doua numere`
- `min = (a<b) ? (a<c)? a : c : (b<c)? b:c;`
`/* minimul din trei numere (a, b, c) */`

Să presupunem că $a=7$, $b=4$, $c=6$. Se evaluează mai întâi $(b<c)?b:c$ din care rezultă b , deci expresia devine: $(a<b)?(a<c)?a:c:b$. Se evaluează apoi $(a<c)?a:c$ din care rezultă c . În final se evaluează $(a<b)?c:b$; din care rezultă b , adică 4.

- `printf("\n numar %s", (a%2) ? "impar" : "par");`
- `(a&1) ? printf("\n a impar") : printf("\n a par");`

De observat că dacă `expr_2` și `expr_3` sunt de tipuri diferite, operatorul returnează totdeauna rezultatul convertit la tipul cel mai puternic, indiferent de expresia returnată.

Spre exemplu, dacă f este un *float* și i este un *int*, expresia $c ? i : f$, chiar când este c este adevărat (diferit de 0), va returna valoarea lui i , convertită în *float*.

3.1.14 Operatorii de atribuire

Aceștia folosesc pentru:

- atribuire simplă (=);
- atribuirii compuse (*=, /=, %=, +=, -=, >>=, <<=, &=, ^=, |=).

Se evaluează de la dreapta la stânga. Atribuirile se compun cu două categorii de operatori: aritmetici și pe biți. Atribuirile întorc valori, ceea ce permite să se compună cu ele însele sau cu alți operatori.

În ce privesc atribuirile compuse, o expresie $a \text{ operator} = b$ este echivalentă cu $a = a \text{ operator } b$. Compunerea urmărește nu numai compactarea codului, ci are implicații majore prin faptul că reduce numărul de adresări. În toate atribuirile compuse, membrului stâng i se calculează adresa o singură dată, chiar dacă apare și în dreapta ceea ce conține mai ales când această adresare presupune multe calcule, ca în expresia:

$$x[i+j*v[k]] += v[k];$$

Exemple:

```
s+=1; /* incrementarea lui s; s=s+1 */
a=b=c=1, a+=b+=c+=7; /* după evaluare se obțin valorile: c=8, b=9, a=10 */
a*=b+c; /* echivalentă cu a=a*(b+c) */
x[a=b]=3; // Se atribuie valoarea 3 lui x[b], iar a primește valoarea b.
```

3.1.15 Operatorul virgulă

“,” separă expresii ce vor fi evaluate individual, de la stânga la dreapta, sau elemente într-o listă. Ultima expresie evaluată este interpretată ca valoare a întregii expresii virgulă și deci poate participa în continuare în expresii compuse.

Exemple:

```
temp=a, a=b, b=temp;
/* interschimbarea valorilor variabilelor a cu b */
```

Expresia: $d += a + (c = 4, b * = c)$; dacă $d = 2$, $a = 5$, $c = 8$, $b = 3$, se evaluează astfel: mai întâi paranteza $c = 4$, $b * = c$, de la stânga la dreapta, deci $c = 2$ iar $b = 6$, din această expresie virgulă rezultatul care intră în calcul mai departe

este cel obținut din expresia de după ultima virgulă, deci 6. Această valoare se adună la a , iar în final se obține $d=13$.

În privința operatorilor se impun unele **observații**:

- operatorii pe biți se aplică numai tipurilor integrale (*int* și *char* cu atributele *short*, *long*, *unsigned*);
- nu este definit în limbaj operatorul de exponențiere, suplinat de funcția *double pow(double x, double y);* cu prototipul în *<nath.h>*

Pentru calculul puterilor de numere întregi este mult mai performantă o funcție bazată pe înmulțiri repetate:

```
long p(int a, int b)
{
    long r=1;
    while ( b )
        if(b&1) r*=a, b--;
        else b/=2, a*=a;
    return r;
}
```

pentru a ridica pe a la puterea b (a^b).

Se observă că s-a urmărit efectuarea unui număr cât mai mic de înmulțiri în funcție de exponent: dacă este par, atunci $a^{2n}=(a^n)^2$, dacă este impar, atunci $a^{2n+1}=(a^n)^2 \cdot a$.

- principalele clase de operatori și direcția în care se evaluează sunt prezentate în tabelul 3.2.

În vederea exemplificării folosirii operatorilor în programe C, se va scrie programul pentru afișarea unui întreg lung în binar folosind operatori și măști pe biți.

```
#include <stdio.h>
void main()
{
    long a;
    unsigned long m=0x80000000;
    printf("\n Dati numarul:");
    scanf("%ld",&a);
    while(m)
        printf("%d", m&a ? 1:0), m>>=1;
}
```

Operatorul	Semnificația	Asociativitatea
()	funcție	stânga-dreapta
[]	dimensiune masiv	
.	calificare în structură	
->	calificare în structură	dreapta-stânga
!	negare logică	
~	negare pe biți	
-	schimbare semn	
+	precizare semn	
++	incrementare	
--	decrementare	
&	extragere adresă	
*	adresare indirectă	
cast sizeof()	conversie impusă evaluare lungime	stânga-dreapta
*	înmulțire	
/	împărțire	
%	modulo	stânga-dreapta
+	adunare	
-	scadere	stânga-dreapta
<<	deplasare stânga	
>>	deplasare dreapta	stânga-dreapta
<	mai mic	
<=	mai mic sau egal	
>	mai mare	
>=	mai mare sau egal	stânga-dreapta
==	egal	
!=	diferit	stânga-dreapta
&	SI pe biți	
^	SAU-EXCLUSIV pe biți	stânga-dreapta
	SAU-INCLUSIV pe biți	stânga-dreapta
&&	SI logic	stânga-dreapta
	SAU logic	stânga-dreapta
? :	operator condițional	dreapta-stânga
= op=	atribuiri	dreapta-stânga
,	operator virgulă	stânga-dreapta

Tab. 3.2 Tabelul de precedență pentru operatori

3.2 Conversii în expresii

Cea mai mare parte a instrucțiunilor C evaluează expresii. După momentul evaluării, expresiile pot fi:

- *expresii constante*, care se evaluează la compilare, deoarece la acel moment se cunosc valorile operanzilor;

Exemplu:

```
#define NMAX 5
double m;
int x=3*NMAX+sizeof(m); // expresie constanta
```

- *expresii variabile*, evaluate la momentul execuției în funcție de valorile operanzilor.

Exemplu:

```
x=2*a+b*f(t);
```

În funcție de context, o expresie poate fi interpretată ca:

- expresie aritmetică;
- expresie logică.

În principiu, există doar expresii aritmetice, care în anumite contexte sunt interpretate ca expresii logice, după regula *diferit de 0 înseamnă adevărat, 0 înseamnă fals*. Un astfel de context poate fi cerut de o instrucțiune ce folosește expresii logice (*if*, *while*, *for*) sau de prezența operatorilor relaționali, care introduc automat un context logic.

Invers, o expresie logică generată de operatori logici sau relaționali poate fi la rândul ei implicată într-o expresie aritmetică, intrând cu valoarea de calcul 0 sau 1, după cum e falsă sau adevărată.

În expresii pot apare operanzi de tipuri de date diferite. Pentru evaluarea acestor expresii are loc un proces de omogenizare a operanzilor. Acesta presupune ca operanzii să fie convertiți la un tip unic de dată numit *tip dominant*. Un tip de dată este dominant dacă determină conversia celorlalte tipuri la el.

Regulile care controlează aceste conversii mai sunt cunoscute sub numele de *conversii implicite*. Tabelul 3.3 prezintă tipul dominant pentru principalele combinații de tipuri de date de calcul.

Regulile de conversie implicită sintetizate în tabelul 3.3 pot fi rezumate astfel:

- tipurile *char*, *short* se convertesc la *int*;

- pe sistemele pe 16 biți, tipul *long* este mai puternic decât *int*;
- tipul *float* este convertit în *double*;
- un tip *unsigned* este mai puternic față de *signed*.

Tip operand	char, short, int	unsigned	long	float, double
char, short, int	int	unsigned	long	double
unsigned	unsigned	unsigned	long	double
long	long	long	long	double
float, double	double	double	double	double

Tab. 3.3 Conversii implicite între tipuri de date

Pentru operatorul de atribuire (=), se face convertirea rezultatului obținut din evaluarea expresiei din partea dreaptă a operatorului, la tipul membrului din stânga operatorului (numit *lvaloare*). Un exemplu sugestiv poate fi surprins în următoarea secvență de program:

```
double e;
int a=7, b=5;
e=a/b;
printf("\n e=%lf ", e);
```

Expresia $e=a/b$; are operandii ce se află în partea dreaptă a operatorului = de tip întreg. În acest caz, se face împărțirea întreagă dintre 7 și 5, rezultatul fiind 1. *Lvaloarea* este de tip *double* (*e*), deci 1 va fi convertit la tipul *double* și se va afișa 1.00000. Dacă se dorește ca împărțirea să se efectueze astfel încât să se obțină și partea zecimală corectă, trebuie ca, fie unul din operandii participanți la împărțire să fie de tip *float*, *double* sau *long double*, fie să se realizeze o conversie explicită a unui operand la tipul virgulă flotantă.

Conversia explicită se realizează cu operatorul *cast*, ce a fost descris în cadrul grupei operatorilor unari. Revenind la exemplul anterior, pentru asigurarea corectitudinii prin prima modalitate, expresia trebuie redactată sub forma: $e=a/(\text{double})b$; în acest caz tipul dominant devine *double*, deci împărțirea se va face cu păstrarea părții zecimale. Deoarece *lvaloarea* (*e*) este de tip *double*, se va atribui corect rezultatul și se va afișa 1.40000.

Regula tipului la care se evaluează o expresie se aplică și în cazul constantelor. Astfel, $\text{float } x=1/2+1/2$; inițializează pe *x* cu 0.0000 pentru că împărțirea se face între întregi. Într-o expresie $\text{float } x=1/2+1/2.$; inițializarea lui *x* se face cu 0.5, deoarece unul din termeni, datorită constantei cu punct se evaluează în *double*.

Conversiile efectuează unele transformări asupra valorii operandilor:

- o conversie din *double* în *int* se va face cu pierderea părții zecimale;
- o conversie din *int* în *char* presupune pierderea părții mai semnificative;

- o conversie din *float* în *double* presupune completarea cu 0 a preciziei extinse;
- o conversie din *char* în *double* se face în doi pași: întâi se face conversia din *char* în *long* și apoi din *long* în *double*;
- în mod similar, conversia din *unsigned* în *double* presupune trecerea prin tipul *long*.

Un alt tip de conversie explicită, frecvent folosită în programele C este cea care realizează trecerea de la un tip numeric la șir de caractere și invers (conversiile din / în ASCII). Astfel, se pune problema ca având numărul *int* *a = 12565*, să se obțină șirul de caractere "12565" și invers.

Asemenea conversii se realizează folosind funcții specializate, dintre care cele mai importante sunt:

- *atoi* – pentru conversie din ASCII în *int*;
- *atol* – pentru conversie din ASCII în *long*;
- *atof* – pentru conversie din ASCII în *double*;
- *itoa* – pentru conversie din *int* în caractere ASCII;
- *ltoa* – pentru conversie din *long* în caractere ASCII;
- *_fcvt*, *_ecvt*, *_gcvt* – pentru conversie din virgulă flotantă în caractere ASCII.

Funcțiile împreună cu prototipurile lor vor fi prezentate în capitolul 7 - *Funcții*. O parte din ele pot fi ușor scrise și de programator.

Exemplu:

```
#include <stdio.h>
#include <stdlib.h>
void main()
{
    char sir[]="1452";
    int t;
    printf("\n %s",sir);
    t=atoi(sir);
    t++;
    itoa(t,sir,10);
    printf("\n %s",sir);
}
```

Conversiile din / în ASCII se mai pot efectua ușor și prin *read / write* intern, adică sub controlul unui format de editare, cu funcțiile: *sprintf()* (storage printf) și *scanf()*.

Funcția *sprintf()* este similară cu funcția *printf()* doar că ieșirea nu se mai face pe dispozitivul standard de ieșire (monitor) ci se face într-un șir de caractere. Prototipul ei este:

```
int sprintf(sir, format, lista_var_iesire);
```

unde, *sir* reprezintă șirul de caractere care va stoca ieșirea apelului, iar ceilalți parametri au aceleași semnificații ca și la funcția *printf()*.

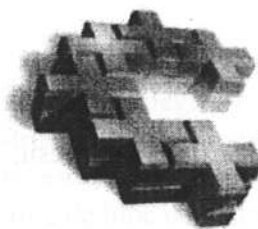
Funcția *sscanf()* este de asemenea similară cu funcția *scanf()* doar că citirea nu se mai face pe dispozitivul standard de intrare (tastatura) ci se preiau caracterele dintr-un șir de caractere. Prototipul ei este:

```
int sscanf(sir, format, lista_adrese_var_intrare);
```

unde, *sir* reprezintă șirul de caractere din care se vor prelua datele pentru a fi citite sub un anumit format, ceilalți parametri au aceleași semnificații ca și la funcția *scanf()*.

Programul următor va afișa același lucru ca și cel anterior, folosind pentru conversie funcțiile *sscanf()* și *sprintf()*:

```
#include <stdio.h>
void main()
{
    char sir[]="1452";
    int t;
    printf("\n %s",sir);
    sscanf(sir,"%d",&t); //conversie ASCII to integer; read intern
    t++;
    sprintf(sir,"%d",t); // conversie integer to ASCII; write intern
    printf("\n %s",sir);
}
```



Capitolul IV

INSTRUCȚIUNI DE BAZĂ ALE LIMBAJULUI C / C++

- **Structura liniară**
- **Structura alternativă**
- **Structuri repetitive**
- **Selecția multiplă**
- **Alte instrucțiuni**

4.1 Structura liniară

Se regăsește în limbaj sub forma evaluării unor expresii.

Exemple:

```
i++;
a=7;
printf("\n a=%d",a);
x[5]=j&cl1;
; // instructiunea vida
```

4.2 Structura alternativă

Este implementată în variantele:

pseudoalternativă (figura 4.1):

```
if(expr) instr;
```

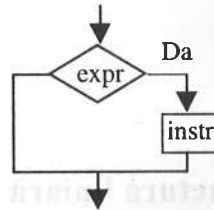


Fig. 4.1 Structura pseudoalternativă

alternativă (figura 4.2):

```
if(expr) instr_1;
else instr_2;
```

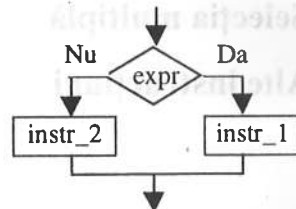


Fig. 4.2 Structura alternativă

și *alternativă multiplă:*

```

if(expr_1) instr_1;
else if(expr_2) instr_2;
/* ... */
else if (expr_n) instr_n;
else instr_n+1;

```

Prin *expr* se înțelege o expresie ce se evaluează logic, iar prin *instr* se înțelege o instrucțiune simplă sau un bloc de instrucțiuni.

else se asociază cu ultimul *if* activat; pentru a modifica asocierea se folosesc notațiile de bloc de instrucțiuni { }:

Example:

- a) if(n>0) printf("\n n pozitiv");

- b) if(a) printf("\n a nenul");
 else printf("\n a nul");

- c) if(n>0)
 if(n<1) printf("\n n pozitiv subunitar");
 else printf("\n n supraunitar sau 1");

- d) if(n>0)
 {
 if(n<1) printf("\n n pozitiv subunitar");
 } // fortare asociere else
 else printf("\n n negativ sau 0");

Ca exercițiu, se va scrie programul care verifică dacă trei numere naturale pot fi laturile unui triunghi; în caz afirmativ se va preciza tipul triunghiului: echilateral, isoscel, dreptunghic sau oarecare.

```

#include <stdio.h>
void main()
{
    unsigned a,b,c;
    printf("\n Dati laturile:");
    scanf("%u%u%u",&a,&b,&c);
    if(a==b && b==c)
        printf("\n Triunghi echilateral!!!!");
    else if(a==b || b==c || c==a)
        printf("\n Triunghi isoscel!!!!");
    else if(a*a==b*b+c*c || b*b==a*a+c*c || c*c==a*a+b*b)
        printf("\n Triunghi dreptunghic!!!!");
    else
        printf("\n Triunghi oarecare!!!!");
}

```

4.3 Structuri repetitive

Acestea sunt implementate în limbajul C în următoarele variante:

- condiționată anterior
(figura 4.3):

while(expr) instr;

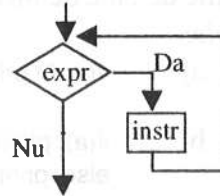


Fig. 4.3 Structura repetitivă condiționată anterior

- condiționată posterior
(figura 4.4):

do
instr;
while(expr);

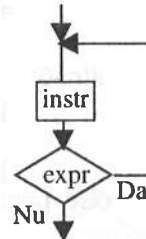


Fig. 4.4 Structura repetitivă condiționată posterior

- cu contor (figura 4.5):

for(expr_1; expr_2; expr_3) instr;

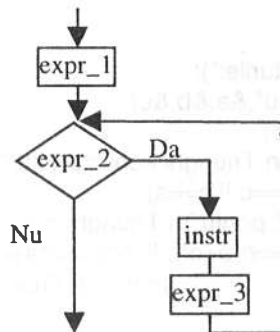


Fig. 4.5 Structura repetitivă cu contor

unde:

- *expr_1* se evaluează o singură dată, la începutul repetiției;

- *expr_2* se execută repetat, la începutul fiecărui ciclu; ea este cea care decide dacă se continuă repetiția sau nu, în funcție de rezultatul evaluării; în caz că acesta este diferit de 0 atunci se execută instrucțiunea, altfel se termină ciclul;
- *expr_3* este expresia care se evaluează repetat, la sfârșitul fiecărui ciclu.

Exemple:

```
for(s=i=0; i<n; i++) s+=x[i];
```

// realizeaza adunarea elementelor dintr-un vector

```
s=i=0; for(;n--;) s+=x[i++];
```

/* se realizeaza acelasi lucru dar lipseste *expr1* si *expr3* din *for* */

```
s=i=0; while(i<n) s+=x[i],i++;
```

/* se realizeaza acelasi lucru folosind instructiunea *while* */

```
for(s=i=0; i<n; s+=x[i++]);
```

/* se realizeaza acelasi lucru comprimand instructiunea in *expr3*, *instr* este instructiunea vida */

```
s=i=0; do s+=x[i],i++; while(i<n);
```

/* realizeaza acelasi lucru folosind instructiunea repetitiva conditionata posterior */

```
while(*ps++=*ps1++);
```

/* secventa realizeaza copierea sirului dat de *ps1* in sirul dat de *ps* */

```
for(; ;) printf("\n Ciclu infinit!");
```

Momentul efectuării testului trebuie strâns corelat și cu diferența dintre post și pre incrementare / decrementare, atunci când acestea se constituie în expresii ce joacă rol de condiție logică pentru continuarea iterațiilor într-o instrucțiune repetitivă.

Se poate observa că în instrucțiunea *for* pot lipsi unele din expresii sau chiar toate (ciclu infinit), însă este obligatoriu de scris separatorii *;*. De remarcat că în forma: *for(;;)*, condiția logică se consideră a fi întotdeauna adevărată ca și în forma: *for(1;)* și produce o ciclare infinită. Pentru celelalte structuri repetitive expresia cu rol de condiție trebuie întotdeauna să existe; pentru ciclu infinit minimul ce trebuie scris cu rol de expresie este 1.

În următoarea secvența de program se urmărește introducerea unui număr întreg cu validare; acesta va trebui să aparțină unui interval de valori [-10,10].

```
int a;
```

```
do { printf("\n Dati numarul intre -10 si 10 :"); scanf("%d",&a); }
```

```
while(a<-10 || a>10);
```

Terminarea instrucțiunilor repetitive, mai ales cele care ciclează la infinit se poate forța cu instrucțiunile: *break* sau *return*, ori cu apelul funcției *exit()*. Instrucțiunea *continue* întrerupe doar ciclul curent.

O greșeală frecventă în folosirea instrucțiunilor repetitive este terminarea lor cu “;”, astfel încât ciclează o instrucțiune vidă și nu cea dorită. În acest caz nu se semnalează nici un avertisment din partea compilatorului, eroarea fiind de natură logică.

Să se scrie programul care determină cel mai mare divizor comun dintre două numere naturale, prin metoda scăderilor succesive:

```
#include <stdio.h>
void main()
{
    unsigned a,b;
    printf("\n Dati numerele:");
    scanf("%u%u",&a,&b);
    while(a!=b)
        a>b ? a -=b : b -=a;
    a==1 ? printf("\n Numere prime intre ele!!!") :
        printf("\n Cel mai mare divior comun este %u",a);
}
```

4.4 Selecția multiplă

Selecția multiplă (figura 4.6) este implementată în limbajul C/C++ cu ajutorul instrucțiunii **switch**, care are forma:

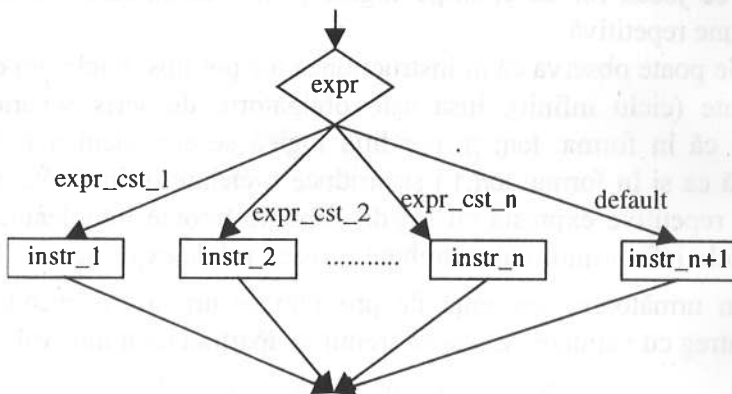


Fig. 4.6 Selecția multiplă

switch(expr)

```
{
    case expr_cst_1: instr_1 ; break;
    case expr_cst_2: instr_2 ; break;
    /* ..... */
    case expr_cst_n: instr_n ; break;
    default      : instr_n+1 ;
}
```

Pentru o folosire corectă a acestei instrucțiuni trebuie să se respecte următoarele reguli:

- *expr* trebuie să fie reductibilă la un tip integral (*char*, *int*, *short*, *long*, *unsigned*);
- *expr_cst* sunt expresii constante, reductibile la tipul integral, exprimate numeric sau simbolic; caracterul constant al expresiei și tipul integral impus limitează utilizarea instrucțiunii;
- varianta implicită (*default*) poate lipsi, dar când apare, ea trebuie să fie ultima; când nu apare, iar toate celelate constante nu se potrivesc cu valoarea expresiei de selecție, instrucțiunea *switch* nu execută nimic.
- *break* este opțional și are ca scop terminarea instrucțiunii *switch*.

Exemplu:

```
switch (rasp)
{
    case 'd':
    case 'D': printf("\n raspuns pozitiv"); break;
    case 'n':
    case 'N': printf("\n raspuns negativ"); break;
    default : printf("\n raspuns ambiguu"); break;
}
```

Se observă că dacă în mai multe cazuri trebuie să se execute aceeași acțiune, case-urile se pun unele după altele, *break* se scrie după ce s-a explicitat acțiunea. Pentru o mai bună înțelegere a instrucțiunii *break* în *switch* se dă secvența de program:

```
switch(i)
{
    case 1: printf("\n 1");
    case 2: printf("\n 2"); break;
    case 3: printf("\n 3"); break;
    default: printf("\n i = %d", i);
}
```

care, dacă: *i*=1, afisează 1 2; *i*=2, afisează 2; *i*=3, afisează 3; *i*=9, afisează *i*=9.

4.5 Alte instrucțiuni

Limbajul mai dispune de următoarele instrucțiuni:

- *break*, pentru a forța ieșirea dintr-un ciclu *while*, *do*, *for* sau pentru a termina instrucțiunea *switch* care o conține. Controlul trece la prima instrucțiune după structura repetitivă sau *switch*;

Ca exemplificare, se va scrie programul care determină dacă un număr natural este sau nu prim. Se vor căuta divizori numărului în intervalul 2 și radical din număr. Dacă nu se găsește nici un divizor în acest interval atunci numărul este prim. Instrucțiunea *break* se utilizează pentru a forța terminarea ciclului când s-a găsit un divizor.

```
#include <stdio.h>
#include <math.h>
#include <process.h>
void main()
{
    unsigned a,i,ra;
    printf("\n Numarul:");
    scanf("%u",&a);
    if(a<=2)
        { printf("\n Numarul ESTE prim!!!"); exit(1); }
    for(i=2,ra=sqrt((double)a); i<=ra; i++) if(!(a%i)) break;
    a%i ? printf("\n Numarul ESTE prim!!!"): printf("\n Numarul NU este prim!!!");
}
```

Apelul funcției *exit()* cu prototipul în *process.h* termină execuția programului.

- ; desemnează instrucțiunea vidă;

- *continue* forțează terminarea iterației curente într-un *while*, *do* sau *for* și asigură reluarea ciclului cu iterația următoare, evident după ce s-a evaluat expresia ce are rol de condiție;

De remarcat că instrucțiunea *continue* se referă doar la conținutul ciclului, nu și la alte operații ce însoțesc ciclul; spre exemplu într-un *for*, *expr_3* se execută chiar dacă s-a întâlnit instrucțiunea *continue* în blocul de repetat, deoarece ea nu face parte din bloc.

Ca exercițiu, se va scrie programul care primește de la tastatură un șir de caractere și va afișa doar caracterele cifră din șir.

```
#include <stdio.h>
#include <ctype.h>
```

```
void main()
{
    int c;
    printf("\n Sirul:");
    do
    {
        c=getchar();
        if(c=='\n') break;
        if(isdigit(c));
        else continue;
        putchar(c);
    } while(1);
}
```

Funcțiile *getchar()* și *putchar()* sunt folosite pentru citirea / afișarea unui caracter de la / pe dispozitivul standard. Macrodefiniția *isdigit()* returnează adevărat (diferit de 0), dacă parametru primit este caracter cifră ('0'-'9'), altfel fals (0).

- *goto* realizează salt necondiționat la o etichetă din program. Eticheta este un identificator cu lungime ce depinde de implementare, urmat de simbolul **:**. Folosirea instrucțiunii *goto* se evită pentru a asigura programarea structurată.

Exemplu:

```
et:
    /* .... */
    goto et
/* .... */
```

- *_asm* permite inserarea de instrucțiuni scrise în limbaj de asamblare în cadrul unui program C. Ea are forma:

```
_asm instr_asm;
```

unde, *instr_asm* reprezintă cod sursă *asm*.

Ca exemplu, se va scrie programul care împarte două numere întregi reprezentate pe 32 biți, și afișează câtul și restul. Împărțirea propriu-zisă se va face în assembler, folosind instrucțiuni *asm*, rezultatele se vor stoca în variabilele anterior definite. Avantajul acestei modalități îl constituie faptul că se obține câtul și restul într-o singură secvență, fără a mai utiliza cei doi operatori, / pentru împărțire și % pentru determinarea restului.

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int a=7,b=5,c,r;
```

```
    _asm
```

```
    {
```

```
        mov eax,a
```

```
        cdq
```

```
        // pregatire deimpartit
```

```
        // conversie 32->64 biti (edx:eax)
```

```

        div b           // efectuare impartire
        mov c,eax       // mutarea citului in variabila c
        mov r,edx       // mutarea restului in variabila r
    }
    printf("\n Citul: %d, restul: %d",c,r);
}

```

Ca exemplificare de folosire a instrucțiunilor, se vor enunța următoarele probleme și se vor scrie programele C aferente rezolvării lor.

- A. Să se scrie programul care rezolvă ecuația de gradul al II-lea:
 $ax^2+bx+c=0$

```

#include<stdio.h>
#include<process.h>
#include<math.h>
void main()
{
    int a,b,c,d;
    printf("\n Coef lui x**2:");      scanf("%d",&a);
    if(!a) { printf("\n Nu este ecuatie de gr. II !!"); exit(1); }
    printf("\n Coef lui x:");        scanf("%d",&b);
    printf("\n Termen liber:");      scanf("%d",&c);
    d=b*b-4*a*c;
    if(d>0)
    {
        printf("\n x1=%7.2lf",(-b+sqrt((double)d))/2.*a);
        printf("\n x2=%7.2lf",(-b-sqrt((double)d))/2.*a);
    }
    else if(d<0)
    {
        d= -d;
        printf("\n x1=%7.2lf+(%7.2lf)i",-b/2.*a,sqrt((double)d)/2.*a);
        printf("\n x2=%7.2lf-(%7.2lf)i",-b/2.*a,sqrt((double)d)/2.*a);
    }
    else printf("\n x1=x2=%7.2lf",-b/2.*a);
}

```

- B. Să se determine numărul minim de biți necesari pentru reprezentarea unui număr întreg pozitiv. Spre exemplu fie numărul 9, pentru a-l reprezenta în binar sunt necesari cel puțin 4 biți, $9=(1001)_2$.

```

#include<stdio.h>
void main()
{
    int a,t,nb;
    printf("\n Numarul :");  scanf("%d",&a);
    if(!a) { printf("\n Nr. de biti necesari reprezentarii nr. 0 este 1."); return; }
    nb=0; t=a;
    while(a) a>>=1, nb++;
    printf("\n Nr. de biti necesari reprezentarii nr. %d este %d.",t,nb);
}

```

C. Să se genereze și să se afișeze numerele naturale a căror pătrate formează un număr care are în componență toate cifrele de la 1 la 9. Fiecare cifră trebuie să apară o singură dată.

De exemplu, numărul 24807 are pătratul 615387249, ce corespunde cerinței enunțate.

```
#include<stdio.h>
#include<math.h>
#include<stdlib.h>
void main()
{
    char v[9],vb, sir[15];
    int i;
    long li=sqrt(123456789.), ls=sqrt(987654321.), elc;
    for(elc=li;elc<=ls;elc++)
    {
        for(i=vb=0; i<9; i++) v[i]=0;
        long t=elc * elc; ltoa(t,sir,10);
        for(i=0;i<9;i++)
            if(sir[i]!='0' || v[sir[i]-'0'-1]) { vb++; break; }
            else v[sir[i]-'0'-1]++;
        if(vb) continue;
        printf("\nNumarul %d, patratal %ld",elc,t);
    }
}
```

Pentru rezolvare, se poate observa faptul că s-au ales limitele intervalului de valori așa încât să se limiteze domeniul de valori în care se va face căutarea.

Pătratul numărului de testat este transformat în șir de caractere și apoi se prelucrează astfel încât să nu existe cifre de la 0 la 9 de mai multe ori. Matematic se știe că funcția x^2 este crescătoare și continuă pe intervalul $[0, \infty)$.

D. Pornind de la tranșele de impozitare și de la salariul brut al unei persoane, să se calculeze și afișeze impozitul total și salariu net.

```
#include <stdio.h>
#define MAXLONG 0x7fffffff
void main()
{
    long sal,imp,calc_imp(long);
    printf("\nSalariul brut: "); scanf("%ld",&sal); imp= calc_imp(sal);
    printf("\n Salariul net: %ld - %ld= %ld ", sal,imp,sal-imp );
}

long calc_imp(long sal)
{
    long niveluri[] = {0,800000,2000000,4000000,MAXLONG},imp=0;
    int procent[] = {0,15,20,40}, i;
```

```

for(i = 1; (niveluri[i] < sal) && (i < sizeof(niveluri)/sizeof(long)); i++)
    imp += (niveluri[i] - niveluri[i-1]) * procent[i-1] / 100;
imp += (sal - niveluri[i-1]) * procent[i-1] / 100;
return imp;
}

```

Se observă că tranșele de impozitare se dau într-o tabelă aparținând funcției, programul aliniindu-se automat la modificările în tabelă atât ca limite ale intervalului cât și ca număr de intervale. Fiecărui interval, $niveluri[i] - niveluri[i+1]$ i se aplică procentul de impozitare $procent[i]$.

E. Să se determine numerele naturale din intervalul $(1, x]$ (x - introdus de la tastatură) care au proprietatea că sunt egale cu suma divizorilor lor. De exemplu numărul $28 = 1 + 2 + 4 + 7 + 14$, corespunde cerinței enunțate.

```

#include <stdio.h>
#include <math.h>
void main()
{
    int i, x, s, t, rn;
    printf("\n Dati numarul:"); scanf("%d", &x);
    for(i=2; i<=x; i++)
    {
        for (m=sqrt((double)i), s=1, t=2; t<=rn; t++) if(!(i%t)) s+=t, s+=i/t;
        if(i==s)
        {
            printf("\n %d=1", i);
            for(t=2; t<=rn; t++) if(!(i%t)) printf("%d+%d", t, i/t);
        }
    }
}

```

F. Să se tabeleze valorile funcției:

$$f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_2 x^2 + a_1 x + a_0$$

pentru $x \in [a, b]$ cu pasul pas .

```

#include <stdio.h>
#include <math.h>
#include <process.h>
void main()
{
    int pol[100], a, b, n, i;
    double pas, x, s;
    printf("\n Dati gradul polinomului:"); scanf("%d", &n);
    for(i=0; i<=n; i++) { printf("Coeficientul de rang %d =", i); scanf("%d", &pol[i]); }
    printf("Limita inferioara a intervalului:"); scanf("%d", &a);
    printf("Limita superioara a intervalului:"); scanf("%d", &b);
    if(a>=b) { printf("\n LIMITE INCORECTE!!!!"); exit(1); }
    printf("Pasul:"); scanf("%lf", &pas);
    for(x=a; x<=b; x+=pas)
    {
        for(s=pol[0], i=1; i<=n; i++) s+=pol[i]*pow(x, i);
        printf("\nf(%lf)=%lf", x, s);
    }
}

```

5.1 Pointeri și aritmetica de pointeri

Pointerul este o variabilă care conține adresa altei variabile. El se utilizează pentru a face referire la o dată cunoscută prin adresa zonei de memorie alocată ei. Un pointer va conține adresa unei variabile de tip bine precizat. Definirea se face cu ajutorul operatorului de indirectare `*`.

```
int x, *px, *py;
```

S-au definit trei variabile: x de tip întreg și px , py pointeri de întreg; $*px$ semnifică faptul că nu px este de tip întreg, ci obiectul pointat este întreg (deci px conține o adresă de întreg).

```
px=&x;
```

Dacă px conține adresa variabilei x , atunci expresia $*px$ este echivalentă cu x și reciproc, px este echivalentă cu $\&x$.

Exemplu:

```
x=3; ⇔ *px=3;
y=x+200; ⇔ y=*px+200;
py=px; ⇔ py=&x;
```

După cum se poate observa, operatorii unari `&` cu semnificația de extragere a adresei unei variabile și `*` ce presupune referirea obiectului pointat de variabila pointer (extragerea conținutului) sunt indispensabili în definirea și utilizarea variabilelor pointer.

Exemplu:

```
int x=7, *px=&x;
```

La declarare pointerul poate fi inițializat cu adresa unei variabile deja declarate, ca mai sus; de remarcat că inițializarea este de forma $px=\&x$, căci `*` are rol doar în declararea pointerului. Figura 5.1 ilustrează grafic legătura dintre variabilă și adresa ei ($adr(x)$).

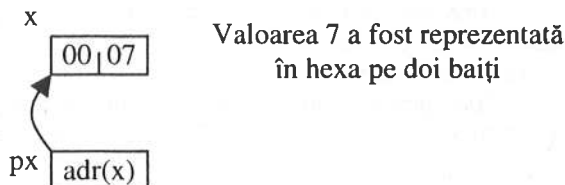


Fig. 5.1 Corespondența dintre variabilă și adresă

Un pointer poate fi refolosit, în sensul că la momente de timp diferite poate conține adrese diferite, deci referi obiecte diferite:

Exemplu:

```
int a=7, b=5, *p;
p=&a; // variabila p contine adresa lui a;
p=&b; // variabila p contine adresa lui b.
```

Modificatorul *const* poate fi folosit la declararea unui pointer având semnificații diferite în funcție de locul ocupat în declarație:

- definirea unui *pointer constant* se face în forma:

*tip *const nume;*

Dacă se definește pointerul:

```
char *const ps="arc";
```

variabila *ps* nu poate fi alterată, adică operații de forma *ps++*, *--ps*, *ps=p* etc. nu sunt permise, pointerul fiind constant;

- definirea unui *pointer la o zonă de memorie constantă* se face astfel:

*tip const *nume;*

Fiind dată declarația:

```
char const *ps="arc";
```

conținutul zonei de memorie referit de pointerul *ps* nu poate fi alterat, o operație de forma: **ps='t'*, nefiind permisă.

Un pointer poate pointa o variabilă care se află în același segment de memorie cu el, iar adresa variabilei va conține doar deplasamentul (offsetul) față de începutul segmentului, adresa de început a segmentului fiind implicită. În acest caz pointerul ocupă doi baiți și se numește pointer *near*. Într-un pointer *near* se poate reprezenta o adresă de maxim $2^{16} = 64$ KB adică în intervalul unui segment.

Pentru pointerii care referă variabile care se află în alt segment decât pointerul, precum și pe sistemele care lucrează numai cu adrese pe 32 biți (de exemplu variantele noi de Windows) este nevoie de patru baiți, iar acești pointeri se numesc pointeri *far*. Tipul pointerului se poate cere în mod explicit la declararea unei variabile pointer:

$\text{tip} \left[\begin{matrix} \text{near} \\ \text{far} \end{matrix} \right] * \text{nume}$

Exemplu:

```
char far *pv; // pv este un pointer far (pe patru baiți)
```

Pointerii stau la baza adresării indirecte, permițând construirea de programe cu grad înalt de generalitate și abstractizare. Ei folosesc nu numai ca alternativă de adresare, ci realizează sarcini specifice cum ar fi:

- adresarea indexată, caracteristică lucrului cu masive;
- alocarea și gestiunea memoriei dinamice;
- adresarea funcțiilor.

O eroare frecventă în lucru cu pointeri o constituie referirea conținutului pointat înainte ca pointerul să fi fost inițializat sau încărcat prin atribuire, cu adresa unei zone de memorie alocată anterior (adresă validă). Acest lucru se explică prin faptul că la definire variabila pointer va conține o valoare reziduală ce poate fi interpretată eronat ca adresa unei zone de memorie.

Obținerea unei adrese valide se poate face:

- prin extragerea adresei unei zone de memorie alocată la definirea unei variabile:

```
int a=30,*pa;
pa=&a;
```

Nu este permisă extragerea adresei unei constante ($pa=&562$);

- prin alocare dinamică de memorie.

1. Alocarea dinamică de memorie se poate face utilizând operatorul *new* care are forma:

```
pointer = new tip;
```

unde:

- *pointer* – reprezintă variabila pointer ce urmează a se încărca;
- *tip* – reprezintă un tip de dată pentru care se face alocarea zonei de memorie; se folosește pentru a determina mărimea în baiți a zonei de memorie ce urmează a se alocă și care este egală cu $sizeof(tip)$.

```
int *pi;
pi=new int;
*pi=5;
printf("\n %d",*pi);
```

În cazul în care se dorește a se alocă o zonă de memorie care să stocheze mai mulți întregi (*nr*), atunci se va folosi operatorul *new* în forma:

```
pointer=new tip[nr];
```

unde:

- *tip* – este tipul de dată referit de pointer;
- *nr* – reprezintă numărul de elemente de tipul *tip*;
- [] – fac parte din sintaxă.

Mărimea în baiți ce se va alocă se obține conform relației $nr* sizeof(tip)$.

Alocarea dinamică implică și operația complementară, de dealocare a memoriei care se face cu operatorul *delete*, ce are forma:

```
delete pointer;
delete [nr] pointer;
```

unde:

- *pointer* – reprezintă variabila *pointer*;
- *nr* – reprezintă numărul de elemente de tipul referit de variabila *pointer*.

Ca exemplu, se va alocă dinamic o zonă de memorie capabilă să stocheze *n* structuri de forma:

```
struct produs
{
    int codp;
    char denp[50];
    double val;
};
```

după care spațiul va fi dezalocat.

```
produs *pprod;
int n=3;
pprod=new produs[n];
.....
delete [n]pprod;
```

Memoria alocată dinamic își păstrează conținutul până când se dezalocă în mod explicit în cadrul programului.

2. Alocarea dinamică de memorie se mai poate face și cu funcții de bibliotecă specializate. Una din funcții are prototipul:

```
void *malloc(int);
```

care se află în fișierul header *alloc.h* sau *stdlib.h*. Parametrul are ca scop dimensionarea zonei de memorie în bați; funcția returnează adresa la care s-a alocat zona de memorie. Se observă că această funcție returnează un *pointer* la *void*, adică un *pointer* generic, care nu poate fi folosit în cele mai multe din operațiile uzuale ce se efectuează asupra pointerilor. De aceea, el trebuie convertit într-un *pointer* spre un tip de dată bine precizat, stocat în acea zonă.

Pentru dezalocarea zonei de memorie ce a fost alocată anterior cu funcția *malloc*, biblioteca limbajului pune la dispoziție funcția *free* cu prototipul:

```
void free(void *);
```

Parametrul funcției *free* trebuie să fie un *pointer* încărcat printr-un apel al funcției *malloc*.

În secvența:

```
int *pa, n=5;
pa=(int*) malloc (n*sizeof(int));
// prelucrari ce se efectueaza asupra zonei de memorie
free(pa);
```

s-a alocat o zonă de memorie capabilă să stocheze 5 variabile de tip întreg. Adresa zonei a fost stocată în variabila pointer *pa*. După ce s-au efectuat prelucrările necesare asupra zonei de memorie și ea nu mai este necesară în program, se poate elibera spațiul, cu funcția *free*.

Variabilele pointer sunt supuse unor prelucrări alcătuind așa numita **aritmetică de pointeri**. Principalele operații care pot fi efectuate cu pointeri sunt:

a) **extragerea obiectului referit de către pointer:**

```
int x, z=10, *px=&z;
x=*px; // x ia valoarea 10
```

b) **extragerea adresei pointerului**

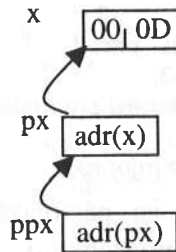


Fig. 5.2 Dublă indirectare

```
int x=13, *px, **ppx;
px=&x;
ppx=&px; // extragere adresa de pointer
printf("\n %d", **ppx); // afiseaza valoarea 13
```

Acest lucru se poate reprezenta ca în figura 5.2.

Variabila *ppx* folosește la adresarea dublu indirectată și este un *pointer la pointer* de întreg.

c) **atribuirea între doi pointeri**, se face cu condiția ca aceștia să pointeze spre același tip de dată:

```
float a=6, *pa=&a, *pb;
pb=pa; // ambii pointeri vor pointa spre aceeași zona de memorie
```

O atribuire de forma **pa=*pb*; nu afectează valoarea pointerilor, ci a obiectelor referite de aceștia. În cazul în care *pa* conține adresa variabilei *a*, iar *pb* conține adresa variabilei *b*, atunci atribuirea de mai sus este echivalentă cu *a=b*.

d) **incrementarea / decrementarea** pointerilor presupune folosirea operatorilor ++ / --; din punct de vedere al evaluării lor se respectă regulile ce au fost deja prezentate în capitolul 3.

Particularitatea constă în aceea că operația se face diferit, ținând seama de tipul de dată al obiectului referit de pointer. În general, dacă

pointerul p este spre tipul T ($T *p$) atunci $p++$ presupune ca adresa conținută în p să fie mărită cu lungimea tipului de dată T , adică se adună la p valoarea lui $sizeof(T)$, $p--$; are ca efect scăderea valorii p cu $sizeof(T)$.

Exemplu:

```
float *p=new float[3];
*p=2.5;
++p; *p=7.3;
p++; *p=9.34;
p--; --p;
```

Secvența alocă o zonă de memorie capabilă să stocheze trei valori de tip *float* (12 baiți), iar prin incrementarea pointerului se referă pe rând cele trei locații pentru încărcare; ultimile decrementări readuc pointerul pe adresa inițială.

Ținând seama de faptul că operatorii $*$, $++$, $--$ au aceeași prioritate și se asociază de la dreapta la stânga, rezultă următoarele interpretări ale expresiilor:

- $*++p$; preincrementează pointerul și apoi va extrage conținutul;
- $*p++$; extragerea conținutului zonei de memorie pointată de p , apoi postincrementare de pointer;
- $(*p)++$; extragerea conținutului zonei de memorie pointată de p , apoi postincrementare conținutului;
- $--*p$; predecrementare de conținut;
- $++*--p$; predecrementare de adresă și incrementarea conținutului de la noua adresă.

e) **adunarea / scăderea pointerilor cu o valoare întreagă** se face ținând seama de tipul de dată spre care pointează variabila pointer. În general, dacă px este un pointer spre tipul de dată T , atunci $px+n$ (n - întreg) determină obținerea unei adrese mai mari cu $n* sizeof(T)$.

Exemplu:

```
float *p=new float[3];
for(int i=0; i<3; i++) *(p+i)=i;
```

Se declară o zonă de 12 baiți și se inițializează cu valorile 0, 1, 2. De observat că spre deosebire de incrementare/decrementare, adunarea/scăderea unui întreg la un pointer nu modifică adresa conținută în pointer, ci permite doar obținerea adresei unei alte locații de memorie, într-o expresie temporară de adresă.

f) **diferența a doi pointeri** impune ca ei să poarte același tip de dată, iar ca rezultat se obține numărul de elemente de tipul respectiv ce încap între cele două adrese (și nu numărul de baiți).

Exemplu:

```
float *pi=new float[5], *pf;
```

```
pf=pi+4;
printf("\n Nr. de elemente = %d", pf-pi);
```

Rezultatul diferenței $pf-pi$ va fi 4, adică patru elemente de tip *float* despart cele două adrese.

În general, dacă *pf* și *pi* sunt pointerii spre un tip de dată *T*, atunci $pf - pi$ va determina în fapt următorul calcul $(pf - pi)/sizeof(T)$.

g) **comparații între pointeri.** Aritmetica de pointeri permite utilizarea operatorilor relaționali între operanzi de tip pointer. În general, rezultatele comparației sunt corecte, dar pentru a asigura totdeauna rezultate bune în cazul în care se folosesc mai multe segmente de memorie, pointerii trebuie să fie *huge*. Acest lucru se asigură prin alegerea modelului de memorie *huge* ce lucrează cu pointeri *far* normalizați (adică au repartizați pe adresa de bază cea mai mare parte din adresă, în zona de deplasare fiind doar precizia adresării, adică o cifră hexa).

Operatorii de egalitate ($==$ și $!=$) se pot utiliza pentru a compara pointerii cu o constantă simbolică definită într-un fișier header sub forma:

```
#define NULL 0
```

În general, pointerul *nul* semnifică faptul că el nu definește o adresă validă.

Următoarea secvență de program realizează alocarea și încărcarea unei zone de memorie și efectuează suma elementelor. Traversarea zonei de memorie s-a făcut folosindu-se variabile pointer; se poate observa că operația de comparație dintre pointeri are rolul de a identifica ultima locație.

```
int n=3;
float *p=new float[n],*pi,*pf,*pc,s;
for (int i=0; i<n; i++) *(p+i)=i+1;
for (s=0,pi=pc=p,pf=p+n-1; pc<=pf; pc++) s+=*pc;
printf("\n Suma elementelor %f",s);
```

h) **conversia pointerilor** se efectuează similar conversiilor prezentate în capitolul de expresii în limbajul C, folosind facilitatea de *cast*.

Uneori nu cunoaștem anticipat tipul de dată ce va fi stocat într-o zonă de memorie alocată dinamic; în acest caz funcțiile de alocare a memoriei întorc pointer spre un tip nedefinit încă, numit *void*. Ulterior adresa de tip *void* va fi convertită la tipul de dată stocat în această memorie.

Exemplu:

```
int *p=(int*) malloc (3*sizeof(int));
```

Diferitele tipuri de date impun restricții de aliniere în memorie (*short* se aliniază la adresă multiplu de 2, *float* la multiplu de 4, *double* la multiplu de 8), astfel încât forțarea unor conversii prin *cast* între doi pointeri diferiți ca tip pointat ar fi un nonsens, în cele mai multe din cazuri.

După cum se observă, operații precum adunarea a doi pointeri, înmulțirea / împărțirea lor cu constante etc. nu au sens în aritmetica de pointeri.

5.2 Masive uni și multidimensionale

Masivul, prin definiție, este o structură de date omogene cu un număr finit și cunoscut de elemente, ce ocupă un spațiu contiguu de memorie.

Un masiv este precizat prin clasa de memorie, tipul de dată asociat elementului, numele masivului și numărul de dimensiuni ale acestuia. Pentru fiecare dimensiune se va preciza și numărul de elemente.

Exemplu:

static	float	x	[20]	
↑	↑	↑	↑	↑nr. elemente
clasa	tip	nume	dimensiune	

Masivul *x* este unidimensional (vector) și are 20 elemente de tip *float*.

În cadrul acestei structuri de date, elementele sunt accesate prin numărul de ordine la care se află. Numerotarea elementelor începe cu 0, de aceea nu trebuie să se confunde declarația de masiv cu adresarea indexată a elementului.

Exemplu:

```
int v[50];           // declarația unui masiv cu 50 de componente;
v[50]=7;            // atribuire incorectă deoarece nu există elementul cu
                    // numărul 50, ultimul este 49, iar primul este 0.
```

Aplicând operatorul *sizeof* unui masiv se obține numărul de baiți ocupați de masiv. Astfel, definindu-se masivul *double y[10]*; *sizeof(y)* va întoarce 80, adică (10 elemente * 8 baiți fiecare). *sizeof(y)/sizeof(y[0])* este o expresie curent folosită pentru determinarea numărului de elemente ale unui masiv.

O secvență utilă în aplicații ce folosesc vectori este aceea de încărcare a lor prin citire de la tastatură fără a introduce în prealabil numărul efectiv de elemente al vectorului:

```
int a[50],n=0;
while ( printf("a[%d]=",n), scanf("%d",&a[n]) !=EOF ) n++;
for(int i=0;i<n;i++) printf("\n %d",a[i]);
```

Se observă că, citirea se termină la tastarea lui CTRL/Z, iar numărul de elemente introduse nu trebuie să depășească numărul maxim de elemente ale vectorului (nu se face verificarea încadrării în dimensiune).

În cazul masivelor multidimensionale, declararea dimensiunilor se face în paranteze drepte separate, sugerând că abordarea masivului se face în limbaj prin compunerea pe diferite nivele; astfel o matrice este un vector de vectori, un masiv tridimensional este un vector de matrice ș.a.m.d.

Abordarea este vizibilă și prin maniera în care se face inițializarea masivului.

Exemplu:

```
int v[] = {1,2,3}; // masiv unidimensional:
int a[ ][3] = {
    {1,2,3},
    {4,5,6}
}; // masiv bidimensional
```

După cum se poate observa numărul de elemente asociat dimensiunii poate lipsi, în cazul masivului unidimensional, dacă la declarare se face și inițializarea, el fiind calculat din lista de inițializatori. Pentru masivele multidimensionale numărul de elemente pe fiecare dimensiune, exceptând prima, trebuie specificat explicit chiar când masivul se inițializează la declarare.

Masivele definite globale sau statice se inițializează automat cu 0.

Este acceptată și inițializarea, în cazul masivelor multidimensionale, fără folosirea acoladelor interioare: `int a[ ][3] = {1,2,3,4}`. Când însă acoladele apar, atunci ele controlează modul de asociere a valorilor la elementele masivului:

```
int b[ ][3] = { {1,2}, {3,4} };
```

Aceste inițializări determină următoarea componență a masivelor:

$$a = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 0 & 0 \end{pmatrix} \quad b = \begin{pmatrix} 1 & 2 & 0 \\ 3 & 4 & 0 \end{pmatrix}$$

Se observă că ultima dimensiune a masivului crește cel mai repede, ceea ce înseamnă că o matrice este stocată linie după linie.

Conținând mai multe elemente, un masiv nu poate avea clasa de memorie *register*.

Tratarea masivelor este făcută foarte apropiat de abordarea pointerilor; astfel un masiv nu este altceva decât un pointer constant către zona de memorie de la care începe alocarea spațiului pentru elementele lui. Numele masivului apare ca pointer constant, pentru că la momente de timp diferite, el **nu** poate fi asociat cu adrese de memorie diferite așa cum puteam refolosi un pointer oarecare.

Ilustrarea comparabilității dintre pointer și masiv se face în exemplul de mai jos:

```
short int x[5] = {1,5,7,9,2}, *px, i=2,n;
```

```
px=x;           // px pointeaza pe primul element al masivului
n=x[i]; n=px[i]; // adresari echivalente, folosind pointerul ca vector
n=*(px+i); n=*(x+i); // folosirea vectorului x ca pointer
n=i[x]; n=i[px]; // adresari echivalente acceptate
```

Exemplul este ilustrat grafic în figura 5.3.

Se observă că adresarea $px[i]$ este admisă deși px nu este declarat ca masiv; în realitate, expresia de adresare indexată $x[i]$ este evaluată de compilator ca expresie de adresare indirectă $*(x+i)$; cum adunarea este comutativă, este ușor de înțeles de ce multe versiuni de compilatoare acceptă și expresiile $n=i[x]$, $n=i[px]$; ca în exemplul de mai sus, deși i nu este masiv și nici pointer.

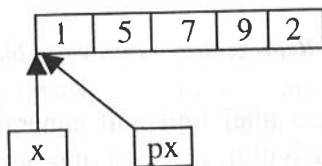


Fig. 5.3 Echivalența masiv pointer

Unele diferențieri între vector și pointer (x și px) apar totuși. Astfel, referitor la dimensiune, în timp ce $sizeof(x)$ va întoarce numărul de baiți ocupați de vectorul x , adică 10, $sizeof(px)$ întoarce numărul de baiți ocupați de pointer, adică 2 sau 4, *near* sau *far*.

O altă diferență este dată de aritmetica redusă de pointeri la care poate participa vectorul, ca pointer constant. Astfel, nu sunt acceptate expresii precum $++x$, $x--$, $x=px$, $x+=2$ etc., care presupun modificarea adresei constante x . O nuanțare apare totuși în cazul în care masivul este transferat ca parametru în funcție; transferul făcându-se implicit prin adresă, adresa constantă este memorată pe stivă și deci beneficiază de o locație de memorie unde este stocată; multe compilatoare acceptă în acest caz și expresiile care modifică această adresă.

Pentru a întregi semnificația atribuită numelui unui masiv se va prezenta modul de adresare a elementelor unui masiv bidimensional (matrice).

Exemplu:

```
float a[3][5];
```

Adresele fizice $\&a[0][0]$, $a[0]$ și a coincid ca valoare, dar diferă total din punct de vedere semantic (figura 5.4):

a - pointer spre un masiv, ale cărui elemente sunt linii formate din cinci elemente de tip *float*; $a+1$ va crește adresa cu $5 \times 4 = 20$ B;

$a[0]$ - pointer către prima linie ce conține elemente de tip *float*; $a[0]+1$ determină o adresă cu 4B mai mare;

$\&a[0][0]$ - pointer de *float*, semantic echivalent cu $a[0]$.

În același sens, rezultă că $\text{sizeof}(a)$ va returna numărul de baiți ocupat de întreg masivul ($3 \times 5 \times 4 = 60$), iar $\text{sizeof}(a[0])$ numărul de baiți ocupați de o linie ($5 \times 4 = 20$).

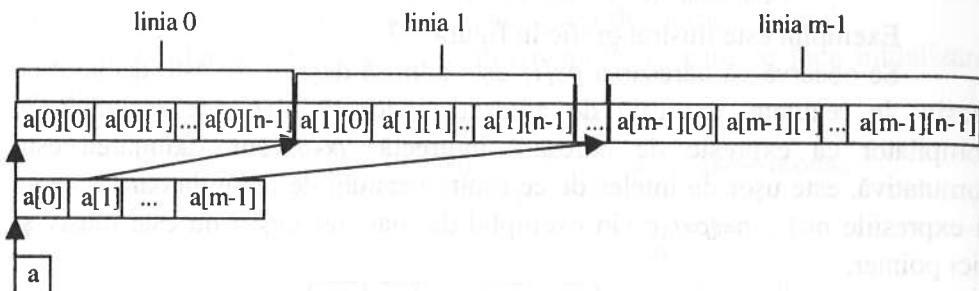


Fig. 5.4 Reprezentarea masivului bidimensional $a(m,n)$

Pentru a determina adresa unei linii i în general, se începe prin a exploata semnificația numelui masivului, descrisă anterior și va rezulta o expresie de genul $a[i]$ (folosirea adresării indexate); dacă se dorește realizarea aceluiași lucru folosind adresarea indirectă atunci se va folosi expresia $*(a+i)$.

Determinarea elementului j , de pe linia i având calculată adresa liniei i se va face, fie folosind o expresie de genul $a[i][j]$ (două indexări), fie $*(a[i]+j)$ (o indexare și o indirectare), fie $*(a+i)[j]$ deci o indirectare și o indexare, sau $*(*(a+i)+j)$, deci două indirectări:

$a[i][j]$
 $*(a[i]+j)$
 $*(a+i)[j]$
 $*(*(a+i)+j)$

Rezultă din cele prezentate că dacă a este un masiv bidimensional atunci se pot stabili următoarele echivalențe în adresarea elementului particular $a[0][0]$:

$**a \Leftrightarrow *a[0] \Leftrightarrow a[0][0]$ (modalități de a obține elementul de rangul 0, dintr-o matrice);

$*a \Leftrightarrow a[0] \Leftrightarrow \&a[0][0]$, pointer de *float* ce dă adresa liniei 0.

Pentru a ajunge la nivel de element se observă că totdeauna trebuie să avem numărul total de operatori (indexări și indirectări) egal cu numărul de dimensiuni ale masivului.

Adresa elementului de rang j , din linia i dintr-o matrice se mai poate determina dacă se cunoaște dimensiunea submasivului. Presupunând că un vector (o linie) are n componente, atunci adresa elementului (i, j) al matricei x se calculează ca $\&x[0][0] + n*i + j$, iar elementul se determină ca fiind $*(\&x[0][0] + n*i + j)$ sau $*(x[0] + n*i + j)$.

Această funcție de adresare explică de ce la inițializare sau când masivul este transmis ca parametru în funcție i se poate omite doar prima dimensiune, toate celelalte fiind implicate în semantica pointerului.

Pentru masivele tridimensionale lucrurile stau în mod similar. O declarație de forma `double a[3][2][5]` indică faptul că se alocă spațiu pentru trei matrice, fiecare din ele având două linii și cinci coloane. Spre deosebire de masivul bidimensional, cel cu trei dimensiuni mai adaugă un nivel de indirectare și anume cel care conține adresele matricelor. În acest caz `a[i]` referă adresa matricei *i*, `a[i][j]` adresa liniei *j* corespunzătoare matricei *i*, iar `a[i][j][k]` se folosește pentru accesarea elementului *j*, *k* din matricea *i*, în mod indexat. Semantica lui `a++` face ca `a+1` să determine creșterea adresei cu $2 \times 5 \times 8 = 80\text{B}$.

Ca exercițiu, se va construi programul care determină și afișează transpusa unei matrice de dimensiune $m \times n$, citită de la consolă.

```
#include <stdio.h>
void main()
{
    int a[50][50], at[50][50], m, n, i, j;
    printf("\nNr. linii:");    scanf("%d", &m);
    printf("\nNr. coloane:");  scanf("%d", &n);
    printf("Elementele:\n");
    for(i=0; i<m; i++)
        for(j=0; j<n; j++)
            { printf("a[%d][%d]=", i, j); scanf("%d", &a[i][j]); }
    for(i=0; i<m; i++) for(j=0; j<n; j++) (*(at+j)+i)=(*(a+i))[j];
    printf("\n Matricea transpusa este:\n");
    for(i=0; i<n; i++)
    {
        printf("\n");
        for(j=0; j<m; j++) printf("%7d", *(at[i]+j));
    }
}
```

A se observa diferitele modalități de adresare a elementelor matricei.

5.3 Masive de pointeri și pointer la masive

Definirea unui pointer se poate face și în combinații cu tipuri de date, altele decât cele fundamentale, rezultând unele tipuri derivate, utile în diferite aplicații. Ca orice variabile, pointerii pot fi organizați în masive.

Definirea unui masiv de pointeri presupune existența operatorului de indexare [] specific lucrului cu masive, dar și cel de referire conținut *, specific pointerilor.

```
int *a[10]; // vector de 10 pointeri la tipul întreg
```

În acest moment este util să se precizeze regulile prin care se identifică tipul unei variabile, deoarece declarațiile pot deveni foarte criptice.

O primă modalitate de depistare a semnificației unei variabile constă în a aplica precedența și asociativitatea operatorilor ce intervin în declarație.

Pentru `int *a[10]` se observă că prioritatea cea mai mare o are operatorul [], ceea ce în declarație specifică faptul că *a* este în primul rând vector, apoi urmează *, ce precizează că vectorul este de pointeri.

Un alt mod de identificare a tipului unei variabile este cel prin care se interpretează operatorii ce-l însoțesc în ordinea dreapta-stânga, până la epuizarea lor, pornind de la identificatorul variabilei.

În exemplul `int *a[10]`, în dreapta lui *a* parantezele pătrate exprimă faptul că este vorba de un vector cu 10 componente, iar în stânga * precizează că vectorul este de pointeri; *int* stabilește tipul de dată spre care puntează pointerul.

După aceleași reguli se poate stabili faptul că declarația `int (*a)[10]` exprimă faptul că *a* este un pointer la un masiv unidimensional cu 10 elemente de tip *int*, iar `int (*a[3])[10][10]` precizează că *a* este un vector de pointeri, cu trei componente pointând la masive bidimensionale, 10x10 de tip întreg.

Un pointer la masiv este de fapt *adresa numelui masivului*, lucru pus în evidență prin exemplul:

```
int a[10][10], (*pa)[10][10];
pa=&a;
```

```
:
```

Utilitatea *vectorului de pointeri* este evidențiată mai ales prin faptul că în acest mod se pot defini, manipula și inițializa în mod unitar zone de memorie cu un anumit grad de eterogenitate. Să presupunem că operăm cu șiruri de caractere (vectori) care prezintă diferențe sub aspectul lungimii lor. Organizarea vectorilor în matrice ar fi inefficientă deoarece nu toate șirurile au aceeași lungime, iar o parte din memoria alocată nu ar fi utilizată (șirurile scurte). Putem lăsa vectorii independenți, dar prelucrările sunt omogene, încât se justifică gestiunea unitară a tuturor vectorilor, lucru posibil printr-un vector de pointeri, în care fiecare element referă câte un șir.

Exemplul de mai jos sortează alfabetic zilele săptămânii folosind un vector de pointeri la caracter și funcția *strcmp()* pentru a compara două șiruri.

```
#include<stdio.h>
#include<string.h>
```

```

void main()
{
    char z0[]="duminica",z1[]="luni",z2[]="marti",z3[]="miercuri",z4[]="joi",
        z5[]="vineri",z6[]="simbata";
    char *s[]={z0,z1,z2,z3,z4,z5,z6}, *aux;
    int i,j;
    for(i=0;i<6;i++)
        for(j=i+1;j<7;j++)
            if(strcmp(s[i],s[j])>0)
            {
                aux=s[i]; s[i]=s[j]; s[j]=aux;
            }
    for(i=0;i<7;i++) printf("\n %s",s[i]);
}

```

Se poate observa că au fost interschimbați pointeri și nu șiruri de caractere, lucru care face interschimbul mult mai rapid.

Dacă am fi dispus de încă un vector de pointeri la *char*, am fi putut gestiona aceleași zone de memorie după criterii diferite (ordinea logică a zilelor și ordinea lor alfabetică).

Pointerul la masiv își relevă utilitatea în cadrul programelor C și la definirea masivelor de pointeri la funcții, așa cum se va vedea în capitolul 7.

Cum numele unui masiv este el însuși pointer, reiese că numele unui masiv de pointeri este în fapt un pointer la pointer; trebuie totuși făcută distincția dintre o declarație `int **ppx`; și `int *x[30]`; distincție care ține de diferența dintre pointer variabil și pointer constant.

Pointerul la masiv este tot un pointer la pointer, dar distincția dintre o declarație `int **ppx`; și `int (*pm)[30]`; este una fundamentală; în timp ce `ppx+1` asigură creșterea adresei cu 2 sau 4 baiți în funcție de lungimea pointerului `px` a cărui adresă e conținută de `ppx`, `pm+1` determină o creștere a adresei cu numărul total de baiți ocupați de cei 30 întregi ai masivului.

O altă utilitate majoră a pointerilor apare în cazul sistemelor ce segmentează memoria în segmente de 64 KB; care nu permit ca o variabilă, fie ea chiar masiv, să depășească limitele unui segment. În acest caz, un vector de pointeri va ține adresele blocurilor în care a fost spart masivul, iar echivalările în adresare vor permite adresarea indexată, ca și cum variabila ar fi un masiv oarecare.

Să se construiască o funcție care alocă dinamic o matrice pornind de la numărul de linii și coloane al ei. Construcția va permite accesarea elementelor masivului folosind adresarea indexată tipică lucrului cu matrice (`matrice[i][j]`).

```

#include <stdio.h>
#include <stdlib.h>

```

```

int **alloc_mat(int nl, int nc)
{
    int **pmat,i;
    pmat=(int **)malloc(nl*sizeof(int*));
    for(i=0;i<nl;i++) pmat[i]=(int*) malloc(nc*sizeof(int));
    return pmat;
}

```

Se observă că alocarea de memorie necesară stocării liniilor matricei nu s-a făcut într-un spațiu contiguu de memorie. Legătura între linii se face prin intermediul vectorului de pointeri *pmat* ca în figura 5.5. Această modalitate surclasează una din restricțiile majore ale limbajului C, aceea de a da încă din compilare dimensiuni constante pentru masivul de alocat.

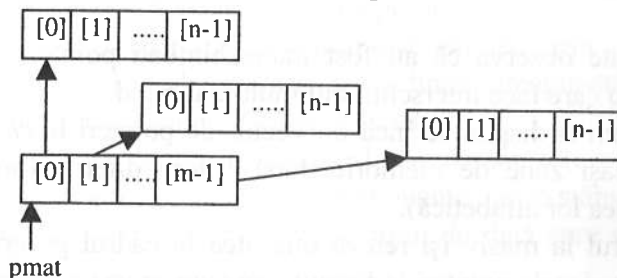


Fig. 5.5 Alocare dinamică de memorie pentru o matrice

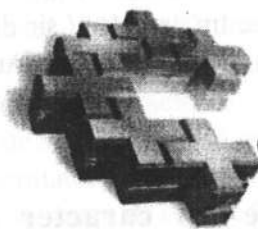
Se construiește programul care va folosi funcția de alocare a matricei și va citi și afișa elementele matricei în vederea testării funcției.

```

void main()
{
    int **a,i,j,m,n;
    printf("\n Nr. linii:");    scanf("%d",&m);
    printf("\n Nr. coloane:");  scanf("%d",&n);
    a=alloc_mat(m,n);
    for(i=0;i<m;i++)
        for(j=0;j<n;j++)
        {
            printf("\n Elementul [%d][%d]=",i,j);
            scanf("%d",&a[i][j]);
        }
    printf("\n Matricea introdusa:\n");
    for(i=0;i<m;i++)
    {
        printf("\n");
        for(j=0;j<n;j++) printf("%7d",a[i][j]);
    }
}

```

Se observă că semantica pointerului *a* este indusă prin alocarea în trepte a vectorului de pointeri la linii, apoi a liniilor propriu-zise.



Capitolul VI

LUCRU CU ȘIRURI DE CARACTERE

- **Variabile și constante de tip caracter / șir de caractere. Pointeri la caracter**
- **Funcții de intrare / ieșire pentru caractere / șiruri de caractere**
- **Funcții pentru prelucrarea șirurilor de caractere**

Se prezintă constantele și variabilele de tip caracter / șir de caractere, operațiile de I/E adecvate, alte operații și funcții de prelucrare a șirurilor.

6.1 Variabile și constante de tip caracter / șir de caractere. Pointeri la caracter

În limbajul C, constanta de tip caracter se reprezintă printr-un singur caracter, încadrat de apostrof. De exemplu reprezentările 'A', 'a', '9', ' ' corespund constantelor caracter ASCII: A, a, 9 și blank (spațiu). De remarcat că '9' reprezintă valoarea $(57)_{10}$ în ASCII, care diferă de reprezentarea numerică internă a cifrei 9 în memoria calculatorului $(00001001)_2$.

Toate caracterele se reprezintă pe un bait și se comportă ca întregi reprezentabili pe un bait, putând intra în calcule specifice aritmeticii cu întregi. Astfel în C au sens operații 'c' - 'a', 'A' + '7', ale căror rezultate pot fi folosite în construirea algoritmilor de conversie a literelor mici în litere mari, a unei valori numerice de calcul într-o valoare de editare, construirea de coduri etc.

De exemplu, programul:

```
void main()
{
    printf("\n valoare numerica%d", '1' - '0');
    printf("\n valoare numerica%d", '2' - '0');
    /* .... */
    printf("\n valoare numerica%d", '9' - '0');
}
```

afișează cu descriptor de format numeric, rezultatul operațiilor cu caractere. De fapt, operațiile sunt:

$$(49)_{10} - (48)_{10} = 1; (50)_{10} - (48)_{10} = 2; \dots (57)_{10} - (48)_{10} = 9.$$

O secvență: `printf("\n conversie in litera mare %c ", 'a' - ' ');` realizează conversia lui 'a' în 'A', iar afișarea rezultatului este efectuată cu descriptorul `%c`, specific editării caracterelor.

Secvența:

```
char c='z';    printf("\n %c", c-( 'a'-'A' ) );
c='M';        printf("\n %c", c+( 'a'-'A' ) );
```

ne arată un mod de a converti orice literă mică în literă mare, scăzându-i decalajul care există în codul ASCII, între 'a' și 'A'; adunând același decalaj

realizăm conversia unei litere mari în literă mică.

Constantele de tip șir de caractere, denumite uzual și literali se reprezintă printr-o succesiune de caractere încadrate între ghilimele; de exemplu "text", "123 aB" sunt constante de tip șir de caractere, având lungimile 4 și respectiv 6. Șirurile putând fi de lungime variabilă, nelimitată superior decât de resursele de memorie disponibile, se impune folosirea unui delimitator de sfârșit de șir, la reprezentarea acestora în memorie.

Delimitatorul de sfârșit de șir este valoarea zero binar, căreia în C îi corespunde secvența de escape '\0', care ocupă ultimul bait din șir; deci șirurile anterioare vor ocupa în memorie 5 și respectiv 7 baiți.

Pentru constantele șiruri de caractere declarate într-un program, problema delimitatorului este rezolvată de compilator, care adaugă terminatorul '\0'. De asemenea, funcțiile de lucru cu șiruri de caractere produc șiruri valide (cu terminatorul '\0'). În situația în care șirul este un rezultat al prelucrărilor proprii ale unui program, cade în sarcina programatorului gestiunea terminatorilor șirurilor prelucrate.

Între 'c' și "c" există deci o deosebire esențială la reprezentarea în memorie, întrucât "c" ocupă 2 baiți (primul corespunzând caracterului ASCII c și al doilea lui zero binar), în timp ce 'c' ocupă doar un bait. Nu trebuie uitat că există constante de tip caracter care deși se redau prin mai multe caractere grafice ele reprezintă totuși un caracter și ocupă un bait în memorie; spre exemplu, '\t' reprezintă caracterul TAB, '\x41' este forma hexazecimală a lui 'A', iar '\101' forma octală a lui 'A', deși între apostrofuri sunt patru semne grafice.

Definirea variabilelor de tip caracter se realizează cu declarația char. Variabila de tip șir apare ca un masiv de caractere, fiecare nivel elementar posedând delimitatorul de sfârșit de șir. De exemplu, programul:

```
void main()
{
    char a[] = { 't','r','e','i', '\0' };
}
```

construiește un masiv de caractere inițializat cu șirul "trei", care în memorie se reprezintă ca în figura 6.1:

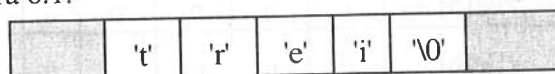


Fig. 6.1 Memorarea unui șir

Inițializarea masivului se putea face mai simplu prin folosirea unei constante șir de caractere: char a[]="trei";

Folosind variabile de tip pointer la caracter, inițializarea de mai sus se poate realiza și în maniera: char *pa = "trei";

În faza de compilare se construiește șirul "trei" într-o zonă disponibilă de memorie, a cărei adresă este memorată în pointerul *pa*. Adresarea caracterului 'r' de pe poziția a doua din șir se poate face prin construcțiile: *a[1]*, **(a+1)* sau **(pa+1)*, care reflectă aritmetica de pointeri pentru șiruri.

Între declarațiile *char a[]="trei";* și *char *pa="trei";* există deosebiri esențiale privind dimensiunea memoriei ocupate, precum și libertatea de a modifica adrese.

În varianta a doua se rezervă în plus 2 sau 4B pentru pointerul *pa*. În timp ce în primul caz, odată alocată memoria pentru vectorul *a*, adresa acestuia nu poate fi modificată, în al doilea caz, pointerul *pa* poate fi refolosit ulterior, punctând alte adrese de variabile tip caracter, după dorința programatorului (cu riscul pierderii adresei constantei șir, de la inițializare).

Lucrul cu pointeri la caractere impune o atenție sporită, știind că adresele pe care aceștia le conțin pot referi orice zonă de memorie, generând erori imprevizibile în situația gestionării lor inadecvate.

Reprezentarea șirurilor ca masive de caractere și modul de interpretare a masivelor în C asigură ca într-un masiv cu două dimensiuni, adresarea cu un indice să individualizeze un anumit șir din mulțimea șirurilor, iar adresarea cu doi indici să localizeze un caracter.

De exemplu, în programul următor se prezintă modul de inițializare a unui masiv bidimensional și posibilitățile de adresare, folosind unul sau doi indici. Masivului *zile* îi corespunde reprezentarea din figura 6.2.

	0	1	2	3	4	5	6	7	8
<i>zile[0]</i>	l	u	n	i	\0				
<i>zile[1]</i>	m	a	r	t	i	\0			
<i>zile[2]</i>	m	i	e	r	c	u	r	i	\0
<i>zile[3]</i>	j	o	i	\0					
<i>zile[4]</i>	v	i	n	e	r	i	\0		
<i>zile[5]</i>	s	a	m	b	a	t	a	\0	
<i>zile[6]</i>	d	u	m	i	n	i	c	a	\0

Fig. 6.2 Reprezentarea masivului *zile*

La rulare se afișează în două variante zilele săptămânii, precum și prima literă a fiecărei zile.

```
#include<stdio.h>
void main()
{
    char zile[7][10]=
        { "luni","marti","miercuri","joi", "vineri","sambata","duminica" };
    int i,j;
    for(i=0;i<7;i++) printf("\n %s",zile[i]);
    for (i=0;i<7;i++)
    {
        printf("\n");
        for(j=0; j<10 && zile[i][j]!='\0';j++)
            printf("%c", zile[i][j]);
    }
    for(i=0;i<7;i++) printf("\n%c", zile[i][0]);
}
```

Rezolvarea aceleiași probleme folosind pointeri evidențiază și mai pregnant tratarea ca entitate distinctă a fiecărui șir, a cărei adresă va fi conținută într-un element al vectorului de pointeri *pzile*.

pzile[0] →	l	u	n	i	\0				
pzile[1] →	m	a	r	t	i	\0			
pzile[2] →	m	i	e	r	c	u	r	i	\0
pzile[3] →	j	o	i	\0					
pzile[4] →	v	i	n	e	r	i	\0		
pzile[5] →	s	a	m	b	a	t	a	\0	
pzile[6] →	d	u	m	i	n	i	c	a	\0

Fig. 6.3 Vector inițializat de pointeri la caracter

```
#include<stdio.h>
void main( )
{
    char *pzile[7]={ "luni","marti","miercuri","joi", "vineri","sambata","duminica"};
    int i,j;
    for(i=0;i<7;i++) printf("\n %s",pzile[i]);
    for (i=0;i<7;i++)
    {
        printf("\n");
        for(j=0;*(pzile[i]+j]!='\0';j++) printf("%c",*(pzile[i]+j));
    }
    for(i=0;i<7;i++) printf("\n%c", *pzile[i]);
}
```

Adresele șirurilor vor fi memorate în elementele masivului de pointeri ca în figura 6.3.

De remarcat că din punct de vedere al limbajului C, o constantă șir de caractere este o adresă și poate participa la calcule specifice aritmeticii pointerilor constanți; astfel `printf("\n %s", "abcdef"+2 )`; afișează subșirul `cdef`, care începe cu 2 B după începutul șirului dat, iar `printf("\n %c", *("abcdef"+3))`; afișează caracterul `d`, care este conținutul de la locația care se află la 3 B distanță față de începutul șirului dat.

6.2 Funcții de intrare / ieșire pentru caractere / șiruri de caractere

Pentru realizarea operațiilor de intrare/ieșire, limbajul C, prin descriptorii `%c` și `%s` în funcțiile `scanf` și `printf`, oferă o modalitate comodă de citire și afișare a caracterelor și a șirurilor de caractere.

Un inconvenient al funcției `scanf` este acela că blankul, TAB, New Line acționează implicit ca separatori și deci forțează prin apariția lor sfârșitul șirului, neputând fi citit într-o singură variabilă un text format din mai multe cuvinte.

În completarea acestor funcții standard de intrare/ieșire, limbajul C pune la dispoziție și funcții de intrare/ieșire pentru un caracter și respectiv pentru șiruri.

Funcția `c=getchar()` asigură preluarea într-o variabilă de tip `int` sau `char` a unui caracter de la terminal. Varianta `c=getch()` realizează același lucru fără a returna și ecoul caracterului citit; în plus, la citirea unui caracter acesta este imediat preluat din buferul tastaturii, fără a se aștepta prezența vreunui terminator (Enter). Există și combinația celor două efecte, varianta `c=getche()`, care citește fără terminator, dar cu ecoul caracterului citit.

Echivalentul lor pentru ieșire este funcția `putchar(c)`.

De exemplu, programul de mai jos realizează citirea și scrierea unui caracter:

```
#include<stdio.h>
void main ( )
{
    int c;
    c = getchar ( );  putchar (c);
}
```

A fost necesară definirea cu `int` a lui `c`, datorită prototipului funcției `int getchar()`; gândit astfel pentru a putea prelua în afara caracterelor obișnuite și

pe cele care transmit 2B la apăsare (exemplu tastele săgeți) sau coduri asociate funcțiilor de fișier: New line, CTRL/Z (EOF), necesare testării unor condiții. Spre exemplu, programul de mai jos copiază din fișierul standard de intrare, stdin, pe fișierul standard de ieșire, stdout, caracter cu caracter, până la detectarea sfârșitului de fișier:

```
#include<stdio.h>
void main ( )
{
    int c;
    while ((c=getchar())!=EOF)  putchar (c);
}
```

Tastând în intrare CTRL/Z, variabilei *c* i se atribuie valoarea -1, asociată și constantei simbolice EOF, în fișierul <stdio.h>.

Caracterele speciale (TAB, ESC, CR, FF, BACKSPACE etc.) la imprimare execută diverse funcții terminal. Dacă se dorește simpla lor afișare, fără a fi executate, simbolurile asociate lor trebuie filtrate și transmise în conformitate cu convențiile limbajului C, ca șiruri de caractere aferente secvențelor de escape.

Un astfel de filtru este realizat prin programul de mai jos:

```
#include <stdio.h>
#include <conio.h>
#include <ctype.h>
void main ( )
{
    int c;
    while ((c = getch ( ))!= 0x1a)
        switch (c)
        {
            // executie coduri de control imprimare
            case '\n': printf("\n"); break;
            case '\r': printf("\r"); break;
            case '\t': printf("\t"); break;
            case '\v': printf("\v"); break;

            // ...

            default:
                if (isprint(c))      putchar (c); // coduri printabile
                else if (c>=1 && c<=26)
                    printf("^%c",c+'A'-1); // ^X -codurile pentru CTRL/X
                else printf("\\%3o",c);break; // restul, afisate in octal
        }
}
```

Programul găsește forme adecvate pentru tipărirea tuturor codurilor ASCII citite de la terminal, fără ecou la citire și fără a aștepta terminator de introducere.

Verificarea apartenenței unui cod la o clasă se poate face folosind

macrodefiniții din fișiere de includere standard (*ctype.h*, *stdio.h*, *stdlib.h*). Ele se aplică unei variabile de tip *int* și returnează 1 când codul caracterului aparține clasei testate și 0 în rest.

Prezentăm în continuare numele unor astfel de macrodefiniții și clasa pe care o testează:

<code>isalpha()</code>	litere;
<code>isupper()</code>	litere mari;
<code>islower()</code>	litere mici;
<code>isdigit()</code>	simbolurile bazei zecimale;
<code>isxdigit()</code>	simbolurile bazei hexazecimale;
<code>isalnum()</code>	litere sau cifre;
<code>isspace()</code>	spațiu, tab, CR, NL, VT, FF;
<code>ispunct()</code>	semne de punctuație;
<code>isprint()</code>	caractere imprimabile (codurile ASCII 32 ₁₀ - 126 ₁₀);
<code>isgraph()</code>	simboluri grafice (imprimabile, mai puțin spațiul);
<code>iscntrl()</code>	codurile de control (ASCII 0 ₁₀ - 31 ₁₀);
<code>isascii()</code>	codurile ascii (ASCII 0 ₁₀ - 127 ₁₀);

Programul de filtru, propus anterior, împarte deci codurile ASCII în patru clase:

- caractere imprimabile - pe care le tipărește ca atare;
- caractere de control - pe care le tipărește ca două caractere, din care primul este ^;
- secvențe de evitare - pe care le tipărește conform convențiilor C;
- alte coduri - imprimate sub forma secvențelor de evitare, date în octal.

Pentru intrări/ieșiri de șiruri există funcții similare.

Funcția `gets(char *sir)` citește un șir de pe fișierul standard de intrare; numai new-line este interpretat ca sfârșit de șir, fără a fi memorat, dar asigură generarea delimitatorului de sfârșit șir ('\0').

Funcția `puts(char *sir)` tipărește pe fișierul standard de ieșire un șir, după care inserează new-line.

Programul:

```
#include<stdio.h>
void main ( )
{
    char sir[81]; int ns; ns=0;
    while (gets(sir)!=(char *) NULL)  ns++;
    printf ("\n numar siruri:%d",ns);
}
```

citește, numără și scrie șiruri de caractere până când funcția returnează un

pointer nul; în mod normal funcția returnează pointerul spre variabila șir, trimisă spre încărcare; la tastare CTRL/Z funcția returnează NULL, convertit în pointer *char*; la tastare *Enter*, singur pe o linie (în loc de șir), funcția întoarce tot adresa lui *sir*, dar acesta nu va conține decât terminatorul '\0'; deci trebuie făcută distincția dintre un șir nul (vid) și un pointer nul de șir.

6.3 Funcții pentru prelucrarea șirurilor de caractere

Își au prototipul în *string.h* și sunt destul de numeroase. Ca exercițiu, pentru unele din aceste funcții se prezintă texte sursă cu utilizarea lor; pentru celelalte se prezintă doar obiectivul funcției, parametrii de intrare și rezultatul returnat.

Funcția *strlen()* determină lungimea unui șir, transmis printr-un pointer, dedusă pe baza terminatorului de șir; face parte din biblioteca standard C, dar poate fi ușor construită și de programator, sub forma:

```
int strlen(char *psir)
{
    for(int n=0; *psir!='\0'; n++) psir++;
    return n;
}
```

Copierea unui șir într-o zonă rezervată se efectuează prin apelarea funcției *strcpy* având ca parametri pointerii către șirul destinație și respectiv către șirul sursă. O alternativă de programator pentru această funcție ar putea fi:

```
char *strcpy(char *pdest, char *psursa)
{
    while(*pdest++ = *psursa++) ;
    return pdest;
}
```

De remarcat că deși parametrii de intrare sunt pointeri de șir, ei trebuie să fie în funcția apelatoare descriși ca vectori, sau să fie pointeri, dar încărcăți în prealabil cu adrese de vectori sau cu adresa unor zone de memorie alocată, capabilă de a stoca șirurile sursă și destinație.

Varianta *strncpy()* din biblioteca standard realizează copierea a cel mult *n* caractere ale unui șir, oferind posibilitatea verificării încadrării în capacitatea șirului destinație; parametrul întreg *n* se furnizează pe poziția a treia, după pointerii șirurilor.

Compararea lexicografică a două șiruri având pointerii *psir1*, *psir2* este realizată de funcția *strcmp*, care returnează 0 dacă șirurile sunt identice, o

valoare pozitivă dacă $sir1 > sir2$ și o valoare negativă, dacă $sir1 < sir2$. În cazurile în care șirurile au lungimi diferite, rezultatul se referă la compararea caracterelor până la apariția unei diferențe. De exemplu, "Pop" < "Popescu" deși coincid pe porțiunea dată de lungimea celui mai scurt, căci '\0' terminatorul primului șir este mai mic decât 'e' din al doilea șir.

Prezentăm două versiuni ale unei astfel de funcții, versiuni ce pot fi testate, făcând pe rând pe câte una comentariu; altfel s-ar semnala eroare la redefinirea unei funcții cu același prototip, chiar dacă odată apare pointer de *char*, iar altă dată vector de *char*:

```
#include<stdio.h>
#include<string.h>
/*
    int strcmp (char *psir1, char* psir2)
    {
        for ( ; *psir1 ==*psir2; psir1++ , psir2++ ) ;
        return (*psir1-*psir2);
    }
*/
int strcmp (char sir1[], char sir2[])
{
    int i = 0;
    while(sir1[i]==sir2[i]) i++ ;
    return sir1[i]-sir2[i];
}

void main ( )
{
    char s1[20]="Pop", s2[]="Popescu ";
    if ( strcmp(s1,s2) < 0) printf("%s < %s\n", s1,s2);
    else if ( strcmp(s1,s2) > 0) printf("%s > %s\n", s1,s2);
    else printf("%s == %s\n", s1,s2);
}
```

Funcția *int strcmp* (char *psir1, char *psir2, int n); realizează compararea doar pentru maxim primele *n* caractere ale celor două șiruri.

Funcțiile *stricmp* și *strnicmp* sunt similare celor două funcții prezentate anterior doar că realizează comparația dintre șiruri considerând caracterele literă mică egale cu caracterele literă mare (case insensitive).

Pentru concatenarea lui *sir2* la *sir1*, precizate prin pointeri, putem folosi funcția *char *strcat*(char *sir1, char *sir2) sau una din cele două variante ale funcției *strcat*, descrise mai jos.

```
#include<string.h>
char *strcat (char *psir1, char *psir2)
{
    char *cap = psir1;
    psir1 += strlen(psir1);
    // salvare pointer initial
    // salt la sfirsit
```

```

while (*psir2)    *psir1 ++ = *psir2 ++;    // copiere propriu-zisa
*psir1 = '\0';    // punere terminator
return cap;
}

```

sau:

```

char * strcat (char* psir1, char* psir2)
{
    strcpy(psir1 + strlen(psir1), psir2); // cu functie de biblioteca
    return psir1;
}

```

Șirul destinație trebuie să fie astfel dimensionat încât să poată conține și sursa și destinația, concatenate! Concatenarea doar a n elemente din *sir2* la *sir1* permite un control (pe maxim) al lungimii preluate din *sir2* și se efectuează cu funcția:

```
char * strncat (char *psir, char *psir2, int n);
```

Tot în domeniul prelucrării șirurilor operează și funcțiile de mai jos:

`char * strchr (char *psir, int c)` returnează pointer spre prima apariție în șir a caracterului *c*;

`char * strrchr (char *psir, int c)` returnează pointer spre ultima apariție în șir (cea mai din dreapta) a caracterului *c*;

`char * strpbrk (*char *psir1, char *psir2)` returnează pointer spre prima apariție în *sir1* a oricărui caracter din componența lui *sir2*;

`char * strtok (char *psir, char * pdlm)` returnează adresa fragmentelor de șir, obținute prin împărțirea lexicografică a șirului cu ajutorul unui delimitator, dintr-o mulțime de delimitatori posibili.

Funcția este frecvent apelată la construirea unor analizoare sintactice. Pentru înțelegerea exactă a efectului funcției *strtok* se consideră exemplul următor:

```

#include<stdio.h>
#include<string.h>
void main()
{
    char sir[]="sir de spart, in atomi; pot fi cuvinte!", dlm[ ]=".,; !?";
    char *psir=sir, *pssir=psir;
    while( psir =strtok(psir,dlm) )
    {
        printf("\n%s",psir);
        psir=(char*)NULL;
    }
}

```

La prima apelare *psir* va conține adresa șirului de analizat. La următoarele apeluri *psir* va conține valoarea NULL, pentru a specifica faptul că se continuă analiza de la elementul din șir la care a rămas la apelul

precedent. Funcția "știe" la ce adresă a rămas, grație unei variabile locale statice, care datorită clasei de memorie *static* păstrează conținutul între două apeluri succesive. Programul afișează pe câte un rând fiecare cuvânt din text, datorită delimitatorilor propuși, care coincid cu ceea ce înțelegem în mod uzual prin "separatori" de cuvinte. Modificând conținutul vectorului *dln* putem controla împărțirea textului în tipurile de entități dorite.

Funcția *int strspn* (*char * sir1*, *char * sir2*) numără de la începutul șirului *sir1*, caracterele consecutive, care se regăsesc și printre elementele lui *sir2*.

Spre deosebire de ea, funcția *strcspn* (*char *sir1*, *char *sir2*) are efect "complementar", în sensul că numără caracterele consecutive din *sir1*, până la găsirea unui caracter ce nu se află și în *sir2*.

De exemplu, programul:

```
# include <string.h>
# include <stdio.h>
void main( )
{
    char *sir1, sir2[]="abcdefghijk";
    sir1 = "calculator personal";
    printf ("\nlg. segment cu car. existente in sir2: %d", strspn (sir1, sir2));
    printf ("\nlg. segment cu car. ce nu exista in sir2:%d", strcspn (sir1,sir2));
}
```

va afișa numărul 2, ca lungime a fragmentului de șir "ca" format cu litere existente în "calculator personal" și în "abcdefghijk", respectiv 0, la apelul lui *strcspn*, deoarece *sir1* începe cu o literă ce se află și în *sir2*!

La un apel:

```
printf ("\nlg. segment cu car. ce nu exista in \"s\":%d", strcspn (sir1,"s"));
```

se va afișa numărul 14, ca fiind lungimea fragmentului de șir "calculator per", format cu litere care nu se regăsesc în componența celui de-al doilea argument, alcătuit doar din litera s.

Funcția *char *strstr*(*char **, *char **) realizează una dintre cele mai frecvente operații cu șiruri de caractere și anume căutarea unui subșir într-un șir dat, întorcând pointer la prima sa apariție. Viteza de execuție a acestei funcții este esențială în performanța căutărilor dintr-o bază de date sau pentru editoarele de text, când se lucrează cu fișiere mari. Majoritatea implementărilor funcției *strstr*() folosesc algoritmul **Boyer-Moore** (ACM, no. 20/oct. 1977) pentru găsirea unui subșir într-un șir dat. Pentru o bună înțelegere și eventuale aplicări specifice ale acestui algoritm, dăm principalii pași și o implementare a acestuia.

1. se numerotează literele subșirului 0,...,n
2. se caută direct pe poziția finală:
 - dacă se găsește o literă ce nu aparține subșirului se sar din nou n

caractere;

- dacă litera există în subșir, pe poziția *poz*, se sar încă *n-poz* litere:
 - dacă finalul coincide se fac și comparațiile între cele două poziții
 - altfel, se reia algoritmul de pe ultima poziție explorată.

O posibilă implementare a acestui algoritm ar putea fi următoarea:

```
#include<stdio.h>
#include<string.h>
#define Abs -1
#define CSS 255
void main()
{
    int pozsub(char*,int,char*,int,int []),vec[CSS],ls,lss;
    void contl(char*,int[]);
    char s[80],ss[80],*pss;
    printf("\n Dati sirul de caractere:\n"); gets(s);
    printf("\n Dati subsirul:\n"); fflush(stdin); gets(ss);
    pss=ss; ls=strlen(s); lss=strlen(ss); contl(pss,vec);
    printf("\n Poz de inceput= %d",pozsub(s,ls,pss,lss,vec));
}

int pozsub(char *sir,int lsir,char *subs,int lsubs,int contl[])
{
    int res=Abs,i,possib,primcar;
    char carfin=subs[lsubs-1],ccar,*ps,*pss;
    if(lsir>=lsubs)
    {
        if(lsubs==1)
            for(i=0;i< lsir && res == Abs;++i)
                if(carfin==sir[i])res=i;
        else
        {
            i=lsubs-1;
            do
            {
                if(((possib=contl[ccar=sir[i]])==Abs) i+=lsubs;
                else if(ccar==carfin)
                {
                    primcar=i-lsubs+1;
                    if(strncmp(&sir[primcar],subs,lsubs)==0) res = primcar;
                    if(res==Abs) ++i;
                }
                else i += lsubs - possib-1;
            } while(i<lsir&&res==Abs);
        }
    }
    return(res);
}
```

```
void contl(char *subs,int cont [])
{
    int i;
    for(i=0; i <= CSS-1; i++) cont [i] = Abs;
    for( i=0; *subs ; ++i, ++subs) cont [*subs] = i ;
}
```

Dati sirul de caractere:

adavindsc dsc

Dati subsirul:

vin

Pozitia de inceput= 3

Funcțiile de bibliotecă pentru lucru cu șiruri de caractere își găsesc o largă aplicabilitate în validarea datelor.

Să se determine frecvența de apariție a caracterelor dintr-un șir de caractere introduse de la tastatură.

```
#include <stdio.h>
void main()
{
    int ascii[256],i;
    char sir[100],*ps=sir;
    for(i=0;i<256;i++) ascii[i]=0;
    printf("\n Dati sirul:"); gets(sir);
    while(*ps) ascii[*ps++]++;
    for(i=0;i<256;i++)
        if(ascii[i]) printf("\n Caract %c cu frecventa %d",i,ascii[i]);
}
```

7.1 Conceptul de funcție și transferul parametrilor

Implementarea structurilor fundamentale de programare într-o formă pură, precum și posibilitatea de a apela funcții, conferă programelor scrise în limbajul C un nivel înalt de modularizare și structurare. Însuși programul principal este tratat ca funcție (*main*), putând avea sau nu parametri de intrare.

Numărul mic de instrucțiuni este completat cu numeroase funcții din biblioteca standard a limbajului.

Prin însușirea corectă a operațiilor pe care le efectuează funcțiile, a modului de apelare și a naturii rezultatelor returnate, programatorul în limbajul C poate elabora programe cu un înalt grad de compactizare, la nivele ridicate de complexitate a prelucrărilor.

Funcția reprezintă cea mai mică entitate C compilabilă separat; ea dispune de un nume, tip returnat și de o listă de parametri.

O funcție are deci forma:

```
tip nume (lista parametri)
{
    declarații + instrucțiuni + apeluri de funcții;
}
```

Funcția reprezintă modul de organizare a unui program C, delimitat pe domenii de competență, în sensul că un fragment de program are cunoaștință și acces la un alt fragment de program sau date.

În C, conceptul de funcție corespunde celui de *function*, din teoria programării, în sensul că poate returna cel mult o valoare, nemaexistând varietățile *subroutine*, sau *procedure*, ca în alte limbaje.

Declararea unei funcții trebuie făcută în funcția care o apelează sau la nivel global și se face prin specificarea tipului funcției, a numelui și a tipului parametrilor de apel. Acest lucru se mai numește și prototipul funcției. Declararea nu este obligatorie dacă funcția a fost definită înainte de a fi apelată.

În general, se face distincție între *declararea* unei funcții (adică precizarea prototipului funcției) și *definirea* ei, adică descrierea codului specific funcției. Parametrii precizați la definirea funcției se numesc *parametri formali*. Definirea se realizează în afara oricărei alte funcții.

Apelul unei funcții se distinge prin aceea că el devine instrucțiune și se termină cu “;”. În acest caz, se spune că funcția primește *parametri actuali*.

De cele mai multe ori, codul funcției nu este dat de programator, ci face parte din bibliotecă, astfel încât programele descriu doar prototipul funcțiilor folosite, sau citează în *#include* un fișier antet conținând prototipul funcțiilor folosite.

Transmiterea parametrilor se face prin tehnica **call by value** (apel prin valoare) ceea ce revine la crearea unor duplicate ale parametrilor de apel, valorile originale nefiind alterate pe parcursul derulării funcției. Rezultatul este transmis de funcție prin instrucțiunea: `return expr;`. Se poate deduce de aici că o funcție returnează cel mult un rezultat.

În cazul în care se dorește ca funcția să opereze chiar asupra originalului variabilelor din funcția apelatoare, se impune utilizarea tehnicii **call by reference** (apel prin referință). Ea presupune ca funcția să primească adresele parametrilor. Acest lucru se poate realiza în două moduri: să se primească o *referință* a variabilei sau să se primească un *pointer* la variabilă.

În ambele situații prelucrările din funcții modifică valorile parametrilor actuali, din funcția apelatoare. Diferența între transferul prin adresă și cel prin referință este în principal de ordin sintactic, la transferul prin referință nemaioperându-se direct cu adresa parametrului, dereferirea făcându-se automat.

O altă diferență, majoră de această dată, apare atunci când la apel sunt efectuate conversii pentru adaptare la prototipul funcției. La transferul prin adresă conversia de pointeri este mult mai restrictivă, așa cum s-a văzut la aritmetica de pointeri. La transfer prin referință, conversiile generează obiecte temporare a căror referință se va transmite funcției; în acest caz, deși transferul e prin referință, prelucrările se efectuează pe originalul obiectului temporar rezultat din conversie și nu pe originalul din funcția apelatoare. Versiunile mai noi de compilatoare restricționează și conversiile de referințe, evitând astfel anomaliile de mai sus.

Dacă se dorește evitarea transferului de parametri și întoarcerea rezultatelor în / din funcție se pot utiliza variabilele globale.

Exemplu:

Să se scrie o funcție ce realizează adunarea a două numere folosind modalitățile de transfer ale parametrilor prezentate mai sus.

a) Funcția va primi cele două numere prin valoare și va returna rezultatul:

```
int add1(int a, int b)
{    return a+b;    }
```

b) Funcția va primi cele două numere prin valoare și o referință la rezultat. Ea nu va returna nimic:

```
void add2(int a, int b, int &c)
{
    c=a+b;
}
```

c) Funcția va primi cele două numere prin valoare și un pointer la rezultat. Ea nu va returna nimic.

```
void add3(int a, int b, int *c)
{
    *c=a+b;
}
```

d) Funcția nu va primi nici un parametru, dar va returna rezultatul:

```
int add4() { return x+y; }
```

x și y fiind variabile globale, definite anterior în același fișier sau în fișiere diferite caz în care declarația de *extern* apare obligatoriu.

e) Funcția va primi cele două numere și va returna adresa unei variabile statice care va stoca rezultatul adunării:

```
int *add5(int k, int t)
{
    static int c;
    c=k+t;
    return &c;
}
```

Cele cinci definiții de funcții vor fi apelate în următorul program principal:

```
# include <stdio.h>
int x,y;
void main ( )
{
    int a=4, b=7, add1(int, int), add4(), *add5(int,int);
    void add2(int, int, int &), add3(int, int, int*);
    printf("\n %d + %d = %d",a,b,add1(a,b));
    printf("\n %d + %d = %d",a,b,*add5(a,b));
    int c =5, d =2, e;
    add2(c,d,e);
    printf("\n %d + %d = %d",c,d,e);
    add3(c,e,&d);
    printf("\n %d + %d = %d",c,e,d);
    y=7, x=9;
    printf ("\n %d+%d=%d",x,y,add4());
}
```

La cele prezentate până acum se mai impun unele observații:

- masivele se trimit în funcție implicit prin adresă. Dacă se dorește transmiterea unui masiv prin valoare, atunci trebuie ca el să fie inclus într-o structură, ce va fi transmisă funcției prin valoare;

- parametri trebuie să concorde în general, din punct de vedere al numărului, ordinii și tipului cu definirea și cu declararea prototipului

funcției; între anumite limite, compilatorul efectuează conversii ale parametrilor de intrare pentru adaptare la prototipul funcției a apelului ei;

- există posibilitatea în limbajul C de a construi funcții ce lucrează cu un număr variabil de argumente;

- parametrii de intrare în funcție pot fi folosiți pentru inițializarea variabilelor locale aparținând funcției;

- s-a precizat că o funcție poate returna cel mult o valoare. Pentru a regăsi mai multe rezultate ale prelucrărilor, există posibilitatea de a grupa datele de ieșire într-un vector sau structură și de a returna adresa vectorului sau structurii.

O altă modalitate se bazează pe folosirea variabilelor globale, ele fiind văzute în toate funcțiile pot participa la calcule direct.

Cea de-a treia modalitate presupune transmiterea prin adresă sau referință a unor parametri, unde se vor regăsi implicit rezultate ale prelucrărilor din funcții.

O funcție poate returna și o referință; referința poate fi a unei zone locale *static* sau a unei zone din afara funcției, caz în care funcția a primit o referință și o returnează mai departe. De remarcat că, în primul caz practic funcția transmite altor funcții dreptul de scriere / citire asupra variabilelor sale locale. Devin corecte sintactic, în acest caz, expresii precum:

```
++f();           f()++;           f()+=g();
```

În cel de-al doilea caz, funcția prelucrează și deține informația de la / la referința primită și returnează referința pentru a se menține continuitatea prelucrării.

Exemplu:

```
# include <stdio.h>
int &f()
{
    static dep=1;
    printf("\n Depozit =%d",dep);
    return dep;
}
void g()
{
    f()++;
}
void main()
{
    for(int i=1; i<=5; i++) g();
}
```

Se observă că deși variabila *dep* (depozit) e locală, incrementarea variabilei *dep* (depozit) este făcută de apelul funcției *g()*.

Pentru a exemplifica diferite modalități de construire și apel a funcțiilor, se vor rezolva problemele:

A. Se va scrie programul care afișează în binar o zonă de memorie. Se vor construi două funcții:

- *afis_b* pentru afișarea în binar a unui bait;
- *afis_z* pentru afișarea unei zone de memorie identificată prin adresa de început și lungime; ea va apela funcția *afis_b*.

```
#include <stdio.h>

// functia care afiseaza un bait in binar, urmat de .
void afis_b(char b)
{
    unsigned char m=0x80;
    while(m) printf("%d",b&m ? 1 : 0 ), m>>=1;
    putchar('.');
}

// functia care afiseaza o zona de memorie data de adresa ei (pz)
// si de lungimea ei in baiți (lg)
void afis_z(void *pz,int lg)
{
    void afis_b(char);
    char *p=(char*)pz;
    while(lg--) afis_b(*p),p++;
}

void main()
{
    void afis_z(void *,int);
    long a= -10;
    short si=16;
    double t=23.67;
    afis_z(&a,sizeof(long));
    putchar('\n');
    afis_z(&si,sizeof(short));
    putchar('\n');
    afis_z(&t,sizeof(double));
}
```

Această problemă constituie un mijloc de a vizualiza modul de dispunerea a octeților în memorie pentru diferite tipuri de date precum și modul de reprezentare a datelor în calculator.

B. Să se scrie funcția care va aduna k matrice pătratice de dimensiune n , ($n \leq 50$).

Numărul de matrice fiind variabil, funcția va primi un vector de pointeri la matrice, numărul efectiv de matrice și dimensiunea unei matrice. Rezultatul fiind tot o matrice, el se va returna prin intermediul unui pointer la matrice.

```
double (*ad_matr(double (*vpm[][50])[50], int n, int k)) [50][50]
{
    static double rez[50][50];
    for( int i=0,j,t;i<n;i++)
        for(j=0;j<n;j++)
        {
            rez[i][j]=0;
            for(t=0;t<k;t++) rez[i][j]+=(vpm[t][i][j]);
        }
    return &rez;
}
```

Funcția *main()*, în care se apelează funcția *ad_matr()* este:

```
void main ( )
{
    double a[50][50]={ {1,2,3}, {6,2,0}, {4,3,2} },
           b[50][50]={ {1,1,1}, {10,10,10}, {0,0,1} };
    double (*vm[][50])[50] = { &a, &b }, (*prez)[50][50];
    double (*ad_matr(double (*[])[50][50], int , int ))[50][50];
    int dim=3,k=sizeof(vm)/sizeof(vm[0]);
    prez=ad_matr(vm,3,2);
    for(int i=0,j;i<3;i++)
    {
        putchar('\n');
        for(j=0;j<3;j++) printf("%7.2lf ",(*prez)[i][j]);
    }
}
```

De reținut descrierea unei funcții (*f*) care returnează un pointer la un masiv, ce are forma:

tip (*(lista_parametri) [dim₁][dim₂].....[dim_k]

Atunci când se returnează o adresă de variabilă locală, definită în funcție (în exemplu, matricea *rez*), variabila trebuie să fie de clasă *static*, pentru a nu dispărea la ieșirea din funcție.

C. Să se scrie o funcție pentru căutarea binară a unui element într-un vector sortat crescător. Să se folosească funcția de căutare de mai sus pentru sortarea, prin inserție, a unui vector.

```
#include<stdio.h>
int caut_bin(int v[],int n,int x,int *ppoz)
{
    int i=0,s=n-1,m;
    while(i<=s) /* obligatoriu si egal ! */
    {
        m=(i+s)/2;
        if(v[m]==x)
        {
            *ppoz=m; return 1; }
        v[m]<x ? i=m+1 : s=m-1;
    }
}
```

```

    *ppoz=i; // pozitia pe care trebuia sa se afle
    return 0;
}

void insertie(int v[],int n,int x)
{
    int i,poz;
    caut_bin(v,n,x,&poz);
    for(i=n-1;i>=poz;i--) v[i+1]=v[i];
    v[poz]=x;
}

void main()
{
    int v[30],i,n;
    printf("\nNr. de elemente din vector:");
    scanf("%d",&n);
    for(i=0;i<n;i++) printf("v[%d]=",i+1), scanf("%d",&v[i]);
    for(i=1;i<=n-1;i++) insertie(v,i,v[i]);
    for(i=0;i<n;i++) printf("\nv[%d]=%d",i+1,v[i]);
    getchar();
}

```

Pentru a răspunde ambelor cerințe, funcția de căutare binară a fost elaborată astfel încât nu returnează poziția elementului căutat în vector, ci o valoare logică, găsit sau negăsit; poziția unde a fost găsit sau unde ar fi trebuit să fie (ultima poziție unde a fost căutat) este returnată implicit prin ultimul parametru, transferat prin adresă. Această poziție este utilă în funcția de sortare, pentru că arată unde trebuie inserat elementul în vectorul sortat, pentru ca acesta să-și păstreze ordinea de sortare.

D. Să se scrie o funcție care să calculeze elementul minim și maxim dintr-un vector de întregi.

```

#include<stdio.h>
int *min_max(int x[],int n)
{
    int *v=new int[2];
    for(i=1,*v=v[1]=x[0];i<n;i++)
        if(x[i]<v[0]) v[0]=x[i];
        else if(x[i]>v[1]) v[1]=x[i];
    return v;
}

void main()
{
    int t[]={7,4,9,1,5},n=sizeof(t)/sizeof(int),*pi;
    pi=min_max(t,n);
    printf("\n Elementul minim= %d, maxim =%d",*pi,pi[1]);
    delete [2]pi;
}

```

A se observa maniera de construire a funcției care returnează adresa unei zone de memorie, alocată dinamic în funcție și care stochează pe prima poziție elementul minim, iar pe a doua elementul maxim. Memoria alocată dinamic are caracter static, în sensul că, zona de memorie se păstrează până la terminarea programului sau până se face explicit eliberarea ei.

E. Legat de modul de definire a funcțiilor și a lucrului cu pointeri, există în limbajul C posibilitatea definirii de **funcții cu un grad mare de generalitate**.

Spre exemplificare, se va construi funcția de căutare a unui element într-un vector dându-se ca parametri:

- adresa de început a vectorului, ca pointer la *void*;
- numărul de elemente;
- mărimea în baiți a unui element;
- adresa elementului de căutat, ca pointer la *void*.

Funcția va returna prima poziție în care a fost găsit elementul (se consideră că primul element din vector ocupă poziția 0), respectiv -1 dacă acesta nu există în vector.

Din descrierea parametrilor de apel se poate ușor deduce că această funcție lucrează pe vectori cu elemente de diferite tipuri de date. Se va apela funcția pentru vectori de: numere întregi, reale (*double*), cât și pentru un șir de caractere.

```
#include <stdio.h>
#include <memory.h>
#include <string.h>

int cauta(void *v,int lg,int size_el,void *el)
{
    char *ec=(char*)v,*esf=(char*)v+lg*size_el;
    for(int i=0;ec<=esf;ec+=size_el,i++)
        if(!memcmp(ec,el,size_el)) return i;
    return -1;
}

void main()
{
    int a[]={3,6,4,1,8},t=4,k;
    (k=cauta(a,sizeof(a)/sizeof(int),sizeof(int),&t))== -1 ?
        printf("\nElement inexistent!!!") : printf("\nElementul pe pozitia %d",k);
    double b[]={3.5,6.2,4.9,28.15},el_r=6.2;
    (k=cauta(b,sizeof(b)/sizeof(double),sizeof(double),&el_r))== -1 ?
        printf("\nElement inexistent!!!") : printf("\nElementul pe pozitia %d",k);
    char *sir="Sir de caractere!!!", el_c='d';
    (k=cauta(sir,strlen(sir),sizeof(char),&el_c))== -1 ?
        printf("\nElement inexistent!!!") : printf("\nElementul pe pozitia %d",k);
}
```

Generalitatea acestei construcții constă în faptul că funcția poate lucra pe masive de diferite tipuri de date fără a se face apel la o facilități implementată în limbaj. Se observă că la apel, deși nu se transmite un pointer la un tip explicit, acesta nu trebuie convertit explicit la tipul *void*.

Facilitățile limbajului C în ce privește lucru cu funcții și cu pointeri, au permis cristalizarea unui nou mod de programare, cunoscut sub numele de *programare generică*.

7.2 Clase de funcții în limbajul C

Spre deosebire de alte limbaje, C/C++ se distinge printr-un număr imens de funcții de bibliotecă. Fișierele de tip h (**header**) oferă o imagine generală asupra principalelor clase de funcții grupate după domeniul prelucrărilor efectuate: prelucrarea fișierelor, manipularea de șiruri, conversii, lucru cu zone de memorie, calcule matematice, alocare dinamică de memorie, grafică etc.

a) funcțiile de *lucru cu fișiere* au prototipul în *stdio.h*, *io.h* și *fcntl.h* pentru fișiere pe medii magnetice și fișiere standard: tastatura, monitor, imprimantă; aceste funcții vor fi detaliate în capitolul de lucru cu fișiere.

b) prelucrările specifice *șirurilor de caractere* au prototipul în *string.h* și au fost detaliate în capitolul 6.

c) principalele *funcții de conversie* își au prototipul în *stdlib.h* cele mai importante fiind:

- `int atoi( const char *sir );` pentru conversie din ASCII în *int*, parametrul este un șir de caractere (*sir*), funcția întoarce întregul rezultat din conversie;
- `long atol( const char *sir );` realizează conversia din ASCII în *long*;
- `double atof( const char *sir );` realizează conversia din ASCII în *double*;
- `char *itoa( int val, char *sir, int baza );` pentru conversie din *int* la ASCII, funcția primește ca parametri valoarea de convertit (*val*), baza de numerație pentru *val* (*baza*) și va stoca rezultatul conversiei în șirul *sir*;
- `char *ltoa( long val, char *sir, int baza );` realizează conversia din *long* în ASCII;

d) funcțiile de *lucru cu zone de memorie* au prototipul în *mem.h* sau *memory.h*:

- `void *memcpy( void *dest, const void *sursa, size_t cont );` realizează copierea a *cont* baiți de la *sursa* la *dest*;

- `int memcmp( const void *z1, const void *z2, size_t cont );` realizează comparea a *cont* baiți între zonele de memorie *z1* și *z2*; funcția returnează un întreg cu semnificațiile: mai mic ca 0, pentru $z1 < z2$, egal cu 0 pentru $z1 = z2$ și mai mare ca 0 când $z1 > z2$;
- `void *memset( void *dest, int c, size_t cont );` realizează setarea a *cont* baiți de la zona de memorie *dest* cu caracterul *c*;
- `void *memchr( const void *buf, int c, size_t cont );` funcția returnează adresa primei apariții a caracterului *c* din șirul *buf* pe o lungime de *cont* baiți, sau NULL dacă caracterul *c* nu există;

e) *math.h* conține prototipul principalelor funcții și constante matematice. Regăsim aici rutine pentru exponențiere, extragere rădăcină pătrată, logaritmare, determinarea valorii absolute (modulul unui număr), funcțiile trigonometrice directe și inverse, funcții pentru controlul erorilor matematice:

- `int abs( int n );` valoarea absolută a numărului întreg *n*;
- `double fabs( double x );` valoarea absolută a numărului real *x*;
- `double sqrt( double x );` rădăcina pătrată a numărului real *x*;
- `double log( double x );` logaritm natural din *x*;
- `double log10( double x );` logaritm în baza 10 din *x*;
- `double pow( double x, double y );` calculează x^y ;
- `double exp( double x );` calculează exponențialul e^x .

Fișierul *complex.h* conține de asemenea funcții matematice, dar specializate în lucru cu numere complexe.

f) pentru *alocarea dinamică a memoriei* există o varietate mare de funcții diferențiate după tipul memoriei dinamice în care se face alocarea (*heap*, *farheap*), inițializările operate asupra zonei alocate, modalitățile de paginare a memoriei etc. Fișierele ce conțin prototipurile sunt: *alloc.h* și *stdlib.h*.

- `void *malloc( size_t nr );` realizează alocarea unei zone de memorie de lungime *nr* baiți;
- `void *calloc( size_t nr, size_t dim_el );` realizează alocarea unei zone de memorie de lungime capabilă să stocheze *nr* elemente, fiecare având *dim_el* baiți; zona de memorie va fi inițializată cu 0;
- `void *realloc( void *memb, size_t dim );` realizează realocarea unei zone de memorie identificată prin *memb*, având noua dimensiune dată de *dim*. Când *dim* este mai mare decât dimensiunea cerută în prealabil, funcția alocă memorie în continuare sau mută integral conținutul memoriei în altă parte unde zona poate fi extinsă la noua dimensiune;

g) funcții specifice *gestiunii timpului și datei* sistem (*time.h*).

h) funcții de *gestionare a resurselor Windows* cu prototipul în fișierul *header windows.h*.

7.3 Funcții recursive

Recursivitatea este o caracteristică foarte frecventă a algoritmilor. Ea definește modalitatea de a determina valoarea unei variabile pe baza valorilor ei anterioare și eventual a altor valori cunoscute.

O parte din limbajele de programare asigură implementarea algoritmilor recursivi prin intermediul structurilor repetitive, folosind atât masive pentru memorarea rezultatelor fiecărei iterații, cât și variabile temporare de lucru.

Limbajul C permite descrierea algoritmilor recursivi prin intermediul funcțiilor care se apelează pe ele însele, direct sau prin intermediul altor funcții.

De asemenea în limbajul C se pot aborda mai ușor algoritmi recursivi, deoarece există posibilitatea de a defini și structuri autoreferite de date.

Vom exemplifica recursivitatea folosind o problemă simplă, de calculul sumei elementelor unui vector.

Pentru vectorul: $A = (a_0, a_1, \dots, a_{n-1})$, suma elementelor se poate descrie recursiv:

$$S_n = \begin{cases} 0, & \text{daca } n = 0 \\ S_{n-1} + a_n, & \text{daca } n > 0 \end{cases}$$

Exemplu:

```
double sum(double v[], int n)
{
    return n ? v[n-1]+sum(v,n-1) : 0;
}
```

O atenție deosebită trebuie acordată controlului finitudinii algoritmului, cât și numărului de apeluri, deoarece apar restricții datorate dimensiunii stivei. Pentru aceasta în componența funcției recursive distingem totdeauna condiția de terminare a procesului recursiv și modalitatea prin care se modifică valoarea argumentelor la apeluri succesive. Practic, prin apelul recursiv, funcția este împărțită în două: o porțiune care se execută repetat cu încărcarea pe stivă a contextului și a adresei de revenire și o porțiune a cărei execuție repetată începe numai după

ce s-a atins nivelul maxim de profunzime a recursivității (impus prin condiția de terminare).

Un exemplu sugestiv în acest sens îl constituie funcția care realizează afișarea unui număr a , din baza 10 într-o bază b , ($b < 10$). Pentru afișare, trebuie să se obțină resturile împărțirilor succesive la bază în ordine inversă, de aceea afișarea cifrei se va face după apelul recursiv.

```
void afis(int a, unsigned b)
{
    if(a < b) printf("%d", a);
    else
    {
        afis(a/b, b);
        printf("%d", a % b);
    }
}
```

Implicațiile pe care le au clasele de memorie asociate variabilelor locale din funcție sunt majore; astfel, variabilele automate există în câte un set pentru fiecare apel recursiv, în timp ce variabilele statice și externe există într-un singur exemplar pentru toate apelurile. Modificările dintr-un apel nu se regăsesc la revenire în cazul variabilelor automate, în timp ce variabilele statice reflectă efectul cumulat al tuturor modificărilor anterioare.

Condiția pentru controlul recursivității se poate baza pe modificarea argumentelor de apel, pe o variabilă statică al cărei conținut se păstrează între două apeluri succesive, sau pe o variabilă globală. Spre exemplificare, funcția de mai jos afișează pătratele numerelor de la 10 la 1, o singură dată. Un eventual al doilea apel, va găsi valoarea lui n ca fiind 0, iar recursivitatea nu se va produce, deoarece inițializarea unei variabile statice se face o singură dată, la compilare.

```
void f()
{
    static int n=10;
    if(n)
    {
        printf("\n patratul lui %d este %d", n, n*n);
        n--;
        f();
    }
}
```

Simplitatea descrierii recursive a unui algoritm poate ascunde calcule repetate nejustificat; spre exemplu o implementare a algoritmului de determinare a termenului de rang n al șirului lui Fibonacci dat de relația:

$$f_n = \begin{cases} n, & \text{daca } n = 1 \text{ sau } n = 2 \\ f_{n-1} + f_{n-2}, & \text{daca } n > 2 \end{cases}$$

sub forma funcției:

```
long fibo(long n)
{
    if(n<=2) return n;
    else return fibo(n-1)+fibo(n-2);
}
```

este total ineficientă, deoarece ascunde calculul repetat al unor termeni. Cauza o reprezintă faptul că o funcție C returnează o singură valoare, iar recurența șirului necesită cunoașterea a doi termeni anteriori. O implementare iterativă a algoritmului este mult mai rapidă:

```
long fibo(long n)
{
    long Xn,Xn_1=2,Xn_2=1;
    for (int i=2;i<n;i++)
    {
        Xn=Xn_1+Xn_2;
        Xn_2=Xn_1;
        Xn_1=Xn;
    }
    return n<=2 ? n : Xn;
}
```

7.4 Pointeri la funcții

Conceptul de pointeri la funcții este strâns legat de posibilitatea transferului unei funcții ca parametru într-o altă funcție. Această nevoie apare destul de frecvent atunci când vrem să scriem o rutină cât mai generală, care să poată fi particularizată apelând la rândul ei un cod executabil pe care nu-l cunoaștem încă, la momentul scrierii rutinei.

Spre exemplu, dorim să dăm o implementare a unui algoritm de integrare numerică pe care s-o putem folosi pentru orice funcție integrabilă, a cărei expresie analitică n-o avem încă.

Un alt exemplu este scrierea unei rutine pentru reprezentarea grafică a unei funcții, pixel cu pixel. Evident, rutina trebuie să fie cât mai generală, urmând ca funcția de reprezentat grafic s-o dăm ca parametru în funcția de reprezentare grafică.

Putem aminti în același sens, elaborarea unei funcții pentru determinarea rădăcinilor unei ecuații prin metoda biseției, ecuație dată ca parametru, cu ajutorul unui pointer la o funcție matematică; și exemplele ar putea continua, chiar fără a ne cantona în domeniul matematicii, ales aici

pentru similitudinea, ușor de sesizat, dintre conceptul de funcție din programare și funcția în accepțiune matematică.

Înainte de a realiza efectiv un exemplu, să ne familiarizăm cu principalele operații legate de pointerii de funcții.

Declararea unui pointer de funcție se face după modelul oferit de prototipul funcțiilor ce vor fi *pointate* cu acest pointer:

tip_returnat (*nume)(*tip_parametri*);

Se observă că în afara parantezelor specifice oricărei funcții, și care încadrează parametrii de intrare ai acesteia, a fost nevoie de încă o pereche de paranteze pentru a mări prioritatea operatorului *. Astfel, o descriere de forma *double* (*pf)(int, float) se va citi: *pf este pointer la o funcție care primește parametri de tip int și float și returnează un double*. Acest lucru este asigurat de faptul că cele două perechi de paranteze au aceeași prioritate (nivel 1 de prioritate) și se evaluează de la stânga la dreapta, deci *pf* este pointer. În lipsa primei perechi de paranteze, *pf* ar fi funcție care primește parametri de tip *int* și *float* și returnează un *pointer* la rezultat *double*.

Încărcarea pointerului se face indicând o funcție care are prototipul impus la declararea pointerului. Procesul este similar încărcării unei variabile cu o constantă, deoarece singura deosebire între funcție și un pointer de funcție este cea dintre o constantă și o variabilă. Mai precis, funcția este un pointer, dar constant (adresa fixată la linkeditare pentru codul executabil referit prin numele funcției), pe când pointerul este o variabilă, ce poate conține la momente diferite, adrese diferite ale unor funcții ce respectă același prototip. Dacă *f* este o funcție și *pf* un pointer de funcție, atunci încărcarea pointerului este o atribuire de forma:

pf = *f* ;

După cum se observă, nu apar paranteze nici la pointer și nici la funcție. Dacă după numele funcției ar apare paranteze, s-ar interpreta ca apel de funcție, iar atribuirea s-ar face cu ce returnează funcția, lucru care ar fi greșit.

Folosirea pointerului de funcție pentru a apela funcția:

(*pf)(*p*₁,*p*₂...*p*_n);

De această dată, parantezele reapar, atât pentru pointer pentru a preciza că este vorba de un apel indirect de funcție, cât și pentru a încadra parametrii de apel ai funcției.

Reunite la un loc, într-un exemplu coerent, toate operațiile de mai sus se prezintă astfel:

#include <stdio.h>

```

int f( float x, float y)
{
    int r = 0;
    /* instructiuni */
    return r;
}

void main( )
{
    int a;
    int (*pf)( float, float);
    pf = f;
    a = (*pf)(1,2);
    printf( "\nRezultat = %d", a);
}

```

O întrebare care s-ar pune este: de ce să folosim pointerul, când puteam foarte bine folosi apelul direct al funcției, sub forma $a = f(1,2)$; răspunsul este puțin mai subtil și nu-l putem găsi în exemplul simplist de mai sus. Folosim pointerul de funcție în locul funcției atunci când nu știm dinainte ce funcție vom apela și vom încărca în pointer adresele unor funcții diferite, în raport de ramura logică pe care se derulează programul până la acel moment. Partea de folosire a pointerului este aceeași, indiferent de adresa funcției pe care o conține și nu se justifică repetarea condițiilor de selecție pentru a putea apela direct funcțiile. În felul acesta obținem programe care se reconfigurează la execuție, fără a da din compilare structura alternativă ce se execută, ci mai degrabă se descrie cum se modifică programul în diverse contexte de rulare.

Așa cum se va vedea la programarea orientată obiect, pointerii la funcții stau la baza mecanismului de *legare întârziată*, adică de amânare a fixării funcției de apelat, de la momentul linkeditării până la momentul execuției, când pointerul de funcție se încarcă cu o adresă de funcție.

Pentru a completa domeniul aplicațiilor pointerilor de funcții, ne vom opri la un exemplu de sortare. Observăm că orice metodă de sortare am folosi, totdeauna vom avea de executat repetat comparații și interschimbări, când nu se respectă ordinea de sortare. Ne propunem să scriem o rutină de sortare generală, care să poată fi folosită și pentru sortare crescătoare și pentru sortare descrescătoare, și pentru sortare lexicografică și pentru o sortare a cărei *ordine de sortare* este una specifică, indicată de utilizator. În acest scop, ordinea de sortare nu o mai includem direct în rutina de sortare, ci presupunem că ea este dată de funcții specifice, care primesc datele de intrare și întorc o valoare booleană, care indică respectarea sau nerespectarea acestei ordini. La rândul ei, rutina de sortare primește în afara datelor de sortat și un pointer la funcția ce dă ordinea de sortare dorită, permițând astfel generalizarea.

```

#include <stdio.h>
#define TRUE 1
#define FALSE 0
int asc( int x, int y)
{
    return x<y ? TRUE : FALSE;
}
int desc( int x, int y)
{
    return x>y ? TRUE : FALSE;
}
void comuta(int &a, int &b)
{
    int temp;
    temp = a; a=b; b=temp;
}
void sortare( int x[], int n, int (*sens)(int, int ) )
{
    register i,j;
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if( !(*sens)(x[i],x[j]) )
                comuta(x[i],x[j]);
}
void scrie( int v[], int n)
{
    printf("\n");
    for(int i=0; i<n; i++) printf(" %d ", v[i]);
}
void main()
{
    int x[]={ 2, -3, 10, 3, 54 }, n= sizeof(x)/sizeof(x[0]);
    sortare(x,n,asc);      scrie(x,n);
    sortare(x,n,desc);     scrie(x,n);
}

```

Se observă transferul prin referință folosit de funcția *comuta()*, pentru ca interschimbul să nu se facă între copii, ci între elementele originale ale vectorului.

Pointerul *sens* este un pointer de funcție prin care se controlează sensul sortării, iar interschimbul se face doar când sensul sortării nu este respectat de cele două elemente comparate.

Exemplul poate fi ușor adaptat pentru a rezolva și sortări alfabeticе, funcția *strcmp()* fiind invocată în funcția apelată prin pointerul *sens*, pentru a testa ordinea alfabetică a două șiruri de caractere.

Un alt avantaj major al apelării funcțiilor prin pointeri de funcții constă în faptul că pointerii pot fi organizați în masive, permițând apelarea simplă a unui set de prelucrări. Să ne amintim că frecvent avem nevoie pentru diferiți vectori de observație să aplicăm aceleași prelucrări statistice, de genul calculul mediei, dispersiei, coeficientului de variație, modulului etc., pentru care am elaborat câte o funcție. În mod normal, ar trebui să scriem setul de apeluri către toate funcțiile, pentru fiecare vector de observație în parte; mai elegant este să încărcăm în prealabil adresele tuturor funcțiilor într-un vector de pointeri la funcții, iar toate apelurile să fie reduse la un singur apel, pus într-un *for*.

Declararea unui *vector de pointeri la funcții* este o combinație între declararea unui masiv și declararea unui pointer de funcție:

```
double (*vpf [10]) ( double [], int);
```

Conform priorității evaluării operatorilor funcție și de indexare, *vpf* este în primul rând un vector (operatorul [] este prioritar operatorului *), dar de pointeri la funcții care primesc vectori de observații *double* cu dimensiunea lor și returnează un rezultat *double*.

De observat că în timp ce pentru vectorul de pointeri la funcții dimensiunea este obligatorie pentru a ști compilatorul câte elemente să-i aloce, pentru vectorul de date de intrare nici numele și nici dimensiunea nu sunt necesare, urmând ca ele să fie specificate la momentul execuției.

Se pot imagina ca date de intrare cifra de afaceri a unei firme pe mai mulți ani, iar adresele conținute de *vpf* vor fi ale unor funcții care determină media, dispersia, coeficientul de variație etc. pentru vectorul cifrei de afaceri.

```
#include <stdio.h>
#include <math.h>
double media(double x[], int n)
{
    double s=0.; int i;
    for(i=0;i<n;i++) s+=x[i];
    return s/n;
}
double dispersia(double x[], int n)
{
    double m, s=0.; int i;
    m=media(x,n);
    for(i=0;i<n;i++) s+= (x[i]-m)*(x[i]-m);
    return s/n;
}
double coef_var(double x[], int n)
{
    return sqrt(dispersia(x, n))/media(x,n);
}
```

```
void main()
{
    double x[]={ 10, 12.5, 13, 13, 14 };
    int n= sizeof(x)/sizeof(x[0]);
    double (*vpf[3])(double [],int);
    vpf[0]=media; vpf[1]=dispersia; vpf[2]=coef_var;
    char *indicator[]=
        {"Media: ", "Dispersia: ", "Coeficient de variatie: " };
    for(int i=0; i<3; i++)
        printf("%s %lf \n", indicator[i], (*vpf[i]) (x,n) );
}
```

Se pot organiza și datele de intrare în matrice de observație, iar rezultatele în vectori de indicatori, astfel încât programul să calculeze medii, dispersii etc. pentru mai mulți indicatori economici (număr de personal, venituri, cheltuieli etc.).

Ca orice vector, vectorul de pointeri la funcții putea fi inițializat la declarare, nemaifiind necesară precizarea dimensiunii, care se calculează din lista de inițializatori:

```
double (*vpf[ ])(double [],int)= { f1, f2, f3, f4 };
```

în care *f1*, *f2*, *f3*, *f4* sunt nume de funcții, declarate până la acest moment.

7.5 Argumentele funcției *main()*

Ca oricare altă funcție, funcția *main()* poate primi și returna valori. Argumentele de intrare provin din linia de comandă, când programul a fost lansat dintr-un sistem de operare orientat pe linie de comandă (DOS, UNIX) sau au fost date într-una din opțiunile mediului de programare, când programul se lansează dintr-un mediu dispunând de interfață grafică cu utilizatorul (în mediul Microsoft Visual C++ în meniul *Project / Settings / Debug*, iar în Borland C++ în meniul *Run*).

Prototipul general al funcției *main()* are forma:

```
void main ( int argc, char *argv[ ], char *envp[ ] )
```

Pentru a ne explica acest prototip trebuie avut în vedere că funcția *main()* primește argumentele de la sistemul de operare și într-un număr variabil. Aceste argumente sunt în format extern (șiruri de caractere separate cu spațiu), fiind gestionate printr-un vector de pointeri la *char*, **argv[]* (argument values) iar numărul lor este indicat de sistemul de operare prin primul parametru *argc* (argument count), de tip întreg.

Ultimul parametru este un vector de pointeri la variabilele de mediu (environment pointers) terminat printr-un pointer nul și este mai rar folosit, fiind o extensie Microsoft a standardului ANSI.

Așadar, sistemul de operare face un "pachet" cu argumentele de transmis programului, punând ca prim argument, *argv[0]*, adresantul (specificația completă a fișierului executabil, cum ar fi de exemplu *C:\\contracte\\facturare.exe*) și anunță programul asupra volumului acestui pachet, prin *argc* (figura 7.1).

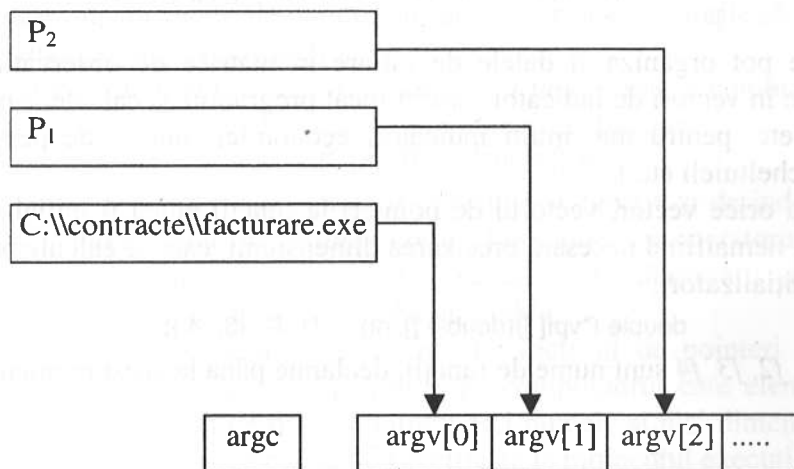


Fig. 7.1 Corespondența dintre argumentele funcției *main()*

Pentru a exemplifica folosirea funcției *main()* cu argumente să elaborăm un program care își listează propriile argumente.

```
#include <stdio.h>
void main(int argc, char *argv[], char *envp[])
{
    while (argc--) printf("\n%s ", *argv++);
    for(int i=0; envp[i]; i++)
        printf("\n envp[%d]=%s", i, envp[i]);
}
```

Probabil că elementele de dificultate în citirea programului de mai sus țin mai degrabă de asocierea și prioritatea operatorilor, decât de argumentele funcției *main()*. *argv* fiind vector de pointeri este pointer de pointeri, motiv pentru care îl găsim uneori declarat sub forma *char **argv*;

De remarcat că vectorul *argv* nu are dimensiunea precizată, deoarece fiind parametru formal, alocarea nu se face aici, ci o face sistemul de operare. Se observă că *argc* este decrementat pentru a asigura afișarea tuturor argumentelor funcției, dar ordinea de afișare se bazează pe incrementarea lui *argv*, care deși nume de vector devine din pointer

constant, pointer variabil, deoarece e parametru de funcție. Așa se explică faptul că argumentele sunt afișate în ordinea în care au fost date în linia de comandă și nu invers, cum descrește valoarea lui *argc*.

Evident, numele argumentelor din *main()* pot fi schimbate, deoarece sunt cuvinte de utilizator, nu cuvinte cheie.

Datorită interpretării dată lui *argv*, *argv[i]* desemnează argumentul *i*, *argv[i][j]* reprezintă caracterul *j* din șirul *i*, iar ***argv*, primul caracter al primului parametru.

Când argumentele din *main()* sunt numere, ele trebuie convertite în reprezentarea internă corespunzătoare tipului lor, înainte de folosire. Spre exemplu, programul de mai jos însumează întregii dați ca parametri la lansarea în execuție:

```
#include <stdio.h>
#include <stdlib.h>

void main(int argc, char *argv[] )
{
    int suma=0,i;
    for(i=1;i<argc;i++)
        suma+=atoi(argv[i]);
    printf("\n%d ",suma);
}
```

Primul argument (*i = 0*) a fost sărit, nefiind număr.

Un alt exemplu poate distinge argumente de tipuri diferite, sau sesiza cazurile în care mai multe date de intrare în program nu sunt separate de spațiu și sunt tratate ca un singur argument.

Se va scrie un program ce primește prin intermediul argumentelor funcției *main* doi operanzi și un operator și va afișa rezultatul operației. Operatorii recunoscuți de program sunt cei aritmetici +, -, * și : (împărțire).

```
#include<stdio.h>
#include<process.h>
#include<string.h>

void main(int argc, char *argv[])
{
    if(argc!=4) // doi operanzi, un operator si argv[0]
    {
        printf("\n Nr necorespunzator de argumente !!!");
        exit(1);
    }
    int a,b;
    if(sscanf(argv[1],"%d",&a)!=1)
    {
        printf("\n Argument nenumeric!!!");
        exit(1);
    }
}
```

```

if(sscanf(argv[3],"%d",&b)!=1)
{
    printf("\n Argument nenumeric!!!");
    exit(1);
}
if(strlen(argv[2])!=1)
{
    printf("\n Operator necunoscut!!!");
    exit(1);
}
switch(argv[2][0])
{
    case '+': printf("Rezultat:%d",a+b); break;
    case '-': printf("Rezultat:%d",a-b); break;
    case '*': printf("Rezultat:%d",a*b); break;
    case '/': printf("Rezultat:%d",a/b); break;
    default: printf("\n Operator necunoscut!!!");
}
}

```

Multe programe sunt elaborate astfel încât să primească switch-uri sau opțiuni la lansarea în execuție, prefixate cu "/", respectiv "-". Lăsăm în sarcina dumneavoastră elaborarea unui program care primește ca argumente numele funcțiilor de executat.

Utilitatea argumentelor funcției *main* este evidentă și în cazul în care un program este lansat în execuție de un alt program; în această situație, argumentele funcției *main* sunt transmise de programul părinte și realizează comunicarea între cele două programe. La terminare, programul fiu poate returna informații părintelui, cu privire la modul de derulare a execuției lui.

Exemplu:

Programul părinte este:

```

#include<stdio.h>
#include<process.h>
void main()
{
    int t;
    printf("\n Program tata!!!");
    t=spawnl(_P_WAIT,"fiu","fiu","Programul_fiu!!!",NULL);
    printf("\n Program fiu terminat cu exit(%d)!! \
        \n Se continua programul tata!!",t);
}

```

Se observă că se lansează în execuție programul numit *fiu* ce va primi prin intermediul argumentelor funcției *main* șirul *Programul_fiu!!!*. Funcția *spawnl* va returna un întreg care indică valoarea parametrului funcției *exit* care duce la terminarea programului *fiu*. În cazul în care nu se apelează funcția *exit* pentru terminarea procesului *fiu*, atunci se va returna

valoarea 0. Dacă funcția *spawnl* returnează valoarea -1, atunci înseamnă că s-a detectat o eroare la proliferarea task-ului fiu.

Programul fiu este:

```
#include <stdio.h>
#include <process.h>
void main(int argc, char *argv[])
{
    printf("\n %s",argv[1]);
    exit(2);
}
```

Dacă se execută programul părinte se va afișa:

```
Program tata!!!
Programul_fiu!!!
Program fiu terminat cu exit(2)!!
Se continua programul tata!!
```

Pentru o execuție corectă a programelor trebuie ca fișierele corespunzătoare celor două procese să se afle în același director. În caz contrar se va da în programul părinte specificația completă a fișierului conținând programul fiu.

7.6 Facilități ale limbajului C / C++ în definirea funcțiilor

Limbajul C / C++ permite:

- suprîncărcarea funcțiilor și operatorilor;
- definirea funcțiilor cu valori implicite pentru parametrii de apel;
- lucru cu funcții cu număr variabil de parametri;
- definirea de șabloane de funcții (template).

Conceptul de supradefinire sau **supraîncărcare** pentru funcții presupune definirea mai multor funcții ce au același nume, ce sugerează obiectivul funcției, chiar când el se realizează pe baza unor algoritmi specifici tipurilor de date de intrare. Problema care apare în acest caz este de a selecta varianta care trebuie executată la un anumit apel de funcție. Soluția este oferită de următoarea restricție impusă prin limbaj: pentru a defini funcții care au același nume, ele trebuie să difere prin numărul de parametri sau prin tipul lor; tipul variabilei returnate de funcție nu constituie criteriu de individualizare a unei funcții.

Utilitatea acestei facilități va fi prezentată prin intermediul unui exemplu de însumare a elementelor unui vector și a elementelor unei

matrice. Se vor scrie două funcții: una ce realizează operația pentru vector și alta pentru matrice. Deoarece diferă numai structurile de date este normal ca funcția să poarte același nume.

```
int suma_el(int v[], int n)
{
    for(int s=0,i=0; i<n; i++)
        s+=v[i];
    return s;
}

int suma_el(int m[][50], int nl, int nc)
{
    for(int s=0,i=0,j;i<nl;i++)
        for(j=0;j<nc;j++) s+=m[i][j];
    return s;
}
```

Apelul celor două funcții pune în evidență avantajul prezentat în sensul că semantica operației este aceeași, dar se aplică unor structuri de date diferite.

```
void main()
{
    int a[]={1,5,2};
    int b[][50]={ {1,2,3}, {1,7,1} };
    printf("\n Suma el vector: %d", suma_el(a,3));
    printf("\n Suma el matrice: %d", suma_el(b,2,3));
}
```

Funcțiile ce au argumente care pot lua valori implicite constituie o altă tipologie de funcții ale limbajului C. Această facilitate se bazează pe faptul că la definire, parametrilor li se pot asocia valori. Dacă la momentul apelului, nu se furnizează parametrii, atunci ei participă la execuția funcției cu valorile date la momentul definirii.

Se va considera exemplul de însumare a argumentelor unei funcții. Pentru a avea un grad mai mare de utilitate, adică funcția să poată însuma două, trei sau patru argumente, ea va fi definită astfel:

```
int aduna (int a, int b=0, int c=0, int d=0)
{
    return a+b+c+d;
}
```

Valoarea 0 s-a ales ca implicită pentru a nu altera rezultatul adunării; în funcția *main* se observă variantele în care este apelată funcția în sensul celor menționate mai sus.

```
void main()
{
    printf("\n %d + %d = %d", 2,3,aduna(2,3));
    printf("\n %d + %d + %d = %d", 2,3,7,aduna(2,3,7));
    printf("\n %d + %d + %d + %d = %d", 1,2,3,7,aduna(1,2,3,7));
}
```

Acest exemplu poate fi rezolvat și folosind o altă facilități a limbajului și anume definirea funcțiilor cu un număr variabil de parametri.

Funcții cu număr variabil de parametri. Uneori nu știm care este numărul exact de parametri cu care se apelează o funcție, el variind de la un apel la altul. Este, spre exemplu, cazul funcțiilor *printf* și *scanf*, cu care putem scrie sau citi un număr variabil de informații. Limbajul prevede în acest caz o formă particulară de a da prototipul funcției, formă în care variabilitatea numărului de parametri se indică prin trei puncte (...), sugerând eventuala continuare a listei parametrilor.

Elementul de dificultate în acest caz este legat de modul în care localizăm parametrii în lista parametrilor actuali, pentru a-i folosi efectiv în funcție, și de numărul lor, pentru a ști unde trebuie să ne oprim în lista parametrilor.

Prima problemă se rezolvă dacă avem adresa de început a listei și dacă găsim o modalitate de a indica tipul parametrilor, de care depinde lungimea memoriei ocupate. Cea de-a doua problemă se rezolvă, fie indicând ca prim parametru numărul efectiv de parametri, variabil la fiecare apel, fie convenind o metodă de indicare a sfârșitului listei de parametri (eventual un cod de sfârșit, 0 sau -1 la parametrii numerici sau caracterul nul pentru parametrii alfanumerici).

Standardele de limbaj propun folosirea unor macrodefiniții (vezi *stdarg.h* și *varargs.h*, cerut pentru compatibilitate cu UNIX V) care să rezolve de manieră unitară această problemă:

va_list, joacă rol de tip de dată pentru a declara lista argumentelor variabile (pointer de listă);

va_start, pentru a inițializa această listă cu adresa listei parametrilor variabili efectivi;

va_arg, pentru a extrage repetat câte un argument din listă;

va_end, pentru a închide lista, prin punerea pe NULL a pointerului de listă.

Ilustrăm folosirea lor pe un exemplu simplu, de calculul mediei aritmetice dintr-un număr variabil de argumente, lista parametrilor terminându-se cu -1.

```
#include <stdio.h>
#include <stdarg.h>
int media( int , ... );
#define EOL -1
void main( )
{
    /* Apel cu 3 parametri, -1 terminator */
    printf( "Media: %d\n", media( 1, 2, 3, -1 ) );
    /* Apel cu 4 parametri, -1 terminator */
```

```

        printf( "Media: %d\n", media( 5, 6, 8, 9, -1 ) );
        /* Apel fara parametri, -1 terminator */
        printf( "Media: %d\n", media( -1 ) );
    }

int media( int p1, ... )
{
    int n = 0, suma = 0, i = p1;
    va_list lista;          /* Declarare lista argumente variabile */
    va_start( lista, p1 );  /* Initializare lista argumente variabile */
    while( i != EOL )
    {
        suma += i;    n++;
        i = va_arg( lista, int); /* Extragere din lista argumente variabile */
    }
    va_end( lista );      /* Resetare lista argumente variabile */
    return( suma ? (suma / n) : 0 );
}

```

Câteva observații se impun:

- Argumentele unei astfel de funcții se împart în fixe și variabile; trebuie ca să existe cel puțin un argument fix.
- Lista argumentelor se inițializează cu ultimul parametru fix al funcției folosind macrodefiniția *va_start*; în esență se va inițializa cu adresa lui de sfârșit, corectată cu eventualele pierderi rezultate din alinierile în memorie, ceea ce înseamnă adresa primului argument variabil.
- Extragerea câte unui parametru, cu macrodefiniția *va_arg*, va determina implicit și incrementarea pointerului la listă, în funcție de tipul de dată a argumentului extras.

Câteva dezavantaje sunt ușor sesizabile în exemplul de mai sus; în primul rând, ce se întâmplă dacă între numerele a căror medie o calculăm se află chiar -1, folosit de noi ca terminator de listă? Chiar folosind 0 ca terminator, problema nu se rezolvă corect, când 0 face parte din șirul numerelor a căror medie o dorim. Evident, soluția o găsim în convenția pe baza căreia determinăm numărul parametrilor efectivi ai unui apel; o modalitate ar fi ca primul parametru să indice totdeauna câți parametri efectivi urmează în listă.

O a doua problemă se referă la faptul că, în exemplul de mai sus, se presupune că toți parametrii variabili sunt de același tip, întreg. Cum rezolvăm localizarea unui argument când argumentele variabile au tipuri diferite? După cum se observă ușor, macrodefiniția *va_arg* primește ca parametri și tipul argumentului de extras din listă, oferind posibilitatea extragerii de argumente de tipuri diferite.

Șabloane de funcții (funcții *template*). Reprezintă un alt mod, particular, de a defini funcții în C++. Avantajul acestor tipuri de funcții

8.1 Structuri, uniuni și definirea tipurilor sinonime de date

8.1.1 Conceptul de structură de date în C

Limbajul C lucrează atât cu tipuri elementare de date (*int*, *float*, ...), cât și cu ansambluri omogene (masive). În afara acestora, limbajul permite și declararea de structuri, ca ansambluri eterogene de elemente simple, sau compuse la rândul lor din masive sau alte structuri, care în ciuda eterogenității pot fi manipulate unitar, ca entități de sine stătătoare, cu nume proprii. Similar tipului *record* din Cobol sau Pascal, o structură este compusă din tipuri diferite de date membre, care trebuie deci descrise fiecare în parte. Forma generală pentru declararea unei structuri este:

```
struct nume_sablon {
    descriere_membru_1;
    .....
    descriere_membru_n;
};
```

Un exemplu de astfel de structură poate fi:

```
struct material
{
    int cod;
    char den[20];
    float pret;
};
```

Șablonul se comportă ca un tip special de declarator, introdus de utilizator; el nu impune și rezervarea de memorie internă, dar permite declararea ulterioară a unor variabile din noul tip, variabile care vor beneficia și de alocare de memorie, corespunzător șablonului.

Spre exemplu, șablonul de mai sus poate sta la baza unei declarații de forma:

```
struct material m1 , m2, *pm;
```

sau și mai simplu:

```
material m1 , m2, *pm;
```

formă acceptată de mai toate versiunile actuale de compilatoare C/C++.

Putem spune deci, că *m1* și *m2* sunt variabile de tip *material*, așa cum spunem că, în declarația *float x,y*; *x* și *y* sunt de tip *float*.

Alocarea unor variabile de tip structură se poate face și concomitent cu descrierea șablonului:

```
struct material
{
    int cod;
    char den[20];
    float pret;
} m1, m2, *pm;
```

Trebuie făcută însă distincție clară între descrierea unei structuri (furnizarea unui șablon) și declararea unei variabile de tip structură. Când șablonul nu este necesar decât pentru o singură folosință, numele șablonului poate lipsi, odată cu definirea lui făcându-se și declararea tuturor variabilelor necesare:

```
struct
{
    int cod;
    char den[20];
    float pret;
} m1, m2, m3, *pm;
```

Memoria ocupată de o structură se calculează pe baza tipului membrilor și a unor eventuale cerințe de aliniere. Se recomandă utilizarea operatorului *sizeof()* pentru determinarea lungimii structurii, deoarece ea poate fi mai mare sau cel puțin egală cu suma dimensiunilor membrilor componenți. Diferența provine din eventualele cerințe de aliniere a datelor în memorie.

Accesul la un membru dintr-o structură se face după modelul:

variabila . membru

Spre exemplu, *m1.cod* desemnează codul primului material, iar *m1.den[0]*, prima literă din denumirea materialului *m1*.

Operațiile permise asupra structurilor sunt următoarele:

- **extragerea adresei** unei variabile de tip structură (&);

pm = &m1;

- **atribuirea** la nivel global:

m1 = m2;

- **transmiterea ca argument** în funcție sau returnarea structurii din funcție;

material f(material m) { material r; /* ... */ return r; }

- **încărcarea** unei structuri; se poate face:

a. la declarare, prin listă de inițializatori, în concordanță cu tipul elementelor din structură:

material m1={101,"Tabla galvanizata",110.5};

b. prin atribuirii ulterioare:

m1.cod=102; strcpy(m1.den,"Aliaj cupru"); m1.pret=14.75;

c. prin citire:

printf("\ncod: "); scanf("%d",&m1.cod);

```
printf("\ndenumire:");    gets(m1.den);
printf("\n pret: ");      scanf("%f",&m1.pret);
```

d. prin copiere în bloc:

```
m1 = m2;
```

Nu se acceptă comparări de forma `str1!=str2`; membrii unei structuri participă însă la toate operațiile specifice tipului lor, inclusiv extragere de adresă.

Să se scrie programul care calculează distanța dintre două puncte. Coordonatele punctelor se vor grupa în structura *Punct*. Considerând notațiile din figura 8.1, relația de calcul este:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

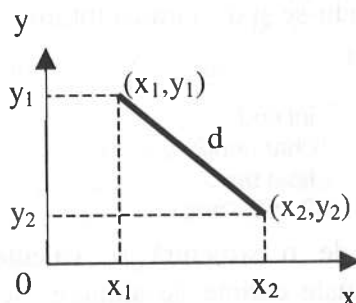


Fig. 8.1 Calculul distanței dintre două puncte

```
#include <stdio.h>
#include <math.h>

struct Punct {    unsigned x;    unsigned y;    };

void main()
{
    Punct a,b;
    double d;
    printf("\n Abscisa punct initial:");    scanf("%d",&a.x);
    printf("\n Ordonata punct initial:");    scanf("%d",&a.y);
    printf("\n Abscisa punct final:");    scanf("%d",&b.x);
    printf("\n Ordonata punct final:");    scanf("%d",&b.y);
    d=sqrt((double)(b.x-a.x)*(b.x-a.x)+(b.y-a.y)*(b.y-a.y));
    printf("\n Distanța este %lf",d);
}
```

8.1.2 Structuri pe șiruri de biți

Limbajul C permite operații la nivel de biți, pe date de tip integral, dar și definirea unor structuri pe șiruri de biți. Ele își găsesc aplicabilitate

ridicată în codificarea și compactarea unor informații, permițând manipularea facilă a unor entități mai mici de 1B.

Cum în general, variabilitatea unei caracteristici comportă mult mai puține stări decât 256 (0-255, cât se poate reprezenta pe 1B), devine eficientă utilizarea unor câmpuri de unul sau mai mulți biți. Spre exemplu, pentru codificarea stării civile a unei persoane sunt suficienți doar 2 biți, care pot să conțină valorile 0,1,2,3 corespunzător stărilor C - căsătorit, N - necăsătorit, D - divorțat, V - văduv, și nu 8 biți cât ar ocupa un caracter; economia de memorie de 75% este cu atât mai semnificativă dacă o raportăm la un fișier de personal, de mii de înregistrări.

De foarte multe ori, căutările în baze de date se efectuează pornind de la valorile unor caracteristici, mecanism bazat pe fișiere de tip "invers"; acestea pot fi create foarte eficient în C, utilizând structuri pe biți.

Definirea unei structuri pe biți se face ca în exemplul:

```
struct sir_biți
{
    unsigned studii:2;
    char sex:1;
    int profesie:3;
    short stare_civila:2;
};
```

Prezența separatorului : și a întregului ce definește numărul de biți ocupați, distinge câmpul ca fiind unul memorat pe biți. Toate câmpurile trebuie să fie de tip integral, deși ele sunt totdeauna cantități fără semn.

Există câteva particularități ale structurilor pe biți. În primul rând, un câmp nu trebuie să taie limita unui cuvânt calculator; în caz că o face, el va fi alocat la începutul celui de-al doilea cuvânt; reiese că nici lungimea totală a unui câmp nu trebuie să depășească lungimea unui cuvânt calculator. Evident, structura în ansamblul ei se poate întinde pe mai multe cuvinte.

Numele de câmp nu este obligatoriu; spre exemplu, *unsigned :3;* este o declarație corectă și servește la alinierea următorului câmp (similar clauzei FILLER din Cobol).

Dacă pentru un câmp lungimea este declarată zero, aceasta semnifică forțarea alinierii câmpului ce urmează, la începutul cuvântului următor, presupunându-se că el ar depăși granițele cuvântului, dacă alinierea s-ar face obișnuit.

Câmpurile pe biți nu au adrese întregi, astfel încât operatorul & nu poate fi aplicat unui astfel de câmp; de aici decurg o serie de alte restricții, privind imposibilitatea utilizării adresării indexate și indirecte. Astfel, câmpurile nu pot fi declarate ca masive, deoarece adresarea indexată este

implementată în C prin adresare indirectă. Un câmp pe biți nu pot fi încărcat direct prin citire, nedispunând de o adresă ce ar trebui furnizată lui *scanf()*; din această cauză, se citește mai întâi valoarea într-o zonă temporară, după care este transferată câmpului:

```
scanf("%u",&temp);
s.cimp=temp;
```

De remarcat că biții din *temp* se "așează" ei prin atribuire pe pozițiile corespunzătoare lui *cimp* din structură, fără a necesita deplasări pe biți, descrise de programator; acesta are doar responsabilitatea ca valoarea din *temp* să poată fi reprezentabilă pe numărul de biți alocați câmpului din structură. Biții pot fi așadar accesați direct, fără a apela la deplasări, sau alți operatori pe biți.

Inițializarea unei structuri pe biți se poate face și la declarare:

```
sir_biți s = {3,1,06,1};
```

În lista de inițializare pot apare constante de tip *char*, *int* sau *unsigned*, în reprezentare zecimală, octală sau hexa. Sensul de încărcare este de la dreapta spre stânga, dar nu trebuie mizat pe acest lucru, deoarece la alte compilatoare se alocă și invers; aceasta înseamnă că manipularea câmpurilor trebuie să se facă totdeauna prin numele lor, nu bazându-ne pe poziția câmpului în declararea structurii.

Fiind considerați ca întregi mai mici, fără semn, cu excepția operației de extragere adresă, elementele unei structuri pe biți pot participa la toate celelalte operații cu întregi (atribuiri, operații logice și de deplasare pe biți, imprimare cu descriptori de format specifici întregilor, etc.)

Cu ajutorul structurilor pe biți se pot citi / seta anumite zone de memorie mai mici de 1B, de la adrese cunoscute, ce au semnificații bine precizate, fără a mai folosi măști și operatori pe biți. În acest caz trebuie cunoscut modul în care o structură pe biți acoperă o zonă de memorie. Acoperirea se face punând în corespondență primul câmp declarat în structură, cu biții cei mai puțin semnificativi ai zonei de memorie.

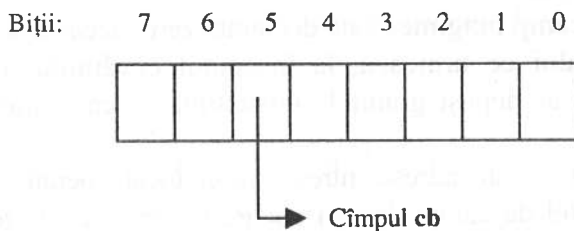


Fig. 8.2 Modul de acoperire a octetului de către structura litera

Caracterele litere mari diferă de caracterele litere mici printr-un singur bit și anume, caracterele literă mare au bitul al șaselea cu valoarea 1 în timp ce caracterele literă mică au acel bit cu valoarea 0. Pentru a transforma caracterele literă mică în literă mare și invers trebuie modificată valoarea acelui bit. În vederea acestui lucru se va construi o structură pe biți, litera care să acopere zona de memorie a unui caracter ca în figura 8.2:

```
#include <stdio.h>
#include <ctype.h>
struct litera
{
    unsigned : 5;
    unsigned cb:1;
    unsigned : 2;
};

void transf_lit(char *car)
{
    litera *l=(litera*)car;
    l->cb=!l->cb;
}

void main()
{
    char sir[50];
    printf("\n Dati sirul:"); gets(sir);
    for(int i=0; sir[i]; i++) if(isalpha(sir[i])) transf_lit(sir+i);
    puts(sir);
}
```

8.1.3 Masive de structuri și structuri incluse

O variabilă de tip structură poate fi declarată și ca masiv; masivele de structuri sunt asociate în general cu conceptul de *fișier în memorie*, deoarece un element conține un articol, iar întregul masiv, un fișier.

De exemplu, definirea:

```
struct student
{
    char nume[20];
    int note[3];
} s[25];
```

alocă suficientă memorie pentru a stoca informații despre 25 de studenți

O construcție de forma *s[10].nume* desemnează numele celui de-al unsprezecelea student, în timp ce *s[0].note[0]* identifică prima notă a primului student; o astfel de adresare corespunde unui masiv ale cărui elemente sunt

variabile de tip structură; ea nu se confundă cu adresarea unui element dintr-un masiv din componența unei variabile de tip *struct*; precizarea elementului unui masiv de structuri se face totdeauna înainte de punct, iar a elementului unui masiv din componența structurii, totdeauna după punct.

Un masiv de structuri beneficiază de posibilitatea de inițializare specifică masivelor:

```
student s[10] = {
    {"Adam Vasile",{7,8,5}},
    {"Avram Ion",{8,9,7}},
    };
```

Câmpurile componente sunt manipulate conform tipului lor; spre exemplu, citirea numelui de la terminal se face ținând seama că descriptorul *%s* solicită în lista de intrare, pointer la șir:

```
scanf("%s",s[5].nume);
```

O structură poate conține în interiorul său alte structuri, dar nu tot de șablonul său (recursivitatea în descriere nu este permisă); poate conține în schimb o variabilă pointer la o structură tot de forma sa; acest mecanism permite definirea așa-numitelor *structuri autoreferite*, de tipul listelor înlanțuite, arborilor, stivelor, etc.

În cazul structurilor incluse, se poate descrie un șablon în șablon, sau șabloanele interioare se descriu în ordinea includerii, pentru a fi disponibile la momentul includerii lor, în alt șablon:

```
struct student
{
    char nume[20];
    struct domiciliu
    {
        char localit[30];
        char str[20];
        int nr;
    } adresa; // nume element in structura mare
    int note[3];
} s;
```

Calificarea unui membru de pe nivelul doi de includere se face folosind de două ori modelul de adresare prin notația punct:

```
s.adresa.nr = 150;
```

Aceeași descriere se poate realiza și astfel:

```
struct domiciliu
{
    char localit[30];
    char str[20];
    int nr;
}; // nu apare variabila adresa
```

```

struct student
{
    char nume[20];
    domiciliu adresa;
    int note[3]
} s;

```

Acest mod de descriere este obligatoriu când se dorește folosirea unor variabile asociate structurilor incluse, de mai multe ori, independent de structura ce le include.

Pentru exemplificare se va construi un fișier în memorie (un vector de structuri) ce va stoca informații despre studenți (număr matricol, nume și prenume și trei note la trei discipline). După încărcarea lui se va afișa o situație școlară ce va conține numărul curent, numele și prenumele, notele și media aferentă.

Încărcarea datelor se face astfel: primele două elemente prin inițializare, următoarea prin atribuire, iar restul prin citire de la tastatură.

```

#include<conio.h>
#include<process.h>
#include<stdio.h>
#include<string.h>
void main()
{
    struct student
    {
        int matricol;
        char nume[30];
        float nota[3];
    } s[25] = {
        {201,"Avram S",{7.5,8.0,6.3}},
        {202,"Vlaicu E",{5.1,6.0,7.3}}
    };

    int i,n,k;
    float tmp;
    s[2].matricol=203;    strcpy(s[2].nume,"Popa P");
    s[2].nota[0]=7.3;    s[2].nota[1]=8.3;    s[2].nota[2]=9.3;
    printf("\n Mai introd date referitoare la un nr de studenti?:");    scanf("%d",&n);
    for(i=3;i<n+3;i++)
    {
        printf("\n Nr. matricol:");    scanf("%d",&s[i].matricol);
        printf("\n Numele:");    fflush(stdin);    gets(s[i].nume);
        for(k=0;k<3;k++)
        {
            printf("\n nota[%d]:",k+1);    scanf("%f",&tmp);
            s[i].nota[k]=tmp;
        }
    }
}

```

```

printf("\n\n\n
      \n
      \n\n
\n=====
      \n Nr.crt      Nume si prenume      Note      Media      \
\n=====
\n");
for(i=0;i<n+3;i++)
    printf("\n %5d  %-30s %6.2f %6.2f %6.2f %6.2f",
        i+1,s[i].nume,s[i].nota[0],*((s+i)->nota+1),s[i].nota[2],
        (s[i].nota[0]+s[i].nota[1]+s[i].nota[2])/3.);
}

```

Funcția *fflush()* se folosește pentru golirea buffer-ului unui fișier, dat ca parametru, în cazul nostru s-a folosit pentru a goli buffer-ul tastaturii (*stdin*).

8.1.4 Uniuni

Uniunea, sau reuniunea cum mai este ea denumită, corespunde unei zone de memorie care la momente de timp diferite poate conține obiecte de tipuri și de lungimi diferite; din această cauză, uniunea se mai definește ca ansamblu de variabile mutual exclusive. Ca și structura, uniunea conține în descriere mai multe câmpuri membre, dar ele își dispută de această dată același spațiu, nu mai sunt alocate unul în continuarea celuilalt.

Compilerul este cel care rezolvă toate problemele de alocare și aliniere în memorie; pentru a putea conține oricare din membrii ei, dimensionarea uniunii se face la nivelul variabilei de lungime maximă, iar funcționarea este comparabilă cu cea a tipului *record* cu *case of* din Pascal.

Uniunea se comportă așadar ca o structură, în care toate elementele sunt aliniate la aceeași adresă:

```

union nume_sablon
{
    descriere_membru_1;
    .....
    descriere_membru_n;
};

```

Șablonul odată descris, poate fi folosit la alocare, ca în exemplul:

```

union date
{
    float f;          long l;
    int i;             char c;
} z;

```

Este posibilă și o inițializare de forma:

```
date z = 17.5;
```

Variabila *z* va fi alocată în acest caz pe 4B (dimensiunea maximă a unui membru), toți ceilalți membri fiind redefiniri ale aceleiași zone.

Interpretarea conținutului zonei la un moment dat ține de context și cade în sarcina programatorului, putând genera rezultate imprevizibile în caz de neatenție. Pentru a degreva programatorul de sarcina urmăririi conținutului unei uniuni, pe toate variantele logice de derulare ale unui program, se poate proceda astfel:

- se codifică tipurile admise în uniune;
- se asociază uniunii o variabilă ce păstrează tipul activ la un moment dat;
- orice încărcare în uniune actualizează și tipul introdus;
- orice prelucrare a uniunii se efectuează numai după validarea prealabilă a tipului pentru care se aplică.

Și uniunile pot fi incluse unele în altele sau în structuri și masive și reciproc. Un exemplu frecvent este acela al citirii aceleiași zone, la nivel de cuvânt sau la nivel de bait (pentru registre e foarte utilizat acest mecanism). În acest scop, zona are două descrieri, ambele de aceeași lungime:

```
struct word
{ unsigned a; };
struct byte
{
    unsigned char ah;
    unsigned char al;
};
```

și disputându-și același spațiu:

```
union zona
{
    word x;
    byte h;
} z;
```

Accesul la nivel de bait se face prin *z.h.al* sau *z.h.ah* (baitii low și high), iar la nivel de cuvânt prin *z.x.a*.

Se va prezenta ca exemplu pentru uniuni de date construirea unei funcții care să returneze fie minimul, fie poziția acestuia dintr-un vector de numere reale. Se va construi, în acest sens o funcție care să returneze o uniune între o variabilă reală (minimul) și una întreagă (poziția acestuia). Pentru a ști semnificația valorii returnate, funcția va primi un parametru de tip caracter. Acesta va avea valoarea implicită *p* pentru a se calcula poziția, sau o altă valoare dată explicit, pentru a calcula valoarea minimului.

```
#include <stdio.h>
union zona
{
    double m;
    int p;
};

zona minim(double pu[], int n, char tip='p')
{
    zona z;
    z.m=pu[0];
    for(int i=1, p=0; i<n; i++)
        if(pu[i] < z.m)
            { z.m=pu[i]; p=i; }
    return tip=='p' ? z.p : z.m;
}
```

Se observă pe *return* o expresie virgulă, elementul returnat fiind mereu uniunea *z*, dar cu semnificații diferite în funcție de ce se indică prin ultimul parametru de apel al funcției.

```
void main()
{
    double a[]={6.7,8.9,3.12,5.1};
    int n=sizeof(a)/sizeof(double);
    printf("\n Elementul minim este %lf",minim(a,n,'v').m);
    printf("\n Pozitia elementului minim este %d",minim(a,n).p);
}
```

8.1.5 Pointeri la structuri și transmiterea structurilor ca parametri în funcție

Așa cum se arată, tipul structură odată definit poate fi aplicat și masivelor și pointerilor:

```
material m[3], *psm;
psm=&m[2]; // încărcare cu o adresa
```

Conform aritmeticii de pointeri, orice operație asupra lui *psm* asigură modificarea adresei cu lungimea în baiți ocupată de întreaga structură. Reamintim că suma dimensiunilor elementelor unei structuri este mai mică sau egală cu lungimea structurii (eventualele diferențe se explică prin golurile generate de cerințele de aliniere, conform tipului fiecărui membru). Compilatorul efectuează și o optimizare, prin reordonarea câmpurilor în structură; se recomandă deci *sizeof(material)* în locul calculării manuale a dimensiunii, de către programator, care poate greși cu privire la modul de aliniere realizat de compilator.

O dată încărcat, un pointer la structură poate fi folosit în adresarea elementelor structurii, folosind operatorul `->`:

`psm->cod=100;`

sau mixt, folosind operatori: `*` și `.`

`(*psm).cod=100;`

Paranteza este necesară pentru că operatorul `.` are precedență mai mare și s-ar interpreta drept conținutul de la adresa dată în `psm.cod`, ceea ce ar fi fals.

Pentru structuri incluse, operatorul `->` poate apare de mai multe ori, sau combinat cu operatorul `.`:

`strcpy (ps->padresa->strada, "Uverturii" );`

sau

`strcpy (ps->adresa.strada, "Uverturii" );`

depinzând cum s-a descris structura, cu membru *adresa* sau cu un pointer *padresa* către o zonă ce conține adresa.

Ca și operatorul `.` operatorul `->` este asociativ de la stânga la dreapta, astfel încât `ps->padresa->strada;` este echivalent cu `(ps->padresa) -> strada;`

Precedența maximă a operatorilor `->` și `.` ne conduc la următoarele interpretări ale expresiilor de incrementare:

`++ps->x` incrementează pe *x*, din structura gestionată prin pointer;

`(++ps)->x` incrementează *ps* și apoi adresează membrul *x*;

`(ps++)->x` adresează *x* folosind pointer, apoi incrementează *ps* (postincrementarea este onorată și în raport cu adresarea în structură);

`*ps->x` obiectul pe care îl pointează *ps->x*;

`*ps->x++` adresează *x*, accesează ce pointează *x* și incrementează obiectul pointat;

`*ps++->x` adresează *x*, accesează ce pointează *x* și apoi incrementează *ps*;

`(*ps->x)++` adresează *x*, extrage obiectul pointat de *x*, incrementează obiectul pointat de *x*.

Pointerii la structură își dovedesc utilitatea în adresarea facilă a membrilor unor masive de structuri, precum și în transferul între funcții a unor structuri, prin adresă.

Deși ansambluri de date, structurile se transmit în funcții implicit prin valoare, nu prin adresă ca masivele. Transmiterea ca parametru, a unui pointer la structură în locul structurii, face mai rapid transferul, evitând încărcarea pe stivă a întregii structuri și asigură totodată lucru pe structura original, nemainecesiând returnarea rezultatului.

Când structura a fost alocată în funcția apelată (obligatoriu static), returnarea adresei acesteia rapidizează transferul rezultatului, adresa putând fi preluată și numai o singură dată, deoarece nu se modifică între apeluri; la fiecare apel, conținutul structurii se va actualiza prin noile prelucrări.

Transmiterea structurii prin valoare asigură și unica modalitate de a transmite un masiv ca parametru în funcție prin valoare, făcându-l element al unei structurii.

Prototipul unei funcții care returnează o structură de tip *student* ar putea fi *struct student f();* sau mai simplu *student f();*

O facilitare deosebită, de neînlocuit, o oferă pointerii la structură în definirea și manipularea structurilor autoreferite.

Astfel, un arbore binar se poate descrie astfel:

```
struct arbbin
{
    int k;           // informatia din nod
    arbbin *ss,*sd; // adresa subarbore stang / drept
};
```

Definirea și prelucrarea recursivă a structurilor autoreferite reprezintă o facilitare deosebită a limbajului C, așa cum se va vedea în continuare.

Să se construiască, în memorie, o structură de date eficientă care să stocheze informații atât despre produsele care se fabrică într-o întreprindere, cât și despre materialele care se consumă pentru fabricarea acestor produse. În figura 8.3 se prezintă înlănțuirea informațiilor; zonele de memorie se vor alocă dinamic, îndeosebi datorită faptului că numărul de materiale consumate pentru fabricarea produselor poate diferi mult de la un produs la altul.

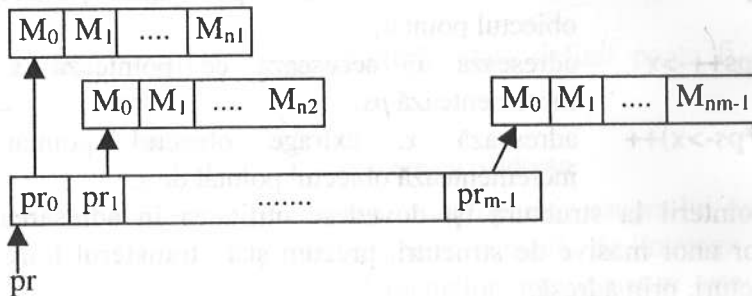


Fig. 8.3 Masive de structuri

unde:

- $pr_i, i=\overline{0, m-1}$, produsul i ;
- $M_{ij}, i=\overline{0, m-1}, j=\overline{0, n_i}$, materialul j care se consumă la fabricarea produsului i .

Informațiile ce se stochează pentru un material sunt:

- codul materialului – *codm*;
- cantitatea consumată – *cant*;
- prețul unitar – *pu*.

Informațiile ce se stochează pentru un produs sunt:

- codul produsului – *codp*;
- denumirea produsului – *denp*;
- numărul de materiale ce se consumă pentru producerea lui – *nrmat*;
- legătura cu informațiile despre materiale – *vpmat*.

Exploatarea acestei structuri este complexă și vom da câteva posibilități împreună cu funcțiile care rezolvă cerințele:

- să se determine consumul total de materiale necesare pentru fabricarea tuturor produselor – *cons_total()*;
- să se determine valoarea consumului de materiale pentru un produs – *cons_mat()*;
- să se determine consumul total dintr-un material indiferent de produsele care-l utilizează – *consum_m()*;

```
#include <stdio.h>
```

```
struct material
```

```
{  
    int codm;  
    int cant;  
    float pu;
```

```
};
```

```
struct produs
```

```
{  
    int codp;  
    char denp[30];  
    int nrmat;  
    material *vpmat;
```

```
};
```

```
void main()
```

```
{  
    produs *pr,*cauta_produs(produs *,int);  
    float consum_m(produs*,int,int),cons_total(produs*,int),cons_mat(produs );  
    int m,i,j,tcodp,tcodm;  
    printf("Numar de produse:"); scanf("%d",&m);  
    pr=new produs[m]; // alocare fisier produse  
    for(i=0;i<m;i++)  
    {  
        printf("\nCod produs:"); scanf("%d",&pr[i].codp); fflush(stdin);  
        printf("Denumire produs:"); gets((pr+i)->denp);
```

```

printf("Nr. materiale:"); scanf("%d",&(pr+i)->nrmat);
pr[i].vpmat=new material[pr[i].nrmat]; // aloacarea si inlantuirea
// materialelor unui produs

for(j=0;j<pr[i].nrmat;j++)
{
    printf("\tCod material:"); scanf("%d",&pr[i].vpmat[j].codm);
    printf("\tCantitatea:"); scanf("%d",&((pr+i)->vpmat+j)->cant);
    printf("\tPretul:"); scanf("%f",&pr[i].vpmat[j].pu);
}
}

printf("\n Consum total de materiale: %7.2f",cons_total(pr,m));
printf("\n Dati codul produsului pentru calculul consumului:");
scanf("%d",&tcodp);
produs *elem=cauta_produs(pr,tcodp);
if(elem)
    printf("\n Pentru produsul %s, consumul este %7.2f lei"
           ,elem->denp,cons_mat(*elem));
else printf("\n Produsul cu codul %d nu exista!!!",tcodp);
printf("\n Codul materialului pentru calculul consumului:");
scanf("%d",&tcodm);
float cons_m=consum_m(pr,m,tcodm);
if(cons_m)
    printf("\n Materialul cu codul %d s-a consumat in valoare de %7.2flei"
           ,tcodm,cons_m);
else printf("\n Materialul cu codul %d nu exista!!!",tcodm);
}

float cons_total(produs *p,int m)
{
    float val=0;
    for(int i=0,j; i<m;i++)
        for(j=0;j<p[i].nrmat;j++) val+=p[i].vpmat[j].cant * p[i].vpmat[j].pu;
    return val;
}

produs *cauta_produs(produs *p,int k)
{
    produs *el=NULL;
    for(int i=0;i<k;i++)
        if(k==p[i].codp) { el=p+i; break; }
    return el;
}

float consum_m(produs *p,int m,int cm)
{
    float val=0;
    for(int i=0,j; i<m;i++)
        for(j=0;j<p[i].nrmat;j++)
            if(p[i].vpmat[j].codm==cm) val+=p[i].vpmat[j].cant*p[i].vpmat[j].pu;
    return val;
}

```

```
float cons_mat(produs e)
{
    float val=0;
    for(int i=0;i<e.nrmat;i++)
        val+=e.vpmat[i].cant*e.vpmat[i].pu;
    return val;
}
```

Din rezolvarea acestei probleme se pot observa modalitățile echivalente de adresare a membrilor unui structuri, cât și diverse forme de transmitere și de returnare a structurilor în / din funcții.

Funcția *cauta_produs()* este una ajutătoare și are ca scop returnarea adresei structurii cu codul de căutat, sau NULL în caz că elementul nu există.

8.1.6 Definirea tipurilor sinonime de date

Limbajul C dispune de o facilitare numită *typedef* pentru a defini noi nume pentru tipurile de date existente sau pentru a construi tipuri de *utilizator*, pornind de la cele cunoscute; de exemplu:

```
typedef int integer,comp;
typedef float real,comp_1;
typedef char character;
```

tipurile *int*, *float* și *char* vor avea de acum și alte nume alternative (*integer* și *comp* pentru *int*, *real* și *comp_1* pentru *float* *character* pentru *char*), ce pot apare în declarații precum:

```
real x,y,*z;    în loc de    float x,y,*z;
```

Efectul realizat prin *typedef* este comparabil cu cel realizat prin directiva *define*: *#define real float* cu mențiunea că obiectivul este realizat de compilator, nu de preprocesor ca în cazul lui *#define*.

Declarația unui tip sinonim de date poate include și dimensiuni de masiv; astfel se poate introduce tipul vector, ca masiv de 10 întregi.

```
typedef int vector[10];
```

Se observă că dimensiunea ține de tip (masiv de 10 întregi), deși este separată prin nume de cuvântul cheie ce definește tipul. Pentru a reține mai ușor, se observă că definițiile de tip sunt identice cu declarațiile de variabile, numai că identificatorul variabilei devine identificator de tip, când declarația începe cu *typedef*.

Declarații corecte sunt și:

```
typedef int matrix[10][10];
typedef char string[60];
```

Tipul *vector* va fi aplicabil unei date, fiind echivalent cu masiv de 10 întregi:

```
vector x,y;
```

Analog matrix *m*; declară pe *m*, drept matrice de 10 x 10 întregi, iar tipul *string* permite o declarare mai simplă a șirurilor de 60 caractere.

Combinat cu *struct*, *typedef* permite construirea de tipuri compuse:

```
typedef struct {double re; double im;} n_complex;
n_complex z1,z2;
```

Facilitatea de *typedef* este utilizată pentru:

- creșterea lizibilității programelor, prin declararea unor tipuri sugestive;
- simplificarea unor declarații complexe:

```
typedef struct list
{
    int k;
    struct list *next;
}*stack;
stack ps; // pointer spre o memorie organizată ca stivă.
```

- ușurarea urmăririi portabilității programelor de pe o mașină pe alta, prin parametrizare, declarând cu *typedef* tipurile a căror implementare depinde de mașină sau de sistem.

Declarațiile cu *typedef* se pot compune, făcând mai rapidă citirea tipurilor complexe (explicitarea tipurilor complexe):

```
typedef int *PI; pointer la întreg;
```

```
typedef PI FPI(); funcție ce returnează pointer la întreg;
```

```
typedef FPI *PFPI(); pointer la funcție ce returnează pointer la întreg;
```

```
typedef PFPI M6PFPI[6]; masiv de 6 pointeri la funcție ce returnează pointer la întreg;
```

```
typedef M6PFPI *PM6PFPI; pointer la masiv de 6 pointeri la funcții ce returnează pointer la întreg.
```

Se observă că fiecare tip nou introdus poate sta la baza construirii altor tipuri prin *typedef*, sau a declarării de variabile sau funcții:

```
FPI x; PFPI y; M6PFPI vp; PM6PFPI pvp;
```

Ca aplicație, folosind facilitatea de *typedef* se definește un tip sinonim pentru descrierea unui număr complex și se vor defini funcții ce realizează operații cu numere complexe.

```
#include <stdio.h>
```

```
#include <math.h>
```

```
#define NOP 2
```

```
typedef double real;
```

```
typedef struct nc { real pr; real pi; } n_complex; // tipul numar complex
```

```

typedef n_complex op_c(n_complex, n_complex); // tipul operatie binara cu
                                                // numere complexe
typedef real f_modul(n_complex); // tipul operatie unara pe numar complex
                                // cu retur de numar real
typedef n_complex f_conj(n_complex); // tipul operatie unara pe numar
                                // complex
typedef op_c *pf_op_c; // tipul pointer la functie care efectueaza o operatie
                        // binara cu un numar complex
typedef pf_op_c vpf_op_c[5]; // tipul vector de pointeri la functii
n_complex c_conj(n_complex z) // calculul conjugatului
{
    n_complex cz;
    cz.pr=z.pr;
    cz.pi=-z.pi;
    return cz;
}

real c_mod(n_complex z) // calculul modulului
{
    return sqrt((double)z.pr*z.pr+z.pi*z.pi);
}

n_complex adun(n_complex a, n_complex b) // adunarea a doua numere
{
                                                // complexe
    n_complex rez;
    rez.pr=a.pr+b.pr;
    rez.pi=a.pi+b.pi;
    return rez;
}

n_complex scad(n_complex a, n_complex b) // scaderea a doua numere
{
                                                // complexe
    n_complex rez;
    rez.pr=a.pr-b.pr;
    rez.pi=a.pi-b.pi;
    return rez;
}

void main()
{
    n_complex x1={3,4}, x2={6,2}, x3, xrez;
    op_c adun, scad; // prototipul functiilor folosite
    vpf_op_c vf = { adun, scad }; // declararea si initializarea unui vector
                                // de operatii pe numere complexe

    f_conj c_conj;
    f_modul c_mod;
    char *sf[]={"\n (%7.2lf%+7.2lfi) + (%7.2lf%+7.2lfi) = (%7.2lf%+7.2lfi)",
               "\n (%7.2lf%+7.2lfi) - (%7.2lf%+7.2lfi) = (%7.2lf%+7.2lfi)" };
    printf("\n l%7.2lf%+7.2lfi=%7.2lf", x1.pr, x1.pi, c_mod(x1));
    for(int i=0; i<NOP; i++)
    {
        xrez=vf[i](x1, x2);
    }
}

```

```

        printf(sf[i],x1.pr,x1.pi,x2.pr,x2.pi,xrez.pr,xrez.pi);
    }
    xrez=c_conj(x1);
    printf("\n Conjugatul numarului (%7.2lf%+7.2lf)=(%7.2lf%+7.2lf)",
        x1.pr,x1.pi,xrez.pr,xrez.pi);
}

```

8.2 Structuri autoreferite

8.2.1 Lista

Lista este o structură secvențială de date, fiecare element conținând informația propriu-zisă și adresa de legătură cu alt nod. Grafic un nod se prezintă ca în figura 8.4.

Informația de bază poate fi un număr, un caracter, sau un șir de caractere, un pointer, un masiv sau o structură eterogenă și poate fi supusă tuturor prelucrărilor admise pentru tipul respectiv. Adresa de legătură este un pointer la o structură de tip similar și permite trecerea la următorul element din listă.

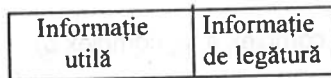


Fig. 8.4 Nodul unei liste simplu înlănțuite

Pentru a avea acces la unul din elementele listei este necesară cunoașterea adresei primului element, numit *cap de listă*. Ultimul element din listă conține în partea de adresă un pointer *NULL* pentru a putea identifica sfârșitul de listă, ca în figura 8.5. Pentru o listă vidă, adresa primului element este tot un pointer *NULL*.

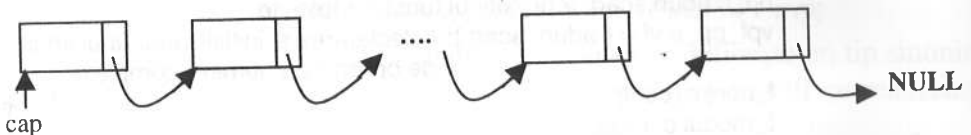


Fig. 8.5 Înlănțuirea elementelor într-o listă simplu înlănțuită

Din punct de vedere al limbajului de programare, lucru cu liste presupune două aspecte majore:

- descrierea acestui tip de structură de date;

- alocarea memoriei dinamice necesară stocării elementelor listei și construirea propriu-zisă.

Pentru simplificarea prezentărilor ulterioare vom considera că informația utilă dintr-un nod al listei este un întreg, dar lucrurile se petrec similar și în cazul general.

Descrierea în C a structurii de tip listă este de forma:

```
struct lista
{
    int inf;
    lista *next;
};
```

Dintre cuvintele care apar în descriere, doar *struct* și tipurile de bază sunt cuvinte rezervate, celelalte fiind alese de utilizator. Caracterul de autoreferit apare prin faptul că structura conține în sine un pointer către o structură de același tip (declararea recursivă a unei structuri nefiind permisă, adică o structură nu poate include o altă structură de același tip, dar poate conține un pointer către ea).

După modul de înlănțuire a elementelor, o listă poate fi:

- *simplu înlănțuită*, când conține un singur pointer, către următorul element și permite parcurgerea ei într-un singur sens, de la primul la ultimul element;
- *dublu înlănțuită*, când fiecare element conține doi pointeri, unul către elementul următor și altul către elementul precedent (figura 8.6); o astfel de listă permite parcurgerea atât înainte, cât și înapoi și are o reprezentare de forma:

```
struct listad
{
    int inf;
    listad *prev,*next;
};
```

Informație de legătură înapoi	Informație utilă	Informație de legătură înainte
-------------------------------------	---------------------	--------------------------------------

Fig. 8.6 Nodul unei liste duble înlănțuite

Listele pot fi făcute și *circulare*, dacă ultimul element în loc să poarte spre NULL pointează spre primul element; chiar lista dublu înlănțuită poate fi circulară:

- într-o direcție când *next* de la ultimul element pointează pe primul element;
- în cealaltă, când *prev* al primului element punctează pe ultimul;

c) în ambele sensuri când și *prev* și *next* al primului / ultimului nod pointează pe ultimul / primul nod al listei.

În ce privește alocarea memoriei necesare stocării unei liste, datorită numărului necunoscut de elemente din listă se apelează la alocarea pe rând a câte unui element, pe măsura constituirii listei; pointerul *void* returnat de funcția de alocare va fi convertit prin *cast* în pointer la o structură de tip listă:

```
lista *l;
l = ( lista *)malloc( sizeof( lista ) );
```

Crearea unei liste se face element cu element. După modul în care se inserează un nou element într-o listă deja existentă deosebim:

- creare prin inserare nod la începutul listei;
- creare prin inserare nod la sfârșitul listei;
- creare prin inserare nod în interiorul listei, spre exemplu când se construiește o listă deja sortată.

Pentru *inserarea nodului la începutul listei*, algoritmul este foarte simplu. El presupune parcurgerea următoarelor etape:

- alocarea memoriei pentru noul element;
- încărcarea valorii noului element;
- înlănțuirea lui în capul listei, prin încărcarea adresei listei preexistente ca adresa de legătură a noului element.

Funcția de inserare se poate prezenta astfel:

```
lista *insi(lista *cap, int k)
{
    lista *nou=(lista *) malloc(sizeof(lista));
    nou->inf=k, nou->next=cap;
    return nou;
}
```

Se observă că nu se face distincție între primul nod și următoarele, inserarea făcându-se similar; este esențial însă, ca nodurile să se insereze la o listă care la început a fost vidă, adică a avut capul ei pus pe *NULL*; altfel parcurgerea ulterioară a listei nu mai dispune de un reper pentru oprire.

De observat că la afișarea elementelor listei, valorile vor apare în ordinea inversă introducerii lor în listă.

Funcția de *inserare a elementelor la sfârșitul listei* în vederea construirii ei, se poate prezenta astfel:

```
lista *inss(lista *cap, int k)
{
    lista *nou=(lista *) malloc(sizeof(lista)), *temp;
    nou->inf=k; nou->next=NULL;
    if(cap==NULL) return nou;
    temp=cap;
```

```

while(temp->next!=NULL) temp=temp->next;
temp->next=nou;
return cap;
}

```

În algoritmul de creare listă prin inserare la sfârșitul listei, valorile vor apare exact în ordinea în care au fost date, dar se va complica puțin procesul de înlănțuire a noului element. După alocarea și încărcarea noului nod, dacă lista nu este vidă ($\text{cap} \neq \text{NULL}$), se va parcurge mai întâi în gol lista folosind un pointer temporar, pentru depistarea ultimului element al listei (cel care trimite spre NULL, adică $\text{temp} \rightarrow \text{next} == \text{NULL}$). Ultimului element din lista deja existentă i se va modifica pointerul, astfel încât el să înlănțuie și noul element.

Parcurea listei se face, fie privind elementul curent, printr-o repetiție de tipul: $\text{while}(\text{temp} \neq \text{NULL}) \text{temp} = \text{temp} \rightarrow \text{next}$, fie privind la următorul element, $\text{while}(\text{temp} \rightarrow \text{next} \neq \text{NULL}) \text{temp} = \text{temp} \rightarrow \text{next}$; prima modalitate este folosită când nu ne interesează ultimul nod pentru a face vreo legătură, ci putem să “călcăm” chiar în gol, ajungând pe un nod NULL. Spre exemplu, numărarea sau însumarea elementelor unei liste se pot face printr-o parcurgere cu test pe nodul curent.

Parcurea cu test pe *next* este necesară când trebuie să ne oprim pe ultimul nod valid al listei, spre exemplu pentru a concatena două liste, sau pentru a insera un nod nou în finalul listei. Desigur am putea și acum să facem testul tot pe nodul curent, dar va trebui să ținem “urma” pe unde am trecut, pentru a avea totuși memorat ultimul nod valid prin care am trecut și de care să legăm nodul inserat sau lista de concatenat.

Structurile autoreferite permit implementarea operațiilor cu ajutorul funcțiilor recursive. Parcurea listei până la ultimul element în scopul inserării unui nou nod la sfârșitul ei se poate realiza și în manieră recursivă. O astfel de funcție reface din aproape în aproape toate legăturile dintre noduri pentru a lega și noul nod introdus:

```

lista *inssr(lista *cap, int k)
{
    if (cap) { cap->next=inssr(cap->next,k); return cap; }
    else
    {
        lista *nou=(lista *) malloc(sizeof(lista));
        nou->inf=k,nou->next=NULL;
        return nou;
    }
}

```

Timpul de construire a unei liste prin inserare la sfârșit de listă este mult mai mare, comparativ cu realizarea aceleiași liste, dar prin inserare la început.

Este posibil să se folosească una din funcțiile de inserare la începutul sau sfârșitul listei pentru a construi o listă ale cărei valori se citesc de la terminal, una câte una, până la tastarea lui *CTRL/Z*. O astfel de funcție de construire listă prin citire de la terminal se poate prezenta astfel:

```
lista *cons_cit()
{
    lista *r=NULL;
    int temp;
    while (printf("\n Nodul: "), scanf("%d",&temp)!=EOF)
        r = inssr(r, temp);
    return r;
}
```

În procesul de construirea a unei liste se poate folosi o funcție de *inserare directă* a unui nod într-o listă. Elementul va fi inserat la locul potrivit în sensul respectării ordinii crescătoare a informațiilor utile din noduri.

```
lista *insd(lista *cap, int k)
{
    lista *nou=(lista *) malloc(sizeof(lista));
    nou->inf=k;
    if ( !cap || k<cap->inf )
        { nou->next=cap; return nou; }
    for( lista *aux=cap; aux->next && aux->next->inf<k; aux=aux->next );
    nou->next=aux->next;
    aux->next=nou;
    return cap;
}
```

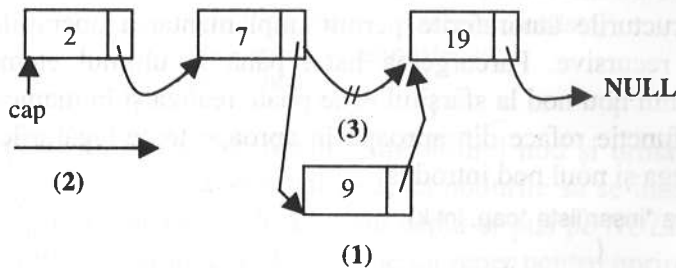


Fig. 8.7 Etape în procesul de inserare directă a unui nod în listă

Funcția realizează următoarea secvență de operațiuni:

1. crearea noului nod și fixarea informației utile;
2. în cazul în care lista e vidă sau nodul ce trebuie inserat are informația mai mică decât cea din capul listei, atunci acest nou element devine capul listei și funcția se termină altminteri, se parcurge lista, fie până nu mai există nici un element, fie până se întâlnește un element cu informația utilă mai mare;

3. se înlanțuie elementul.

Etapele de inserare directă a unui element într-o listă sunt surprinse în figura 8.7. Se presupune o listă cu informațiile: 2, 7, 19 și se dorește a se insera direct nodul cu informația 9.

Această funcție se poate folosi la sortarea unei liste prin inserție directă.

Frecvent este necesară o funcție pentru **scrierea conținutului unei liste**, mai ales pentru a testa corectitudinea funcțiilor de inserare. Varianta nerecursivă a unei funcții de scriere listă ar putea arăta astfel:

```
void afis_lnr(lista* cap)
{
    while(cap) { printf("%5d",cap->inf); cap=cap->next; }
}
```

Varianta recursivă a aceleiași funcții de scriere listă este următoarea:

```
void afis_lr(lista* cap)
{
    if(cap) { printf("%5d",cap->inf); afis_lr(cap->next); }
}
```

Inversând *printf* cu apelul recursiv se obține afișarea elementelor listei în ordine inversă.

Tot în cadrul operațiilor de întreținere liste se include și **concatenarea a două liste**. O variantă de realizare a acestei operații, în varianta nerecursivă, se prezintă prin funcția următoare:

```
lista *concat(lista *l1, lista *l2)
{
    lista *l;
    if(!l1) return l2; // cazul cind l1 este vida
    if(!l2) return l1; // cazul cind l2 este vida
    l=l1;
    while(l->next)l=l->next; // parcurgere in gol
    l->next=l2;           // legare
    return l1;
}
```

. Funcția pentru concatenare, în varianta recursivă, se poate prezenta astfel:

```
void concatr(lista*& cap1, lista *cap2)
{
    if (!cap2) return;
    else if (cap1) concatr(cap1->next, cap2);
    else cap1=cap2;
}
```

În varianta recursivă se observă că prima listă este primită prin referință deoarece în *cap1* se va păstra capul listei concatenate. Cazul în care

capul primei liste se modifică după apel este doar acela când prima listă este vidă.

Una dintre operațiile foarte frecvente asupra listelor este cea de **ștergere a unui element din listă**. O funcție care realizează această operație primește o listă și elementul de eliminat, returnând lista fără toate aparițiile acelui element, în cazul unor apariții repetate.

```
void sterge_nodr(lista*& cap, int k)
{
    if(cap) // se testeaza daca lista nu e vida
        if(cap->inf==k) // daca nodul are inf. cautata
        {
            lista *a=cap; // se retine adresa lui
            cap=a->next; // se actualizeaza legatura
            free(a);      // se sterge fizic nodul
            sterge_nodr(cap,k); // se continua cu nodul curent
        }
    else sterge_nodr(cap->next,k); //altfel se continua cu nodul urmator
}
```

Funcția pentru **numărarea nodurilor dintr-o listă**, în variantă recursivă este:

```
int nrnod_r ( lista *cap )
{
    if( cap ) return 1+ nrnod_r ( cap-> next ) ;
    else return 0;
}
```

În timp ce în varianta sa nerecursivă este:

```
int nrnod( lista *cap)
{
    int n;
    for(n = 0 ;cap; cap = cap->next) n++;
    return n;
}
```

Testul de terminare a ciclului *for*, prezentat sub forma *cap*; trebuie interpretat drept *cap!= NULL*. Se observă de asemenea că transmiterea prin valoare a capului de listă permite alterarea acestuia; acum parcurgerea listei s-a făcut fără a folosi o variabilă auxiliară.

Similar se poate construi și funcția ce calculează **suma informațiilor utile din nodurile unei liste**:

```
int suma_r ( lista *cap )
{
    if( cap ) return cap->inf + suma_r ( cap-> next ) ;
    else return 0;
}
```

Funcția pentru **căutarea unui nod într-o listă**, după informația sa utilă, va returna pointerul la nod, sau *NULL* în caz că elementul nu există:

```
lista *caut(lista *cap, int k)
{
    if(!cap) return NULL;
    else if(cap->inf==k) return cap;
    else return caut(cap->next,k);
}
```

În continuare, se vor prezenta trei funcții de **sortare a elementelor unei liste**. O funcție de sortare prin *inserție directă* care va primi lista de sortat și va returna o nouă listă cu aceleași elemente, dar sortate. Se observă că se folosește aici funcția de inserare directă a unui element într-o listă, prezentată anterior. Parametrul *vb* are și valoare implicită, deci nu complică apelul funcției. S-a folosit pentru a marca primul apel, deoarece capul listei în care urmează să se insereze elemente trebuie pus inițial pe valoarea *NULL* (listă vidă).

```
lista* sortd(lista* cap, int vb=1)
{
    static lista *cp;
    if(vb) cp=NULL;
    if ( cap )
        { cp=insd(cp, cap->inf); return sortd(cap->next,0); }
    else return cp;
}
```

O altă modalitate de a sorta o listă este cea prin *reactualizarea legăturilor* între elemente, astfel încât ele să fie într-o anumită ordine. Metoda nu implică crearea unei noi liste, ci se va lucra pe cea inițială. Funcția va primi capul listei și va returna capul listei sortate, care va fi cu siguranță adresa nodului cu informația utilă cea mai mică, în cazul sortării crescătoare.

```
lista *sortl(lista *cap)
{
    if (!cap || !cap->next ) return cap; // lista cu cel mult un element e sortata
    int vb=1;
    lista *a1,*a2;
    while (vb) // se termina sortarea cind la o trecere prin lista nu se mai
        { // efectueaza nici o reactualizare de legaturi
            vb=0;
            if (cap->inf>cap->next->inf) //reactualizare legaturii intre capul listei si
                { //elementul urmator in scopul conservarii capului listei
                    a1=cap->next;    cap->next=a1->next;
                    a1->next=cap;    cap=a1;
                    vb=1;
                }
        }
}
```

```

a1=cap;
while (a1->next->next) // reactualizare legaturilor intre celelalte noduri
{
    if(a1->next->inf>a1->next->next->inf)
    {
        vb=1;
        a2=a1->next;
        a1->next=a2->next;
        a2->next=a1->next->next;
        a1->next->next=a2;
    }
    a1=a1->next;
}
}
return cap;
}

```

O altă modalitate de sortare este cea prin *interschimbarea informațiilor utile* dintre elementele listei, fără modificarea legăturilor dintre noduri. Se poate observa că se aduce minimul dintre elemente pe prima poziție în listă. Aceeași operație se aplică apoi pe sublistele rămase.

```

void sortiel(lista *cap)
{
    if(!cap || !cap->next ) return ;
    lista *i,*j;
    int t;
    for(i=cap; i->next; i=i->next)
        for(j=i->next; j ;j=j->next)
            if(i->inf>j->inf)
            {
                t=j->inf; j->inf=i->inf; i->inf=t;
            }
}

```

Stergerea unei liste presupune eliberarea zonelor de memorie ocupate de noduri și setarea valorii capului de listă pe *NULL*.

```

void sterge_lista(lista*& cap)
{
    if(cap)
    {
        sterge_lista(cap->next);
        free(cap); cap=NULL;
    }
}

```

Ca aplicație, structura de listă simplu înlănțuită se va folosi la memorarea polinoamelor rare (polinoame în care mulți coeficienți au valoarea 0 și deci nu este necesară memorarea lor). Informația utilă din

nodul listei se constituie la rândul ei într-o structură care memorează coeficientul și gradul elementului.

Se construiesc funcții pentru realizarea operațiilor:

- creare polinom prin citire de la tastatură – *c_pol()*;
- afișare polinom – *afis_pol()*;
- adunarea a două polinoame – *addpol()*;
- derivare simbolică – *deriv()*;
- integrare simbolică – *integrala()*;

Funcțiile *c_nod()* și *sort()* sunt ajutoare și au ca scop crearea unui nod și inserarea lui în capul listei, respectiv sortarea listei în ordinea crescătoare a gradului elementelor.

```
#include<stdio.h>
#include<stdlib.h>

struct ut // descrierea informatiei utile
{
    double coef;
    int grad;
};

struct elem // descrierea unui element al listei
{
    ut e;
    elem *next;
};

elem *sort(elem *l) // sortare lista dupa grad
{
    if( !l || !l->next ) return l;
    elem *i,*j;
    ut u;
    for(i=l;i->next;i=i->next)
        for(j=i->next;j=j->next)
            if(i->e.grad>j->e.grad) { u=i->e; i->e=j->e; j->e=u; }
    return l;
}

elem *c_nod(elem *pol, ut *u) // inserare element in capul listei
{
    elem *tmp=(elem*)malloc(sizeof(elem));
    tmp->e=*u; tmp->next=pol;
    return tmp;
}

elem *c_pol() //creare polinom
{
    elem *pol=NULL,*c_nod(elem *, ut *),*sort(elem*);
    ut u;
    int n,i;
    double tmp;
```

```

printf("\n Dati gradul polinomului:"); scanf("%d",&n);
for(i=n;i>=0;i--)
{
    printf("\n Coeficientul termenului de gradul %d este:",i);
    scanf("%lf",&tmp);
    if(tmp!=0)
    {
        u.coef=tmp;u.grad=i;
        pol=c_nod(pol,&u);
    }
}
return sort(pol);
}

elem *addpol(elem *pol1, elem *pol2) // adunare polinom
{
    elem *pol3=NULL,*c_nod(elem *, ut *),*sort(elem*);
    ut t;
    if(!pol1&&!pol2) { printf("\n Polinoame vide!!"); return NULL; }
    if(!pol1) { printf("\n Primul polinom vid!!"); return pol2; }
    if(!pol2) { printf("\n Al doilea polinom vid!!"); return pol1; }
    while(pol1||pol2)
    {
        if(!pol1)
        {
            while(pol2) { pol3=c_nod(pol3,&(pol2->e)); pol2=pol2->next; }
            break;
        }
        else if(!pol2)
        {
            while(pol1) { pol3=c_nod(pol3,&(pol1->e)); pol1=pol1->next; }
            break;
        }
        if(pol2->e.grad>pol1->e.grad)
            { pol3=c_nod(pol3,&(pol1->e)); pol1=pol1->next; }
        else if(pol2->e.grad<pol1->e.grad)
            { pol3=c_nod(pol3,&(pol2->e)); pol2=pol2->next; }
        else
        {
            t.coef=pol1->e.coef+pol2->e.coef;
            if(t.coef)
            {
                t.grad=pol1->e.grad;
                pol3=c_nod(pol3,&t);
            }
            pol1=pol1->next; pol2=pol2->next;
        }
    }
    return sort(pol3);
}

```

```

void afis_pol(elem *pol,int vb=1) //afisare polinom
{
    if(vb)printf("\n P(x)=");
    if(pol)
    {
        afis_pol(pol->next,0);
        if(pol->e.grad) printf("%+7.2fX**%d",pol->e.coef,pol->e.grad);
        else printf("%+7.2f",pol->e.coef);
    }
}

elem *deriv(elem *pol) //derivare simbolica
{
    elem *polrez=NULL,*sort(elem*);
    ut u;
    while(pol)
    {
        if(pol->e.grad!=0)
        {
            u.coef=pol->e.coef*pol->e.grad;
            u.grad=pol->e.grad-1;
            polrez=c_nod(polrez,&u);
        }
        pol=pol->next;
    }
    return sort(polrez);
}

elem *integrala(elem *p) // integrare simbolica
{
    elem *r=NULL,*sort(elem*);
    ut u;
    while(p)
    {
        u.grad=p->e.grad+1;
        u.coef=p->e.coef/(p->e.grad+1);
        r=c_nod(r,&u);
        p=p->next;
    }
    u.coef=1.; u.grad=0;
    r=c_nod(r,&u);
    return sort(r);
}

void main()
{
    elem *p1=NULL,*p2=NULL,*p3,*pderiv,*pint;
    p1=c_pol(); p2=c_pol();
    p3=addpol(p1,p2);
    printf("\n Primul polinom:"); afis_pol(p1);
    printf("\n Al doilea polinom:"); afis_pol(p2);
}

```

```

printf("\n Polinomul suma:");    afis_pol(p3);
printf("\n Polinomul suma derivat:");
pderiv=deriv(p3);    afis_pol(pderiv);
printf("\n Polinomul suma integrat:");
pint=integrala(p3);
afis_pol(pint);
}

```

Pentru **lista dublu înlănțuită** operațiile sunt similare cu precizarea că mai trebuie gestionată și legătura spre elementul anterior.

Funcția de inserare a unui element în capul listei se modifică astfel:

```

listad *ins(listad *cap, int k)
{
    listad *t=(listad *)malloc(sizeof(listad));
    t->inf=k;
    t->next=cap;
    t->prev=NULL;
    if(cap) cap->prev=t;
    return t;
}

```

Se observă că elementul inserat fiind primul în listă va avea totdeauna pointerul spre nodul anterior (*prev*) pus pe *NULL*. Vechiul cap al listei, în cazul în care aceasta nu a fost vidă, va trebui să-și modifice pointerul *prev*, deoarece acum are înaintea lui un nou element: `if(cap) cap->prev=t;`

În figura 8.8 se prezintă modul de înlănțuire a elementelor unei liste dublu înlănțuite.

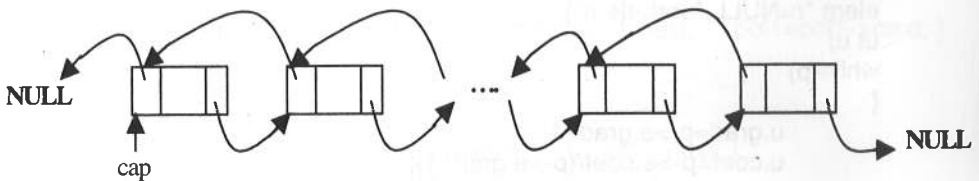


Fig. 8.8 Înlanțuirea elementelor într-o listă dublu înlănțuită

Existența ambelor legături facilitează traversarea listei în cele două sensuri. Funcția ce realizează afișarea unei liste în ambele sensuri este următoarea:

```

void afis(listad *cap)
{
    if(!cap) return;
    while( cap->next )
    {
        printf("%5d", cap->inf);
        cap=cap->next;
    }
}

```

```

printf("%5d",cap->inf);
while( cap )
{
    printf("%5d",cap->inf);
    cap=cap->prev;
}

```

8.2.2 Stiva

Stiva este tot o structură liniară, de tip listă, cu singura deosebire că inserările și extragerile se fac totdeauna la un același capăt al listei, respectând disciplina LIFO – Last Input First Output (ultimul intrat, primul ieșit) ca în figura 8.9. Pentru varietate, vom folosi facilitatea de *typedef* pentru a introduce tipul *pointer de stivă*:

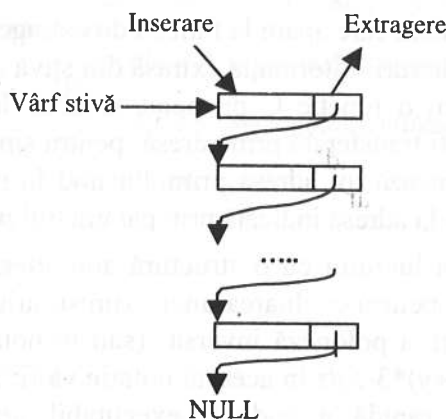


Fig 8.9 Structura de stivă

```

typedef struct list
{
    int info;
    struct list *next;
} *stack;

```

Pointerul la stivă prin care se manipulează structura în ansamblu se numește *vârful stivei*.

Deși operațiile pe stivă sunt multiple, se vor trata numai cele două operații de bază: introducere (*push*) și extragere (*pop*) din stivă.

```

stack push(stack s, int info)
{
    stack t;

```

```

t = (stack) malloc( sizeof(list) );
t->info=info;
t->next=s;
return t;

```

Se observă că s-a folosit tot structura de tip listă, iar tipul *stack* este de fapt echivalent cu *list **; de aceea la alocare a fost nevoie de *list*, nu de *stack*, fiind nevoie de o alocare de nod, nu de pointer de nod.

```

stack pop(stack s, int *pinfo)
{
    stack t;
    if ( !s ) return NULL;
    else
    {
        *pinfo = s->info;  t = s->next;
        free(s);          return t;
    }
}

```

O problemă particulară care apare la funcția de extragere este faptul că trebuie să returneze două lucruri: informația extrasă din stivă și respectiv noul top (adresă) de stivă. Cum o funcție C nu poate returna decât cel mult o valoare, una din valori va fi transferată prin adresă; pentru simetrie cu funcția *push()*, funcția *pop()* returnează tot adresa primului nod în stivă, informația propriu-zisă fiind încărcată la adresa indicată prin parametrul *pinfo*.

Ca exemplificare a lucrului cu o structură autoreferită de tip stivă, vom prezenta un program pentru evaluarea unei expresii aritmetice simple, dată de la terminal, în forma poloneză inversă (sau în notația postfixată). Echivalentul expresiei $((x+y)*3-5)/z$ în această notație va fi: *x,y,+,3,*,5,-,z,/,* și va facilita generarea rapidă a codului executabil necesar evaluării expresiei. Se va trata cazul simplu în care expresia este alcătuită din constante și operatori introduși de la terminal. Funcția *skip()* elimină eventualele spații ce preced expresia, după care funcția *cit()* stabilește dacă este vorba despre un operator și-i identifică tipul sau despre un operand și-l convertește în format intern. Funcția *eval()* face evaluarea propriu-zisă, după regula: dacă este operand îl pune pe stivă, iar dacă este operator extrage de pe stivă un număr adecvat de operanzi, efectuează calculele și pune rezultatul pe stivă.

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

```

```

typedef struct list { int k; struct list *next; } *stack;
enum { NUM, ADD, SUB, MUL, DIV, END };

```

```

char c, tip;
int nr;
int eval()
{
    void cit();
    stack s=NULL, push( stack ,int ), pop(stack , int *);
    char sf=0;
    static int a,b;
    while(!sf)
    {
        cit();
        switch (tip)
        {
            case NUM:
                s=push(s,nr); break;
            case ADD:
                s=pop(s,&a); s=pop(s,&b); s=push(s,a+b); break;
            case SUB:
                s=pop(s,&a); s=pop(s,&b); s=push(s,a-b); break;
            case MUL:
                s=pop(s,&a); s=pop(s,&b); s=push(s,a*b); break;
            case DIV:
                s=pop(s,&a); s=pop(s,&b); if(b)s=push(s,a/b); break;
            case END:
                sf=1; break;
        }
    }
    s=pop(s,&a);
    return a;
}

stack push(stack s,int n)
{
    stack r;
    r = (stack) malloc( sizeof(list) );
    r->k = n;    r->next = s;
    return r;
}

stack pop(stack s, int *n)
{
    stack r;
    if(!s) return NULL;
    else
    {
        *n = s->k;    r = s->next;
        free(s);
        return r;
    }
}

```

```

void skip( )      { while ( (c = getchar( )) == ' ' ) ; }
void cit( )
{
    skip( );
    switch(c)
    {
        case '+': tip=ADD; c=getchar( ); break;
        case '-': tip=SUB; c=getchar( ); break;
        case '*': tip=MUL; c=getchar( ); break;
        case '/': tip=DIV; c=getchar( ); break;
        case '\n': tip=END; break;
        case '0': case '1': case '2': case '3': case '4':
        case '5': case '6': case '7': case '8': case '9':
            tip=NUM; nr=0;
            do {
                nr=10*nr+(c-'0');
            } while ( isdigit(c=getchar( )) ); break;
        default: printf("\n operatie eronata"); break;
    }
}
void main( )
{
    int e;
    e=eval( );
    printf("\nRezultat %d",e );
}

```

8.2.3 Coadă

Coadă este implementată având la bază tot structura de listă. Disciplina de lucru cu coada este FIFO – First Input First Output (primul intrat, primul ieșit). Se poate implementa ușor folosind funcția de inserare a unui nod la sfârșitul unei liste și pe cea de extragere a unui element din vârful stivei. Frecvența mare de operații de acest fel impune o implementare mai eficientă și anume memorarea într-o structură a pointerului la începutul (*prim*) și respectiv la sfârșitul listei (*ultim*) ca în figura 8.10.

Structura de coadă se va manipula astfel, prin intermediul unei structuri și nu printr-un pointer, așa cum a fost cazul listei sau stivei.

Declarația ei este următoarea:

```

struct coada
{
    lista *prim,*ultim;
};

```

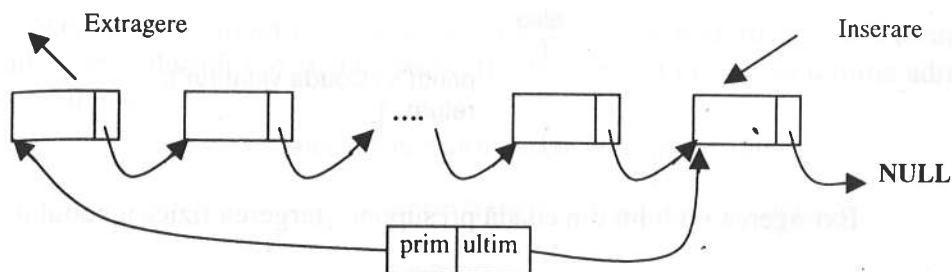


Fig. 8.10 Structura de coadă

Principalele operații de lucru cu coada se referă tot la inserarea, respectiv extragerea unui element în / din coadă.

Coadă vidă are cei doi pointeri din structură puși pe valoarea NULL la definire, o astfel de structură se va inițializa sub forma:

```
coada q= { NULL, NULL };
```

Funcția de inserare a unui element în coadă va primi coada prin referință (pentru a-i putea modifica valoarea pointerilor) și întregul ce urmează a fi stocat. Se observă că funcția realizează crearea noului nod, legarea acestuia la celelalte elemente ale listei, după care se face actualizarea valorii pointerilor din structura de tip coadă.

```
void put(coada &q, int k)
```

```
{
    lista *t= (lista *)malloc(sizeof(lista));
    t->inf=k; t->next=NULL;
    if(q.ultim) q.ultim->next=t;
    else q.prim=t;
    q.ultim=t;
}
```

Funcția de extragere a unui element din coadă primește coada prin referință pentru a-i putea modifica valoarea pointerilor și returnează informația utilă a nodului extras. Când coada este vidă se afișează un mesaj corespunzător și se returnează valoarea -1;

```
int get(coada &q)
```

```
{
    if(q.prim)
    {
        lista *t=q.prim;
        int k=t->inf;
        q.prim=q.prim->next;
        free(t);
        if(!q.prim) q.ultim=NULL;
        return k;
    }
}
```

```

else
{
    printf("\n Coadă vida!!!\n");
    return -1;
}
}

```

Extragerea nodului din coadă presupune ștergerea fizică a nodului.

8.2.4 Arborele binar

Arborii binari sunt formați dintr-o mulțime finită de elemente, care poate fi vidă sau poate conține un element numit rădăcină, iar celelalte elemente sunt distribuite în maxim două submulțimi disjuncte, care reprezintă la rândul lor tot arbori binari. Una din submulțimi este numită subarboarele stâng al rădăcinii, iar cealaltă subarboarele drept.

Într-un arbore binar există noduri care nu mai au la rândul lor subarbori, deci numărul de arce incidente spre exterior este 0. Un astfel de nod se numește frunză sau nod terminal. Un nod se spune că este tată, în raport cu descendenții acestuia, numiți fii. Un nod al unui arbore binar poate avea cel mult doi fii, deci pentru oricare nod al unui arbore binar ordinul maxim este doi. Prin ordin al unui nod se înțelege numărul lui de fii.

Tot pentru un arbore se poate vorbi de conceptul de nivel. Rădăcina unui arbore se află pe nivelul 1. Dacă un nod are nivelul m , atunci fii lui au nivelul $m+1$. Numărul total de nivele al unui arbore determină înălțimea arborelui.

Descrierea structurii pentru un nod al unui arbore binar se poate realiza în limbajul C printr-o structură de forma:

```

struct arbbin
{
    int inf;
    arbbin *ss,*sd;
};

```

Se poate observa că un nod al arborelui are o informație utilă (k), similară structurii de listă, iar informația de legătură asigură conectarea nodului la descendenții lui. Structura în ansamblu se manipulează prin adresa nodului rădăcină numită pentru simplificare în cele ce urmează, *rădăcină*.

Principalele operații cu arbori binari sunt:

a) **Construirea** unui arbore binar se face în două moduri:

- de jos în sus (bottom-up), prin cuplarea a câte două noduri și adăugarea nodului tată al lor; funcția de creare a unui nod după acest

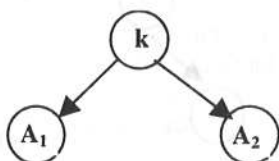
procedeu va primi ca parametri: informația utilă a nodului ce va fi creat și adresele nodurilor din stânga, respectiv din dreapta lui. Se va returna adresa noului nod.

```

arbbin *cnod(int k, arbbin *stg, arbbin *drt)
{
    arbbin *t=(arbbin*)malloc(sizeof(arbbin));
    t->ss=stg;
    t->sd=drt;
    t->inf=k;
    return t;
}

```

- de sus în jos (top - down) prin inserarea câte unui nod în arbore, după o disciplină bine stabilită. Dacă disciplina de inserare conduce la crearea unui arbore, fie vid, fie de forma prezentată în figura 8.11, care satisface proprietățile:



k – informația utilă din nod

A_1 –subarbore stâng

A_2 –subarbore drept

Fig. 8.11 Arbore binar

1) orice cheie k' din A_1 este mai mică decât k ($k' < k$);

2) orice cheie k'' din A_2 este mai mare decât k ($k'' > k$);

3) A_1 și A_2 respectă cumulativ proprietățile 1 și 2;

arborele creat se numește *arbore binar de căutare*.

Funcția de inserare a unui nod într-un astfel de arbore returnează rădăcina și primește ca parametri rădăcina arborelui și informația utilă a nodului de inserat. Pentru înțelegerea funcției se pot face unele observații: se pornește de la un arbore vid, adică rădăcina lui are inițial valoarea *NULL*; nu se admit noduri duplicate; întotdeauna nodul ce va fi inserat va fi o frunză în cadrul arborelui; când arborele este vid, nodul inserat va fi pe post de rădăcină; când nu este vid se caută locul unde ar trebui legat nodul pentru ca arborele să satisfacă proprietățile mai sus menționate.

În figura 8.12 se prezintă în dinamică inserarea nodurilor cu informațiile: 10, 7, 15, 9, 3, 8, 25 într-un arbore binar de căutare. Arborele fiind inițial vid, nodul cu valoarea 10 devine rădăcina (a). Nodul 7 fiind mai mic decât 10 se va insera în subarborele stâng. Acest lucru se poate face deja, deoarece subarborele stâng a lui 10 este vid (b). La fel și pentru nodul 15 numai că, fiind mai mare ca 10 se va insera în subarborele drept și acesta fiind vid (c). Nodul 9 se compară și el cu 10 (valoarea din rădăcină) și fiind

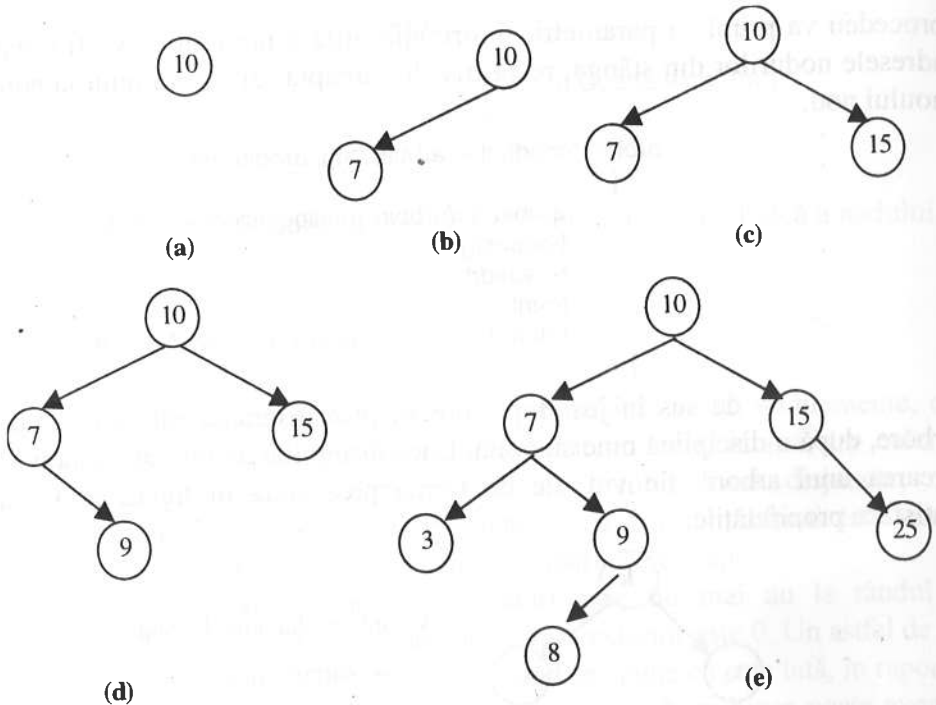


Fig. 8.12 Etape în construirea unui arbore binar de căutare

mai mic se va insera în subarboarele stâng. Deoarece acesta nu e vid se va compara 9 cu 7 (valoarea rădăcinii subarboarelui stâng) și fiind mai mare se va insera în subarboarele din dreapta. Acesta fiind vid, nodul se poate lega în figura 8.12(d). Se continuă în aceeași idee, în final arborele binar de căutare arătând ca în figura 8.12(e).

Funcția se poate face în două maniere:

- nerecursiv:

arbbin *insnr(arbbin*rad, int k)

```

{
    if(!rad) return cnod(k,NULL,NULL); // arbore vid, nodul creat este radacina
    else // arbore nevid
    {
        arbbin *aux=rad;
        while(1)
        {
            if(k<rad->inf) // test daca nodul se va insera in subarboarele sting
            {
                if(rad->ss) rad=rad->ss; // trecerea in subarboarele sting
                else // subarboare vid, deci inserare nod si terminare
                {
                    rad->ss=cnod(k,NULL,NULL);
                    return aux;
                }
            }
            else if (k>rad->inf) // test daca nodul se va insera in subarboarele drept

```

```

    if(rad->sd) rad=rad->sd; // trecerea in subarborele drept
    else // subarbore vid, deci inserare nod si terminare
    {
        rad->sd=cnod(k,NULL,NULL);
        return aux;
    }
    else // cazul in care nodul exista deja
    {
        printf("\n Nod existent!!");
        return aux;
    }
}
}

```

- recursiv:

```

arbbin *ins(arbbin*rad, int k)
{
    if( rad )
    {
        if( k<rad->inf ) rad->ss=ins(rad->ss,k);
        else if( k>rad->inf ) rad->sd=ins(rad->sd,k);
        else printf("\n Nod existent!!");
        return rad;
    }
    else return cnod(k,NULL,NULL);
}

```

b) Funcții de **traversare** a arborelui binar. Parcurgerea poate fi realizată în trei moduri, ele vor fi prezentate în varianta lor recursivă. În general, operațiile pe arbori binari presupun realizarea unei traversări a structurii. Pentru exemplificare, se va traversa arborele în scopul afișării informațiilor utile ale nodurilor.

- parcurgere rădăcină - stânga - dreapta (R S D), sau în preordine

```

void preord(arbbin *rad)
{
    if(rad)
    {
        printf("%5d",rad->inf);
        preord(rad->ss);
        preord(rad->sd);
    }
}

```

- parcurgere stânga - rădăcină - dreapta (S R D), sau în inordine:

```

void inord(arbbin *rad)
{
    if(rad)
    {
        inord(rad->ss);

```

```
printf("%5d",rad->inf);
inord(rad->sd);
```

```
}
}
```

- parcurgere stânga - dreapta - rădăcină (S D R), sau în postordine:

```
void postord(arbbin *rad)
{
    if(rad)
    {
        postord(rad->ss);
        postord(rad->sd);
        printf("%5d",rad->inf);
    }
}
```

c) Funcția pentru **ștergerea unui nod** dintr-un arbore binar de căutare. Această operație va avea în vedere faptul că după efectuarea ștergerii nodului dorit, arborele va trebui să rămână tot de căutare.

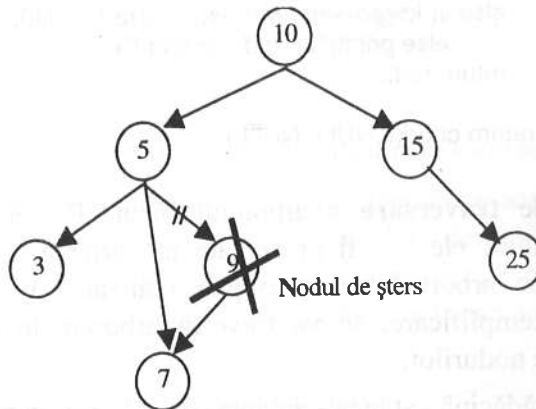


Fig. 8.13 Ștergerea unui nod cu cel puțin un descendent vid

Funcția de ștergere a unui nod trebuie mai întâi să efectueze căutarea nodului cu informația utilă dorită. Ștergerea efectivă a nodului comportă două cazuri:

- nodul de șters are cel puțin un descendent vid, caz în care se reactualizează legătura nodului părinte și se șterge fizic nodul (figura 8.13);

```
void șterge_nod(arbbin*& rad, int k)
{
    arbbin *aux;
    //se cauta nodul de șters
    if(!rad) printf("\n Nodul %d nu exista!!",rad->inf);
    else if(k<rad->inf)șterge_nod(rad->ss,k);
    else if(k>rad->inf)șterge_nod(rad->sd,k);
}
```

```

else
{
    // nodul s-a gasit
    aux=rad;
    // se testeaza daca are un descendent vid
    if(!aux->sd) { rad=aux->ss; free(aux); }
    else if(!aux->ss) { rad=aux->sd; free(aux); }
    else // nu are nici un descendent vid
        rad->inf=sterg(rad->sd);
}
}

```

- nodul de șters are doi fii. În acest caz s-a creat o funcție *sterg* care va rezolva această problemă în următorul mod: nu se va șterge fizic nodul respectiv, ci informația lui utilă va fi înlocuită cu informația utilă a celui mai din stânga nod al subarborelui drept sau cu informația utilă a celui mai din dreapta nod al subarborelui stâng, care apoi va fi șters (figura 8.14). Explicația acestui procedeu constă în faptul că nodul are informația utilă imediat mai mare, respectiv imediat mai mică, decât informația utilă a nodului ce va trebui șters, deci arborele în ansamblu va rămâne tot de căutare.

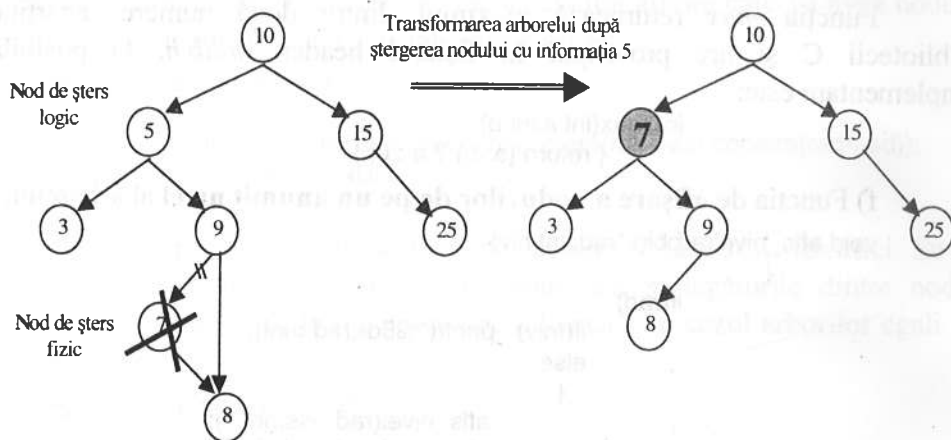


Fig. 8.14 Ștergerea unui nod cu doi descendenți

Această funcție de ștergere va avea forma:

```

int sterg(arbbin*& pa)
{
    if(pa->ss)
        return sterg(pa->ss); // se merge catre cel mai din stnga nod
    else
    {
        arbbin *a=pa;
        int k=a->inf;
        pa=pa->sd;
        free(a);
    }
}

```

```

        return k;
    }
}

```

Ea va returna informația utilă a nodului care se va șterge fizic pentru a înlocui informația utilă a nodului căutat pentru a fi șters.

d) Funcția pentru **numărarea nodurilor** dintr-un arbore:

```

int nr_noduri(arbbin *rad)
{
    if(rad) return 1+nr_noduri(rad->ss)+nr_noduri(rad->sd);
    else return 0;
}

```

e) Funcția ce determină **numărul de nivele** ale unui arbore (înălțimea arborelui):

```

int inaltime(arbbin *rad)
{
    if(rad) return 1+max(inaltime(rad->ss),inaltime(rad->sd));
    else return 0;
}

```

Funcția *max* returnează maximul dintre două numere, aparține bibliotecii C și are prototipul în fișierul header *stdlib.h*. O posibilă implementare este:

```

int max(int a,int b)
{ return (a>b) ? a : b; }

```

f) Funcția de **afișare a nodurilor de pe un anumit nivel** al arborelui:

```

void afis_nivel(arbbin *rad, int niv)
{
    if(rad)
        if(!niv) printf("%5d",rad->inf);
        else
        {
            afis_nivel(rad->ss,niv-1);
            afis_nivel(rad->sd,niv-1);
        }
}

```

Ea primește ca parametru arborele și numărul nivelului de afișat. Se va lua în considerare faptul că rădăcina se consideră a fi pe nivelul 0.

g) Funcția pentru **însurarea informațiilor utile din nodurile frunză** ale unui arbore binar:

```

int sumaf(arbbin *rad)
{
    if(rad)
        if(frunza(rad)) return rad->inf;
        else return sumaf(rad->ss)+sumaf(rad->sd);
}

```

```
else return 0;
```

```
}
```

Se observă că se folosește o funcție ajutătoare ce are rolul de a determina dacă un nod este sau nu frunză. Funcția este:

```
int frunza(arbbin *rad)
```

```
{ return !rad->ss && !rad->sd; }
```

h) Funcția de **ștergere a unui arbore binar**. Acest lucru presupune eliberarea memoriei ocupate de noduri și setarea valorii rădăcinii pe NULL.

```
void sterge_arb(arbbin *& rad)
```

```
{
```

```
    if(rad)
```

```
    {
```

```
        sterge_arb(rad->ss);
```

```
        sterge_arb(rad->sd);
```

```
        free(rad); rad=NULL;
```

```
    }
```

```
}
```

i) Funcția de **copiere a unui arbore binar** primește ca parametru rădăcina unui arbore și returnează rădăcina altui arbore care va avea nodurile cu aceeași informație utilă și legate în același fel.

```
arbbin *copiere(arbbin *rad)
```

```
{
```

```
    if(rad) return cnod(rad->inf,copiere(rad->ss),copiere(rad->sd));
```

```
    else return NULL;
```

```
}
```

j) Funcția ce **verifică dacă doi arbori binari sunt identici**. Se vor verifica atât informațiile utile din noduri, cât și legăturile dintre noduri. Funcția va returna un întreg ce va lua valoarea 1 în cazul arborilor egali și 0 altfel.

```
int egal( arbbin *r1, arbbin *r2)
```

```
{
```

```
    if(!r1) return !r2;
```

```
    else if(!r2) return 0;
```

```
    else return r1->inf==r2->inf&&egal(r1->ss,r2->ss)&&egal(r1->sd,r2->sd);
```

```
}
```

k) Funcția care **afișează drumul de la un nod la rădăcină** are la bază traversarea unui arbore în postordine pentru a păstra pe stivă adresa nodului părinte al celui curent. Ea are ca prim obiectiv căutarea nodului cu informația utilă dorită, după care se oprește procesul recursiv de încărcare a stivei. Variabila statică *gasit* are rolul de a sesiza acest lucru.

```
void afis_dr_inv(arbbin *rad, int k, int vb=1)
```

```
{
```

```
    static gasit;
```

```

if(vb) gasit=0;
if (rad)
    if(rad->inf==k) { printf("%5d",rad->inf); gasit=1; }
    else
    {
        if(!gasit) afis_dr_inv(rad->ss,k,0);
        if(!gasit) afis_dr_inv(rad->sd,k,0);
        if(gasit)printf("%5d",rad->inf);
    }
}

```

1) O mare importanță o prezintă și **tipărirea arborilor** în care se pune în evidență și legătura dintre elemente, nu numai informațiile utile din noduri. Acest lucru se poate face în mai multe forme grafice. Considerăm arborele binar de căutare din figura 8.12.

- varianta de tipărire cu paranteze a arborelui binar:

. forma *răd(st,dr)*, afișează:

10(7(3,9(8,-)),15(-,25))

Funcția are la bază traversarea în preordine:

```

void af_par1(arbbin *rad)
{
    if(rad) // daca exista un arbore / subarbore
    {
        printf("%d",rad->inf); // se afiseaza inf. utila
        if(!frunza(rad)) // daca nu este frunza se trece la
        {
            // afisarea parantezelor si se
            printf("("); // reapeleaza functia
            af_par1(rad->ss);
            printf(",");
            af_par1(rad->sd);
            printf(")");
        }
    }
}

```

else printf("-"); // altfel, se afiseaza -

}

. forma *(st)răd(dr)*, pentru același arbore afișează:

((3)7((8)9(-)))10((-)15(25))

Funcția are la bază traversarea în inordine:

```

void af_par2(arbbin *rad)
{
    if(rad) // daca exista un arbore / subarbore
    {
        if(!frunza(rad)) // daca nu este frunza se reapeleaza
        {
            // functia pentru subarboarele sting
            printf("("); af_par2(rad->ss); printf(")");
        }
        printf("%d",rad->inf); // se afiseaza inf. utila
    }
}

```

```

        if(!frunza(rad)) // daca nu este frunza se reapeleaza
        {
            // functia pentru subarborele drept
            printf("(");  af_par2(rad->sd);  printf(")");
        }
    }
    else printf("-"); // daca nu se afiseaza -
}

```

8.15. • varianta de tipărire a arborelui pe nivele este prezentată în figura

Pentru a realiza acest lucru s-au construit mai multe funcții:

- funcția de afișare a unui număr de spații:

```

void blanc(int n)
{ for(int i=0;i<n;i++) putchar(' '); }

```

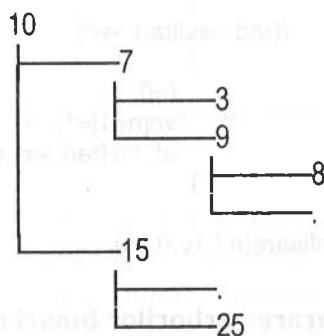


Fig 8.15 Afișarea arborelui binar pe nivele

- funcția de afișare a unui nod, inclusiv a caracterelor ce pun în evidență legătura dintre nodurile arborelui și cel ce urmează a se afișa.

```

void afisare(int n,int f,int k,int vb,int *vn)
{
    int i;
    blanc(4);
    if(n){
        for(i=1;i<n;i++){
            if(vn[i]) putchar(' ');
            else putchar(179);
            blanc(4);
        }
        if(f) putchar(195);
        else putchar(192);
        for(i=0;i<4;i++) putchar(196);
    }
    if(vb) printf("%d\n",k);
    else printf(".\n");
}

```

- funcția care realizează afișarea pe nivele a întregului arbore are la bază traversarea în inordine și a fost realizată într-o manieră recursivă.

```
void af_niv(arbbin *rad,int n=0)
{
    static int vn[50],f=1;
    if(rad)
    {
        afisare(n,f,rad->inf,1,vn);
        if(rad->ssllrad->sd)
        {
            f=1;
            vn[n+1]=0;
            af_niv(rad->ss,n+1);
        }
        if(rad->ssllrad->sd)
        {
            f=0;
            vn[n+1]=1;
            af_niv(rad->sd,n+1);
        }
    }
    else afisare(n,f,10,0,vn);
}
```

m) **Reechilibrarea arborilor binari de căutare.** Pentru a prezenta o funcție care reechilibrează un arbore trebuie precizată noțiunea de arbore binar echilibrat.

Un arbore binar total echilibrat are toate nodurile terminale pe ultimele două nivele sau pentru orice nod, numărul nodurilor din subarboarele stâng diferă de numărul nodurilor din subarboarele drept cu cel mult unu.

Echilibrarea arborilor binari de căutare se poate realiza în două moduri:

- pe măsura inserării sau ștergerii unui nod se face și reechilibrarea (arbori cunoscuți sub numele de AVL – Adelson-Velskii și Landis);
- după construirea unui arbore binar de căutare dezechilibrat se poate aplica un algoritm care-l transformă în arbore binar total echilibrat.

Vom prezenta o funcție care reechilibrează arborele binar de căutare în cea de-a doua variantă. Metoda se bazează pe faptul că se va crea o listă ce va conține nodurile arborelui în ordine crescătoare (parcurea arborelui se va face în inordine). Se vor insera elementele listei în noul arbore aplicând tehnica *divide and conquer* și anume se inserează prima dată elementul din mijlocul listei, apoi elementul din mijlocul primei jumătăți și tot așa până la epuizarea listei. În final va rezulta un arbore binar de căutare total echilibrat.

Funcțiile necesare sunt:

- funcția de inserare a unui element la sfârșitul unei liste simplu înlănțuită ce a fost prezentată și are prototipul: lista* insf (lista *, int);

- funcția de transformare a unui arbore binar de căutare într-o listă, prin traversare în inordine a acestuia. Se vor insera informațiile utile din nodurile arborelui într-o listă și în același timp se face și numărarea nodurilor. Funcția va returna numărul de noduri:

```
int trinord(arbbin *a, lista *&l)
{
    if(a) return trinord(a->ss,l)+(l=insf(l,a->inf),1)+trinord(a->sd,l);
    else return 0;
}
```

- funcția de obținere a informației utile a elementului de rang i dintr-o listă:

```
int lisi(int i, lista *l)
{
    for(int k=1;k!=i;k++) l=l->next;
    return(l->inf);
}
```

- funcția de echilibrare propriu-zisă a arborelui. Ea s-a construit în manieră recursivă și primește ca parametri lista, rădăcina arborelui care se va construi echilibrat, prin referință, limita superioară a intervalului (la început ea coincide cu numărul de elemente ale listei) și limita inferioară care inițial va avea valoarea 1. Funcția este:

```
void reec(lista *cap, arbbin *&rad, int ls, int li=1)
{
    int lj, nrel, i;
    nrel=ls-li+1;
    if(nrel<3) for(i=li; i<=ls; i++) rad=ins(rad, lisi(i, cap));
    else
    {
        lj=(li+ls)/2;
        if((li+ls)%2) lj+=1;
        rad=ins(rad, lisi(lj, cap));
        reec(cap, rad, ls, lj+1);
        reec(cap, rad, lj-1, li);
    }
}
```

Pentru a realiza reechilibrarea se pornește de la un arbore binar de căutare dezechilibrat; fie rădăcina lui $r1$; se urmărește construirea unui arbore echilibrat ($r2$) care, inițial, este vid. Pentru lucru, se va folosi o listă inițial vidă, cu capul l . În aceste condiții apelul funcției de echilibrare devine:

```
reec ( l, r2, trinord (r1,l) );
```

Folosirea arborilor binari de căutare are avantajul ca optimizează timpul mediu de căutare a unui nod.

A. Ca exemplu se va face agregarea seriilor statistice provenind din colectivități statistice diferite, fără o sortare prealabilă a observațiilor. În acest scop se va construi programul de agregare prin cumulara frecvențelor de apariție ale unor valori în serii statistice concatenate, folosind o structură arborescentă.

Se va folosi un arbore binar de căutare, care permite regăsirea rapidă a eventualelor informații care se repetă, în serii statistice diferite.

```
#include <stdio.h>
#include <stdlib.h>

struct obs
{
    float term; // observatie statistica
    int freqv; // frecventa de aparitie
};

struct arbbin // descrierea nodului arborelui binar
{
    obs ut;
    arbbin *ss,*sd;
};

#define NOBS 8

arbbin *c_nod(float t, int f) //functia de construire nod
{
    arbbin *temp=new arbbin;
    temp->ut.term=t; temp->ut.freqv=f;
    temp->ss=temp->sd=NULL;
    return temp;
}

void insarbr(arbbin *&rad, float t, int f) // functia de inserare nod
{
    if(!rad) rad=c_nod(t,f);
    else if(t<rad->ut.term) insarbr(rad->ss,t,f);
        else if (t>rad->ut.term) insarbr(rad->sd,t,f);
            else rad->ut.freqv+=f;
}

void print(struct arbbin *a) // functia de afisare a arborelui
{
    if(a)
    {
        print(a->ss);
        printf("\n %f %t %d",a->ut.term,a->ut.freqv);
        print(a->sd);
    }
}
```

```

void main()
{
    arbbin *arb=NULL;
    struct obs x[NOBS]= {
        { 19500., 29},      { 21300., 20},
        { 30005., 10},     { 41240., 2},
        { 30005., 5},      { 27500., 7},
        { 19500., 11},     { 21300., 20}
    };
    for (int poz=0; poz<NOBS; poz++) insarbr(arb,x[poz].term,x[poz].frecv);
    putchar('\n');
    print(arb);
}

```

De observat funcția *insarbr()* care a fost făcută în manieră recursivă, fără a returna nimic. Rădăcina arborelui care trebuie modificată în anumite condiții se transmite prin referință. Referința la pointer s-a definit în forma:

arbbin *& rad;

B. Să se construiască funcția care determină dacă un arbore binar este sau nu unul de căutare.

```

#include <stdio.h>
#include <basetsd.h>
#include <stdlib.h>

struct arbbin // descrierea nodului arborelui binar
{
    int ut;
    arbbin *ss,*sd;
};

arbbin *c_nod(arbbin *stg, int f, arbbin *drt) // functia de creare nod
{
    arbbin *temp=new arbbin;
    temp->ut=f;
    temp->ss=stg;
    temp->sd=drt;
    return temp;
}

int a_cautare(arbbin *r,int vb=1) // functia ce verifica daca arborele binar
{                                     // e sau nu de cautare
    static iu,vf;
    if(vb)
        vf=1,iu=-MAXINT_PTR; // constanta definita in basetsd.h
    if(r)
    {
        if(vf)
        {
            a_cautare(r->ss,0);
            if(iu<=r->ut) iu=r->ut;

```

```

        else { vf=0; return vf; }
        a_cautare(r->sd,0);
    }
    }
    return vf;
}

void main()
{
    arbbin *r=c_nod(NULL,10,NULL);
    r->ss=c_nod(NULL,6,NULL); r->sd=c_nod(NULL,20,NULL);
    r->ss->ss=c_nod(NULL,3,NULL); r->ss->sd=c_nod(NULL,8,NULL);
    r->sd->sd=c_nod(NULL,12,NULL); r->sd->ss=c_nod(NULL,15,NULL);
    a_cautare(r) ? printf("\n Arbore binar de CAUTARE!!!!") :
        printf("\n Arborele binar NU este de CAUTARE!!!!");
}

```

Verificarea s-a făcut pe baza faptului că o parcurgere a arborelui binar de căutare în ordine trebuie să furnizeze cheile în ordine crescătoare.

A se observa și maniera statică de construire a arborelui binar în funcția *main()*, (nod cu nod).

9.1 Substituirea simbolică. Tipul *enum*

Compilarea în trepte permite și în cazul limbajului C implementarea unor facilități de preprocesare, indicate prin intermediul unor directive. În acest scop, directivele de preprocesare sunt efectuate înaintea compilării și sunt recunoscute de preprocesor prin prezența simbolului `#`. Ele nu se comportă ca instrucțiuni executabile, dar pot determina compilarea condiționată a unor instrucțiuni.

Substituirea simbolică sau definirea de constante simbolice se realizează prin directiva *define*:

```
#define simbol sir
```

Ca urmare, la citarea simbolului în textul sursă, se va produce înlocuirea tuturor aparițiilor lui *simbol*, exceptând prezența sa într-un literal, cu valoarea precizată prin *sir*. Șirul conține toate elementele de pe linie ce urmează spațiului de la sfârșitul simbolului. Pentru continuarea șirului pe mai multe linii se pune caracterul `\` la sfârșitul fiecărei linii ce urmează a fi continuată.

Se recomandă ca simbolurile să fie scrise cu litere mari, pentru a fi ușor recunoscute în cadrul programului.

În general, definirea constantelor simbolice se face în următoarele scopuri:

- a) apropierea între maniera de implementare a structurilor de programare și a operatorilor în diferite limbaje;

Exemple:

```
#define then
#define EQ ==
```

- b) parametrizarea programelor;

```
#define NMAX 10
```

Simbolul *then* este substituit cu spațiu, nefiind prezent în limbajul C; pentru testul de egalitate în programe se poate folosi EQ, deoarece la preprocesare va fi înlocuit cu operatorul `==`.

Abia la compilare, simbolul NMAX înlocuit în prealabil cu textul "10" se comportă ca o constantă de tip întreg, cu valoarea 10. În cazul în care ar fi fost 10., atunci NMAX ar fi devenit o constantă de tip *double*.

Secvența de program:

```

if ( t EQ NMAX) then
    printf("\n t este 10");
else
    printf("\n t este diferit de NMAX");

```

intră în compilare sub forma:

```

if ( t == 10)
    printf("\n t este 10");
else
    printf("\n t este diferit de NMAX");

```

A se observa că în literal, simbolul NMAX nu a fost substituit.

În exemplul:

```

# define TEXT "acesta este un sir"
.....
printf ("% s", TEXT);

```

simbolul TEXT devine după compilare o constantă șir de caractere validă, conținând terminatorul \0, ghilimelele intrând în definiția simbolului.

```
# define PI 3.14592
```

definește constanta simbolică cu valoarea lui π .

c) sporirea lizibilității programelor:

```

# define TRUE 1
# define FALSE 0
# define BEGIN {
# define END }

```

Valabilitatea unei definiri ține până la îndeplinirea uneia din condițiile:

- sfârșitul fișierului sursă;
- redefinirea simbolului respectiv prin apariția unui *define* cu același simbol:

```

#define MESAJ "Mesaj initial"
.....
#define MESAJ "Mesaj redefinit"

```

- până la invalidarea simbolului:

```

#define T 10
.....
#undef T

```

După directiva *undef T* constanta simbolică T nu mai este recunoscută. Necesitatea invalidării apare atunci când mai mulți programatori își definesc propriile simboluri, iar prin concatenarea fișierelor unele funcții ar putea folosi definiții moștenite din secvențele anterioare.

Tipul enumerativ. Similar efectului realizat prin definirea și subsituirea simbolică funcționează în C tipul enumerativ, ce are forma:

```
enum identificador { lista simboluri } lista_variabile;
```

În acest fel se permite manipularea mai ușoară a elementelor unei mulțimi, atât prin numele simbolului, cât și după numărul de ordine atribuit în mulțime.

Exemplu:

```
enum zile { duminica, luni, marti, miercuri, joi, vineri, sambata };
```

În acest fel s-au definit simboluri corespunzătoare pentru zilele săptămânii, atribuindu-le totodată numere întregi, în secvență începând cu 0.

Declarația este echivalentă cu:

```
#define duminica    0
#define luni        1
.....
#define sambata     6
```

Se pot defini fie la declarare, fie ulterior, variabile enumerative:

```
enum zile z1, z2;
```

sau simplu:

```
zile z1, z2;
```

Ordinea simbolurilor este importantă deoarece compilatorul le asociază numere în funcție de ea.

Un simbol ce nu are asociat un număr primește unul în secvență în raport cu cel de dinaintea sa:

```
enum zile { luni=1, marti, miercuri, joi, vineri, sambata, duminica };
```

În acest caz *luni* ia valoarea 1, *marti* 2, ..., *duminica* valoarea 7.

Dacă se dorește ca ordinea simbolurilor din tipul *enum* să fie complet arbitrară, atunci toate elementele enumerării trebuie să aibă precizate explicit numerele asociate:

```
enum culoare { ROSU=4, VERDE=2, ALBASTRU=ROSU+3 };
```

Ca observație se poate preciza că numerele trebuie asociate astfel încât să nu genereze conflicte (două simboluri cu același număr).

Tipul *enum* se poate aplica și pointerilor:

```
culoare *pc, c;
```

unde *pc* este pointer la o zonă de memorie al cărei conținut este unul din numerele asociate culorilor.

Elementele aparținând unei enumerări pot participa la prelucrări specifice tipului:

```
enum zile { luni=1, marti, miercuri, joi, vineri, sambata, duminica } z;
z=miercuri;
printf("\n %d", z);
if (z == sambata || z==duminica) printf("\n zi liberă !");
else printf("\n zi lucrătoare !");
```

În limbajul C++ o atribuire de forma $z=3$ este incorectă; cu toate că enumeratorii sunt de tip întreg trebuie să se folosească $z = \text{miercuri}$; în astfel de situații tipul enumerativ poate fi supus unor conversii prin *cast*: $z=(zile)3$;

Programul de mai jos afișează zilele săptămânii dintr-o lună:

```
#include<stdio.h>
void main()
{
    enum zile { luni, marti, miercuri, joi, vineri, sambata, duminica } z;
    char *zs[]={ "luni", "marti", "miercuri", "joi", "vineri", "sambata", "duminica" };
    z=duminica;
    for(int i=1; i<31; i++)
    {
        z=(zile)((((int)z+1)%7));
        printf("\n %s ",zs[z]);
    }
}
```

9.2 Macrodefiniții

Macrodefinițiile folosesc aceeași directivă pentru declarare ca și substituirea simbolică, dar cu o sintaxă extinsă:

```
#define nume(lista simboluri) sir
```

Paranteza trebuie să urmeze imediat numelui macrodefiniției, altfel declarația este considerată substituire simplă. Macrodefiniția definește o secvență de cod sursă, în care o parte din text este variabilă de la un context la altul. Părțile variabile sunt organizate ca parametri ai macrodefiniției și sunt substituiți diferit în funcție de valorile actuale primite la apel de simbolurile din listă.

În componența șirului, dacă macrodefiniția introduce o expresie, este indicat să se folosească un număr suficient de paranteze prin care să controlăm ordinea de evaluare a expresiei, asigurând corectitudinea expresiei în toate contextele de apel.

De exemplu, se consideră macrodefiniția care realizează ridicarea unui număr la pătrat:

```
#define PATRAT(X) X*X
```

Dacă în program se apelează:

```
k=PATRAT(2+3);
```

atunci apelul se va expanda sub forma: $k = 2+3*2+3$, ceea ce ar conduce la valoarea 11. Dacă macrodefiniția este redactată sub forma:

```
#define PATRAT(X) (X)*(X)
```

atunci k primește valoarea corectă $k=(2+3)*(2+3)$ adică 25.

Atât macrodefinițiile cât și funcțiile au ca scop reducerea dimensiunii codului sursă. Folosirea funcțiilor duce și la reducerea codului executabil, deoarece același cod este executat de mai multe ori (depinzând de numărul de apeluri) și eventual cu alți parametri de apel. Apelul macrodefiniției se face după cum s-a putut observa comparabil cu apelul unei funcții. Diferența constă în aceea că la apelul macrodefiniției se inserează în acel loc, la momentul preprocesării, șirul, reactualizând lista de simboluri. Cu alte cuvinte ea se expandează la fiecare apel, în realitate compilatorul repetând compilarea secvenței pentru fiecare apel al macrodefiniției. Parametrii macrodefiniției pot fi bucăți de text sursă nu neapărat cu semnificație individuală pentru compilator; tot ce contează este ca prin alipirea părților fixe și variabile din text să se obțină un text sursă C , coerent și corect. În exemplul de mai jos porțiunea fixă este un *for* care-și definește și contorul, iar ca părți variabile sunt numărul de iterații și instrucțiunile de ciclat. Testarea se face ciclând pentru afișarea unui mesaj și respectiv, pentru calculul sumei elementelor unui vector.

```
#include<stdio.h>
#define CICLU(n,corp) { for(int i=0;i<n;i++) { corp; } }
void main()
{
    int a[]={1,2,3},s=0;
    CICLU(10,printf("\n Mesaj!!!"));
    CICLU(3,s+=a[i]);
    printf("\n Suma=%d",s);
}
```

O macrodefiniție poate intra în componența alteia; în acest fel se construiesc macrodefiniții compuse. Ca exemplu se va construi macrodefiniția CUB care va folosi macrodefiniția PATRAT, definită anterior:

```
#define CUB(X) (PATRAT(X)*(X))
```

Macroul pentru determinarea maximului dintre două numere se poate construi astfel:

```
#define MAX(X,Y) ((X)>(Y)?(X):(Y))
```

și se poate folosi la determinarea maximului dintre trei numere:

```
MAX(MAX(X,Y), Z)
```

Macrodefinițiile au ca scop reducerea efortului de programare prin substituirea unor construcții ce apar frecvent în program.

Exemplu:

Se va construi macrodefiniția TEST pentru a determina dacă un caracter este literă mică sau cifră între 0 și 5:

```
#define TEST(c) ((c)>='0' &&(c)<='5' || islower(c))
```

În acest fel condițiile devin mult mai simplu de scris:

```
if (TEST(x)) printf ("\n Adev !");
else printf ("\n Fals");
```

Folosirea macrodefinițiilor permite scrierea funcțiilor similare, dar care operează cu variabile de tipuri diferite de dată; o astfel de macrodefiniție lucrează ca un generator de funcții.

Macrodefiniția:

```
# define COMUTA_GEN(TIP) void comuta(TIP& x,TIP& y) \
{   TIP t;   \
    t=x; x=y; y=t; \
}
```

are ca scop definirea unei funcții generice de interschimb a două variabile de tipul generic TIP, prin expandarea sa în mai multe funcții ce diferă prin tipul de date asociat variabilelor. Acest lucru este admis datorită facilității de supraîncărcare a funcțiilor în limbajul C++.

Apelurile:

```
COMUTA_GEN(int);
COMUTA_GEN(float);
```

permit preprocesorului expandarea aceluiași macrou de două ori, având ca rezultat definirea a două funcții de interschimb, cu același nume, una lucrând pe variabile întregi, alta pe flotante.

Macrodefinițiile pot să nu aibă parametri și se comportă ca simboluri simple. De exemplu, macrodefiniția MESAJ nu are nici un parametru și are ca scop afișarea unui mesaj:

```
# define MESAJ { printf ("\n Dați elementele vectorului."); }
```

Parametrii macrodefiniției pot fi prefixați cu simbolurile # sau ##, care au semnificațiile:

- *stringizing operator*, operator de "șirificare", adică de transformare a parametrului macrodefiniției în constantă șir de caractere; în acest sens, textului de substituie furnizat la apelul macrodefiniției i se pune automat ghilimelele de început și sfârșit, iar dacă textul conține caractere ce nu pot fi date decât ca secvențe de escape (de exemplu, ghilimele, apostrof, backslash), acestea primesc automat backslash (\) pentru a deveni secvențe

de escape corecte în interiorul șirului. În plus, dacă șirul de caractere se alătură într-o parte și/sau alta, unui alt șir de caractere, parte fixă a macrodefiniției, se elimină automat ghilimelele de separație, obținându-se un șir unic.

- *merging operator* sau *token pasting*, operator de alipire a parametrului de text fix al macrodefiniției, obținându-se un identificator sau text sursă C, valid.

În macrodefiniția anterioară:

```
# define MESAJ { printf ("n Dați elementele vectorului:"); }
```

ar fi util să se precizeze și numele vectorului pentru care urmează să se introducă elementele. Folosind operatorul **#**, macrodefiniția se modifică astfel:

```
#define MESAJ(X) { printf ("n Dați elementele vectorului"# X":"); }
```

Apelul:

```
MESAJ(vect);
```

are ca efect afișarea pe ecran a mesajului:

Dați elementele vectorului vect:

Dacă într-un program apar expresiile:

```
x1=a1*c1;
```

```
x2=a2*c2;
```

atunci se poate construi următoarea macrodefiniție:

```
#define OP(k) x##k=a##k*c##k
```

care apelată sub forma:

```
OP(1);
```

```
OP(2);
```

va determina generarea unui cod echivalent celor două expresii.

Pentru exemplificare, se va construi programul care va citi de la consolă elementele unui vector și va construi un alt vector având elementele primului, în ordine inversă. Legat de preprocesare se pot urmări următoarele aspecte:

- Pentru a preciza numărul maxim de elemente se va defini constanta simbolică NMAX.
- Se vor construi macrodefinițiile:
 - MESAJ, fără parametri, pentru afișarea unui mesaj pe display;
 - AFIS_EL, cu parametri, care afișează elementul de un anumit rang din vector; pentru a individualiza numele vectorului s-a folosit operatorul de șirificare (**#**);